

分类号 TP302.1

学号

UDC 004.8

密级 公开

硕士学位论文
(专业学位)

面向异构 GPU 集群的多模态大语言模型
并行训练技术研究

硕士生姓名

专业学位类别 电子信息

专业学位领域 计算机技术

研究方向 并行智能计算

指导教师

二〇二六年四月

Research on Parallel Training Techniques for Multimodal Large Language Models in Heterogeneous GPU Clusters

Candidate:

Supervisor:

A dissertation

**Submitted in partial fulfillment of the requirements
for the professional degree of Master of Engineering
in Computer Technology**

April, 2026

目 录

摘 要	i
Abstract	iii
本文中英文对照表	v
第一章 绪论	1
1.1 研究背景与意义	1
1.1.1 多模态大语言模型的训练	1
1.1.2 异构 GPU 集群下的并行训练	4
1.2 国内外研究现状与挑战	6
1.2.1 多模态大语言模型并行训练策略研究现状	6
1.2.2 异构 GPU 集群并行训练策略研究现状	8
1.2.3 研究问题与挑战	9
1.3 研究内容与贡献	10
1.3.1 MMoH: 多模态大语言模型在异构 GPU 集群上的混合并行方法	10
1.3.2 HR-MMoH: 多模态大语言模型在异构 GPU 集群上的重计算方法	11
1.4 论文组织结构	11
第二章 相关工作	13
2.1 多模态大语言模型及并行训练技术	13
2.1.1 多模态大语言模型的模型架构与训练数据	13
2.1.2 多模态大语言模型的训练过程	15
2.1.3 多维混合并行训练方法	16
2.1.4 零冗余优化器 (ZeRO)	20
2.2 异构 GPU 集群并行训练	21
2.2.1 异构 GPU 集群并行训练策略	21
2.2.2 异构 GPU 集群并行训练的工业实践	22
2.3 重计算技术	23
2.3.1 重计算基本原理与分类	23
2.3.2 重计算的主流实现及其局限性	24
2.4 本章小结	25

第三章 多模态大语言模型在异构 GPU 集群上的高效混合并行技术	27
3.1 引言	27
3.1.1 问题与动机	27
3.1.2 MMoH 框架概览	30
3.2 MMoH 并行策略规划器	32
3.2.1 需求驱动的设备拓扑识别	33
3.2.2 复合粒度模型抽象	35
3.2.3 冻结感知的模型划分	36
3.3 提前终止流水线调度策略	38
3.3.1 冗余识别与剪枝规则	39
3.3.2 调度流程与实现细节	40
3.3.3 对成本模型与划分策略的影响	40
3.3.4 实现要点	42
3.4 实验验证	43
3.4.1 实验设置	43
3.4.2 实验结果	44
3.4.3 实验分析	46
3.5 本章小结	48
第四章 多模态大语言模型在异构 GPU 集群上的重计算技术	51
4.1 引言	51
4.1.1 问题与动机	51
4.1.2 HR-MMoH 框架概览	52
4.2 异构 GPU 重计算技术	52
4.2.1 现有的重计算策略与定量分析	52
4.2.2 异构 GPU 重计算策略设计	59
4.3 实验验证	63
4.3.1 实验设置	63
4.3.2 实验结果	64
4.3.3 实验分析	68
4.4 本章小结	68
第五章 总结与展望	71
5.1 全文工作总结	71
5.1.1 主要创新点	71
5.2 局限性及未来工作展望	72
5.2.1 研究局限性	72

5.2.2 未来研究工作	73
致谢	75
参考文献	77
作者在学期间取得的学术成果	89

表 目 录

表 1	本文主要术语中英文对照表	v
表 2.1	多模态大语言模型训练中常用的数据集类型与规模	15
表 3.1	实验用异构 GPU 集群配置	43
表 3.2	实验用 MLLM 模块配置（模型参数）	44
表 3.3	不同冻结场景下的模型划分对比（加速比 Baseline 为 Whale）	47
表 4.1	Megatron-LM 重计算相关配置参数	54
表 4.2	Megatron-LM 选择重计算下可配置的子模块	54
表 4.3	本章主要符号说明	56
表 4.4	单层 Transformer 中间激活量	58

图 目 录

图 1.1 多模态大语言模型的核心组成部分	3
图 1.2 NVIDIA GPU 架构与代表性产品演进	5
图 1.3 使用 Megatron-LM 框架在同构集群上训练多模态大语言模型的典型 并行配置	8
图 2.1 MLLM 多阶段训练中典型模块冻结	16
图 2.2 数据并行示意	17
图 2.3 张量并行示意（左、右分别示意两类典型矩阵切分与通信模式；二 者在注意力子层与前馈子层中均需组合使用）	18
图 2.4 流水线并行示意	18
图 2.5 专家并行中的门控路由与专家分布示意	19
图 2.6 零冗余优化器 ZeRO 示意	20
图 2.7 重计算示意图	24
图 3.1 主流 MLLMs 的编码器—投影层—LLM 结构及冻结方式。	28
图 3.2 LLM 与 MLLM-3B 各层计算开销与显存占用的变化趋势（NF 为不 冻结，FB 为 freeze-both）。	29
图 3.3 不同并行策略在两种冻结场景下训练 MLLM-3B 的迭代时间， MMoH1/MMoH2 为 MMoH 规划器针对各场景得到的策略。	29
图 3.4 MMoH 框架总览。	32
图 3.5 复合粒度抽象示意。（a）编码器：将连续多层合并为编码器层块；（b） LLM Backbone：将单层拆分为注意力块与 FFN 块，不增加流水级间通信量。	35
图 3.6 提前终止调度器效果示意。	41
图 3.7 提前终止在其它流水线调度机制上的适用性。上：Chimera（双向流 水线）；下：Hanayo（波式流水线）。	42
图 3.8 四种冻结场景下 MMoH 与基线方法在两种异构集群上的训练吞吐 对比（samples/seconds）。柱越长表示吞吐量越高，OOM 表示显存溢出。	45
图 3.9 MMoH 与 Whale、Espresso 在 MLLM-12B 上、两种冻结设置下的负 载均衡对比。分布越集中表示划分越均衡，高度越低表示迭代时间越短。	47
图 3.10 提前终止调度器消融。Espresso+ 为接入该调度器的 Espresso，MMoH- 为去掉该调度器的 MMoH。	48
图 4.1 HR-MMoH 框架总览及其与 MMoH 的衔接关系。MMoH 负责划分 与调度；HR-MMoH 在各流水级上配置差异化的重计算策略，以进一步适配 异构 GPU 设备的显存与算力。	53

图 4.2 注意力子层中间激活示意图	55
图 4.3 MLP 子层中间激活示意	57
图 4.4 不同冻结策略下各框架迭代时间对比（自上至下、自左至右依次为 no-freeze、freeze-encoder、freeze-llm、freeze-both）。	64
图 4.5 不同冻结策略下各框架在 none、full、selective 及 HR-MMoH 下的最 短迭代时间对比。	66
图 4.6 不同冻结策略下 MMoH 与 HR-MMoH 的迭代时间对比	67
图 4.7 训练收敛性验证：三种配置下 MLLM-12B 在 Cluster-L freeze-both 场 景的训练损失对比。(a) Espresso + selective（基线），(b) MMoH + 提前终止 + selective，(c) HR-MMoH（提前终止 + 异构重计算）。	68

摘 要

多模态大语言模型（MLLMs）在传统语言模型的基础之上融合了视觉与听觉等多模态能力，成为新的研究热点，然而巨大的参数量和数据量使其训练过程高度依赖分布式并行计算。面向真实场景时，训练中使用的 GPU 集群通常由多代异构的设备组成，显存、算力与互联带宽的代际差异明显，训练系统难以在异构设备间实现稳定高效的资源协同。与此同时，MLLMs 由编码器、投影层与 LLM 基座等异构子模块构成，并普遍采用分阶段冻结训练的训练方式，导致不同训练阶段的计算负载与显存需求差异显著且会动态变化。现有并行训练方法多基于同构 GPU 集群的假设，忽视了异构 GPU 设备之间的性能差异，而且没有联合考虑 MLLMs 的模块结构与分阶段冻结的训练方式，导致在异构 GPU 集群下训练 MLLMs 时容易出现负载不均衡、计算通信冗余以及 GPU 利用率低的问题，进而限制了训练吞吐量。

针对异构 GPU 集群与分阶段冻结训练下并行策略固定、易出现负载不均衡与冗余计算通信的问题，本文提出一种面向异构 GPU 集群的多模态大语言模型高效混合并行方法（MMoH）。该方法通过需求驱动的设备拓扑识别、复合粒度模型抽象与冻结感知模型划分，自动生成面向不同训练阶段的混合并行策略，使并行配置能够随训练阶段动态调整。进一步地，MMoH 通过提前终止调度机制消除连续冻结前缀流水级中的冗余反向计算与通信，降低不必要的计算和通信开销。在两种规模的异构 GPU 集群与两种规模的 MLLMs（3B、12B）上的实验结果表明，MMoH 在多种冻结场景下相较现有训练方法 Megatron-LM、Whale 与 Espresso 均取得性能提升，最高加速约 1.77 倍，验证了该方法在异构 GPU 集群中的有效性。

针对现有全局统一重计算策略在异构 GPU 集群上难以兼顾 GPU 利用率与显存开销的问题，本文在 MMoH 基础上进一步提出一种面向异构 GPU 集群的多模态大语言模型重计算方法（HR-MMoH）。该方法在 MMoH 划分与调度结果基础上，按流水级进行差异化重计算配置，并结合剖析驱动的配置搜索形成“划分—调度—重计算”多级优化方法，从而在不同设备与不同训练阶段间实现更细粒度的计算与显存权衡。在更大批次大小与更长序列长度的端到端实验中，HR-MMoH 在不同冻结场景下均取得最大吞吐，迭代时间相比 Espresso 缩短约 6.1%–23.1%，相对 Whale 的可训练子集缩短约 11.0%–40.6%。上述结果表明，HR-MMoH 能够进一步提升异构 GPU 集群训练 MLLMs 的速度。

关键词：多模态大语言模型；异构 GPU 集群；流水线并行；重计算

Abstract

Training multimodal large language models (MLLMs) on production GPU clusters is a systems challenge: clusters are heterogeneous across generations, while MLLMs are modular and commonly trained with stage-wise freezing. Device heterogeneity (memory capacity, compute throughput, and interconnect bandwidth) and stage-dependent model behavior jointly create time-varying imbalance in computation, memory, and communication. Existing parallel training systems are primarily designed for homogeneous clusters and do not jointly model hardware heterogeneity, module structure, and freeze schedules, which leads to stragglers, redundant communication/computation, and low accelerator utilization in heterogeneous settings.

This thesis presents MMoH, a heterogeneous-aware hybrid parallel method for MLLM training. MMoH combines demand-driven topology identification, compound-granularity model abstraction, and freeze-aware partitioning to generate stage-specific parallel plans and reconfigure execution as training progresses. MMoH also introduces early-termination scheduling, which removes redundant backward computation and communication in consecutively frozen pipeline stages. Across two heterogeneous clusters and two model scales (3B and 12B), MMoH improves performance over Megatron-LM, Whale, and Espresso under multiple freezing scenarios, achieving up to $1.77\times$ speedup.

Building on MMoH, this thesis further presents HR-MMoH, a heterogeneous-aware recomputation method that addresses the inefficiency of globally uniform recomputation on heterogeneous clusters. HR-MMoH applies pipeline-stage-specific recomputation decisions and performs profiling-guided configuration search, yielding multi-level optimization across partitioning, scheduling, and recomputation. In end-to-end runs with larger batch sizes and longer sequences, HR-MMoH delivers the highest throughput across freezing scenarios. Relative to Espresso, HR-MMoH reduces iteration time by 6.1%–23.1%; relative to the trainable subset of Whale, it reduces iteration time by 11.0%–40.6%.

Key Words: Multimodal large language model, Heterogeneous GPU cluster, Pipeline parallelism, Recomputation

本文中英文对照表

表 1 本文主要术语中英文对照表

中文	英文
大语言模型	Large Language Model (LLM)
多模态大语言模型	Multimodal Large Language Model (MLLM)
扩展定律	Scaling Laws
模态编码器	Modality Encoder
输入投影层	Input Projector
LLM 基座	LLM Backbone
输出投影层	Output Projector
模态生成器	Modality Generator
扩散模型	diffusion model
模块异构	module heterogeneity
数据并行	data parallelism (DP)
张量并行	tensor parallelism (TP)
流水线并行	pipeline parallelism (PP)
序列并行	sequence parallelism (SP)
上下文并行	context parallelism (CP)
专家并行	expert parallelism (EP)
混合并行	hybrid parallelism
微批次	micro-batch
一前向一反向调度	One-Forward-One-Backward (1F1B)
全归约	AllReduce
全收集	All-Gather
全对全通信	All-to-All
点对点通信	Peer-to-Peer (P2P)
流水级	pipeline stage
流水线气泡	pipeline bubble
全分片数据并行	Fully Sharded Data Parallel (FSDP)
零冗余优化器	Zero Redundancy Optimizer (ZeRO)
激活 / 激活值	activation

中文	英文
显存溢出	Out Of Memory(OOM)
负载均衡	load balancing
剖析 / 性能剖析	profiling
设备拓扑	device topology
整数线性规划	integer linear programming (ILP)
全部重计算	full recomputation
选择重计算	selective recomputation
细粒度重计算	fine-grained recomputation

第一章 绪论

1.1 研究背景与意义

1.1.1 多模态大语言模型的训练

深度神经网络（Deep Neural Network, DNN）在计算机视觉（Computer Vision, CV）、自然语言处理（Natural Language Processing, NLP）、语音识别（Speech Recognition, SR）等众多前沿领域取得了显著成效，近年来逐步成为诸多关键前沿技术领域的核心基础^[1]。2017 年，Vaswani 等人提出基于自注意力机制（Self-Attention）的 Transformer 架构^[2]，摒弃了传统的循环神经网络（Recurrent Neural Network, RNN）与卷积神经网络（Convolutional Neural Network, CNN）结构，由于其适合并行计算而且擅长捕捉长上下文信息，Transformer 架构迅速成为自然语言处理与跨模态建模等领域的主流模型结构。

在自然语言处理领域，基于 Transformer 的预训练大语言模型（Large Language Models, LLMs）持续刷新多项基准任务的成绩，并推动模型能力由特定任务优化向通用知识学习演进。以 BERT^[3] 为代表的带掩码的双向编码预训练范式在语言理解任务中取得了突出效果，与此同时，GPT-2^[4]、T5^[5] 所采用的自回归（Autoregressive）与编码器 - 解码器（Encoder-Decoder）机制在文本生成场景中展现出更强的能力。随着模型规模进一步扩展，GPT-3^[6] 及后续 GPT-4^[7] 在少样本（Few-Shot）与零样本（Zero-Shot）条件下呈现出极强的语言表达与任务迁移能力。

在计算机视觉领域，Transformer 也取代了传统的卷积神经网络，成为视觉模型的主流选择。Vision Transformer（ViT）^[8] 及其规模化扩展工作^[9] 表明，图像经块划分后输入 Transformer 层进行特征提取，可在 ImageNet 等基准数据集上达到并在部分配置下超越传统卷积神经网络的性能。由此可见，Transformer 架构具备跨模态扩展的能力，其适用范围并不局限于文本建模。

随着单模态模型已经接近甚至超越人类水平，将语言、视觉、音频、表格等多模态信息纳入统一的模型框架进行联合表示与计算，成为当前学术界和工业界研究的热点，这种可以同时处理两种或两种以上模态信息的能力被称为多模态（Multimodality）。在机器学习与人工智能中，常见模态包括文本、图像、视频、音频、表格以及各类传感器数据，这种多种模态的信息更接近人类接触到的复杂信息。多模态大语言模型（Multimodal Large Language Models, MLLMs）是指在原有大语言模型的基础上，增加对图像、声音等其它模态的数据的感知与生成能力的一类模型^[10-12]。MLLMs 的先驱工作 CLIP^[13]（Contrastive Language-Image

Pre-Training) 通过使用图像文本对进行训练, 使模型拥有理解并描述图片的能力, DALL-E^[14] 则与 CLIP 相反, 展示了从文本生成图像的可行性。

近年来, MLLMs 在架构与训练范式上发展迅速。BLIP-2^[10] 提出 Q-Former 连接冻结的视觉编码器模型与 LLM 主体进行训练; InstructBLIP^[15] 在此基础上加强了视觉信息的提取能力, 在多项零样本任务上取得 SOTA 结果; LLaVA^[11] 与 LLaVA-1.5^[16] 采用简单线性投影层实现视觉与语言模态对齐, 验证了轻量连接器与大语言模型组合方案的有效性; Qwen-VL^[17]、InternVL^[18] 等系列多模态大语言模型则进一步在长上下文、高分辨率与多图理解上进一步扩展了模型能力。这类模型既保留传统 LLMs 在语言理解、推理与生成上的强大能力, 又能够直接处理图像、音频等多模态输入, 实现跨模态交互与推理任务, 因而被广泛认为是通往通用人工智能 (Artificial General Intelligence, AGI) 的最有前景的技术手段^[19, 20]。

从功能表现来看, MLLMs 可完成图像描述、视觉问答、图文对话、文档理解, 以及基于文本的图像或视频生成等复杂任务, 在诸多前沿领域例如自动驾驶、智能助手等场景具有极大应用价值。在垂直领域, MLLMs 已延伸至医疗影像理解^[21] 等。多模态大语言模型的相关评估常使用 VQAv2^[22]、GQA^[23]、MMMUI^[24] 等多模态基准, 不断更新的基准测试也对模型规模与训练数据提出了更高要求。从结构上看, MLLMs 通常包含以下核心组件, 如图 1.1 所示: (1) Modality Encoder, 负责将图像、视频等非文本输入编码为特征表示, 常采用预训练的视觉模型 (如 CLIP、ViT); (2) Input Projector, 负责将编码后的多模态特征映射到与 LLM 词嵌入对齐的语义空间, 常见形式包括线性层、Q-Former^[10] 等轻量级小模块; (3) LLM Backbone, 即大语言模型主体, 承担主要的内容理解与生成任务, 处于 MLLMs 架构的核心地位, 多模态能力通常是在其基础上的扩展与增强; (4) Output Projector (在需要多模态生成时), 将 LLM 输出映射到与目标模态生成器对齐的表示空间; (5) Modality Generator (在需要多模态生成时), 负责生成图像、音频等目标模态, 常采用扩散模型 (如 Stable Diffusion) 等。

多模态大语言模型能力的拓展既依赖上述编码器、投影层、基座模型与可选生成器等模块的协同设计, 也依赖参数与数据规模的提升。参数和数据规模决定了模型可达的效果上限, 而多模块的协同设计则影响模型的训练过程。在大语言模型占据主体参数量的架构下, 针对 LLMs 总结出的扩展定律 (Scaling Laws) 与数据规模规律, 同样深刻影响着 MLLMs 训练的资源需求。已有实证研究表明, 模型性能通常会随参数规模与训练数据规模的增加而稳定提升, 这一现象被总结为 “扩展定律” (Scaling Laws)^[25]。Kaplan 等^[25] 的早期工作表明, 在给定算力预算下, 模型规模与数据规模应按幂律关系共同扩展。Hoffmann 等^[26] 进一步指出, 当前大语言模型普遍 “训练不足”, 在计算最优意义下, 训练需要的词元 (token)

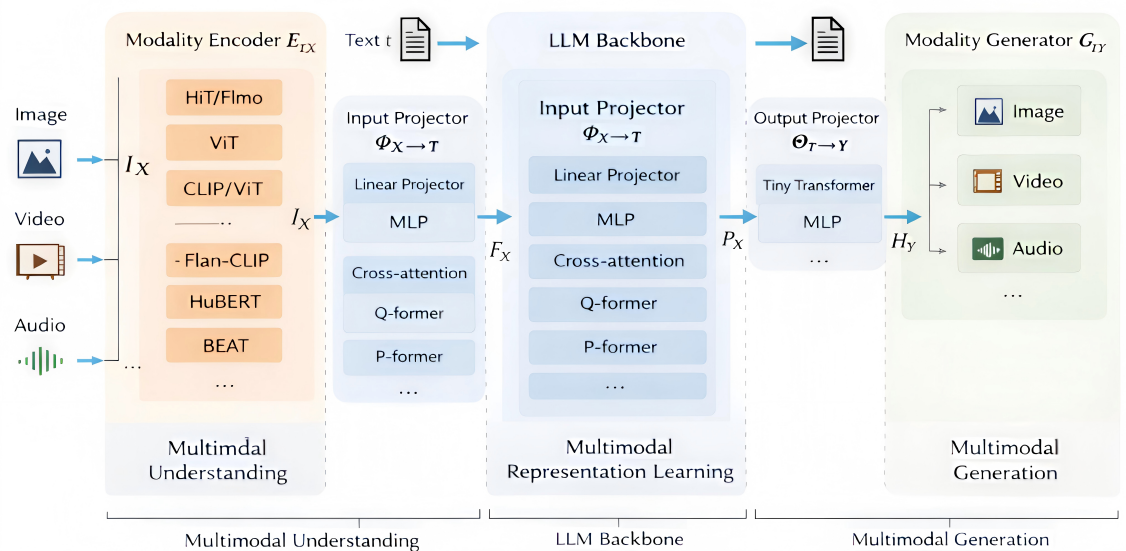


图 1.1 多模态大语言模型的核心组成部分

总数应该与模型参数量近似线性增加，该结论对预训练数据规模与模型架构选择产生了深远影响。Sardana 等^[27] 则从推理成本角度拓展了 Chinchilla 最优，指出在考虑部署与推理时，最优扩展策略会随推理请求规模发生变化。为追求更强性能，工业界与学术界不断推出参数量更大、数据规模更大的模型，从数亿参数（如 BERT）到千亿、万亿参数（如 GPT-3、Gopher^[28]、DeepSeek-V3^[29]、Qwen2.5^[30]、LLaMA 2/3^[31, 32]、Gemma^[33]、Mistral^[34] 等开源与闭源 LLMs），单卡或单机 GPU 的显存与算力已无法容纳整模型的完整训练流程^[35, 36]。相应地，多模态大语言模型的预训练与指令微调所需的数据量也急剧增长，从数百万图文对到数十亿级样本，单机的运算速度已经远远无法满足企业对于模型迭代速度的需求。不仅如此，长上下文与多模态联合基准^[37] 的普及进一步推高了对训练规模与效率的需求。在上述模型与数据规模的约束下，单个计算设备的显存与算力均难以独立支撑完整的训练流程，因此必须借助分布式并行训练技术，将模型参数、优化器状态、激活值与数据分散至多台设备，通过节点间的通信协同完成多模态大语言模型的训练^[38, 39]。

目前，支撑大规模 MLLMs 训练的主流手段包括数据并行、张量并行、流水线并行、序列与专家并行等多维混合并行技术^[35, 36, 40–43]，以及面向参数与优化器状态的 ZeRO 系列显存分片技术^[38, 39]；上述策略常与混合精度训练^[44]、Flash Attention^[46, 47] 等相结合，共同构成当前大规模训练的基础设施。在采用上述并行策略的同时，训练过程仍面对较大的显存压力。前向传播中需要保存中间激活以供反向传播使用，其显存占用随序列长度近似线性增长，在单设备上常构成主要显存压力，从而限制模型训练的上下文长度，进而限制模型性能上限。ZeRO 等分

片技术主要缓解参数与优化器状态的显存占用，对单设备上激活占用的降低有限。为此，重计算（亦称梯度检查点）通过在前向传播中丢弃中间激活、在反向传播时重算，以额外前向计算换取显存空间，使模型在相同硬件条件下能够采用更大批次或更长序列进行训练^[71, 115]。由于如今的模型训练上下文越来越长，重计算成为当前大规模训练的显存优化关键技术，因此 Megatron-LM、DeepSpeed 等主流并行训练框架均支持重计算技术。

然而，上述策略在设计时多假设集群同构且模型是单一 Transformer 层堆叠的语言模型，其划分粒度、负载均衡与重计算策略均围绕“各模型层计算与显存相对均匀”这一前提展开。对于 MLLMs 而言，会引入两个额外问题：其一是模型结构异构，MLLMs 各子模块在算子类型、参数量与激活体积上差异显著。编码器以卷积或 ViT 块为主，投影层参数量较小，而 LLM 基座由大量同构 Transformer 层堆叠且占据绝大部分参数。这种异构性使得传统基于“按层均分”假设的模型划分方案难以适用。其二是分阶段冻结导致的动态负载变化。当某一模块被冻结时，其反向传播计算开销与优化器状态的显存开销显著下降，各模块的计算与显存占比随阶段切换而动态变化。例如，在冻结编码器阶段，编码器侧的显存与计算负载骤减，而 LLM 基座承担大部分梯度计算，切换到冻结 LLM 阶段后，负载分布又会发生变化。这种动态变化特性使得固定的模型划分配置难以在全训练流程中保持高效。

1.1.2 异构 GPU 集群下的并行训练

多模态大语言模型的训练效率与成本高度依赖底层硬件平台。目前，工业界与学术界普遍采用 NVIDIA GPU 作为 MLLMs 训练的主要加速器，并通过多卡、多机集群进行分布式训练。人工智能的快速发展也推动了硬件迭代的速度，大约每 1–2 年，NVIDIA 就会使用新的计算架构并推出多款不同定位的 GPU，这给 MLLMs 的训练带来了新的挑战。

图 1.2 描述了近年来 NVIDIA GPU 的架构演进。2016–2017 年的 Pascal 计算架构（如 NVIDIA P100）与 Volta 计算架构（NVIDIA V100）奠定了数据中心 GPU 与 Tensor Core 的基础。2018 年 Turing 架构（RTX 20 系）引入消费级光追与 INT8 推理。2020 年 Ampere 架构推出 A100（40GB/80GB HBM2e）与消费级 RTX 30 系列（如 RTX 3090 24GB），支持 PCIe 4.0 与 NVLink 3.0，成为当前许多实验室与云平台的主力^[48]。2022 年搭载 Hopper 架构的 H100（80GB/96GB HBM3）针对大语言模型训练做了专门优化^[49]，并广泛配备 NVLink 4.0 与 NVSwitch，单机多卡带宽可达数百 GB/s，同年 Ada Lovelace 架构的 RTX 40 系列（如 RTX 4090 24GB）在消费级市场提供高性价比算力。2024 年起，Blackwell 架构的 B100/B200 等数据

中心 GPU 逐步量产，采用 HBM3e、更高带宽第五代 NVLink 与 PCIe 6.0，进一步拉大与旧代设备的性能与互联差距。据 NVIDIA 在 GTC 等场合公开发布的产品路线图^[50]，后续还将推出 Rubin 架构（约 2026 年）、Rubin Ultra（约 2027 年），以及面向推理与能效深度优化的 Feynman 架构（约 2028 年）。与此同时，不同代际与型号在 PCIe 版本（3.0/4.0/5.0/6.0）、是否配备 NVLink/NVSwitch、以及 InfiniBand 与 RoCE 等节点间网络上的不同，也会带来通信带宽与延迟的代际差异。

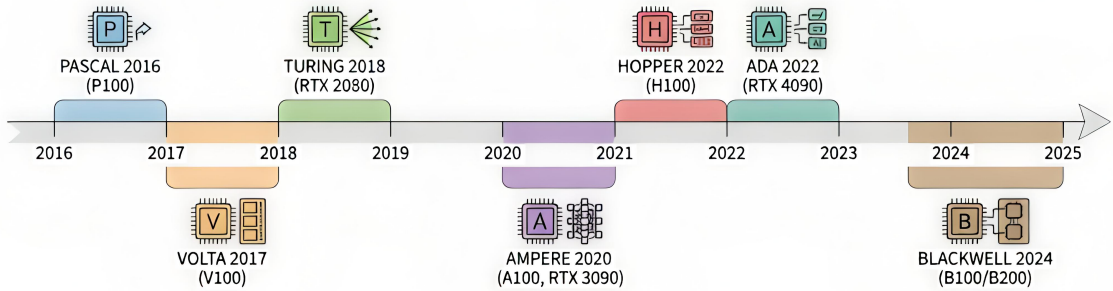


图 1.2 NVIDIA GPU 架构与代表性产品演进

在大多数高校、研究所与中小企业的实际环境中，受预算与采购周期限制，GPU 集群往往经历多次分批采购，而旧设备报废周期较长，集群中常同时包含多代 GPU（例如部分 A100 或 H100 与大量 RTX 3090/4090、甚至更早的 V100/RTX 2080），单机内也可能混用不同显存与互联配置，难以做到“全集群同构”，由此形成显存异构与算力异构并存的局面^[51, 52]。然而，主流并行训练框架（如 Megatron-LM、DeepSpeed）在设计时多假设同构设备与统一拓扑^[35, 38]，在异构集群上直接使用容易出现以下问题：负载无法按算力与显存合理分配，低显存设备成为瓶颈而高端卡显存资源利用率低，以及计算最慢的节点拖慢整体迭代速度^[51, 52]。对于单层计算与显存相对均匀的传统大语言模型，尚可通过“按层数或算力比例”划分在一定程度上缓解异构带来的影响。但是对于 MLLMs，其模块间在层类型、参数量与激活体积上的固有差异与设备能力差异相互叠加，使得简单的模型划分难以负载均衡。因此，在显存与算力异构的 GPU 集群上高效训练 MLLMs，是学术界和工业界共同面临且亟待解决的问题。

从系统层面看，异构带来的问题不仅在于单卡能力差异，还在于拓扑与调度。节点内 NVLink 与节点间 InfiniBand 的带宽可相差一个数量级以上，若流水线并行或张量并行的划分未考虑链路差异，跨节点通信极易成为瓶颈。同时，MLLMs 各训练阶段对编码器、投影层与 LLM 基座的冻结策略不同，导致同一模型在不同训练阶段下各模块的计算与显存开销发生明显变化。若沿用固定的模型划分与调度，难以在整轮训练中保持负载均衡。即便划分与调度已得到优化，在异构流水级上

采用全局统一的重计算配置仍难以同时满足各设备的显存约束。因此，面向异构集群的 MLLMs 训练需要同时考虑硬件拓扑、模型结构、多训练阶段以及流水级差异化的显存—计算权衡，通过自适应的划分、高效的调度与按流水级配置的重计算策略协同提升训练效率。

综上所述，硬件层面的设备异构与模型层面的模块异构、分阶段冻结相互交织，使得异构集群上的 MLLMs 训练同时面临负载分配、算存资源利用、显存—计算权衡等多维约束，构成了当前有限资源下高效训练的核心难点。随着云上与混合部署中弹性训练与按需资源供给的普及，集群内设备型号与拓扑将更加多样，这对异构感知的划分、调度与重计算配置提出了更高要求^[53–55]。为此，本文基于 Megatron-LM 主流并行训练框架，针对 MLLMs 的模块结构异构与分阶段冻结特性，结合异构 GPU 集群中显存与算力的多维差异，研究细粒度的混合并行划分与重计算优化技术，旨在提出一套面向异构 GPU 环境的自适应并行训练方案，以提升异构 GPU 集群下 MLLMs 的训练效率。

1.2 国内外研究现状与挑战

本节从文献角度归纳与本文密切相关的研究脉络，先概述多模态大语言模型并行训练策略研究现状及其同构假设的局限，再综述面向异构 GPU 集群的并行训练工作，最后归纳重计算相关研究及其在异构 GPU 场景下的不足。

1.2.1 多模态大语言模型并行训练策略研究现状

Kaplan 等^[25]通过系统实验提出扩展定律（Scaling Law），指出模型性能会随参数量与训练数据量的增加而提升；Chinchilla^[26]进一步从计算最优角度表明参数规模与训练词元（token）数应近似同比例扩展，后续研究^[27]又将该缩放规律拓展到推理成本约束下的最优配置。基于上述研究，多模态大语言模型普遍沿着“更大参数规模 + 更大数据规模”的路径持续演进。

自 Transformer^[2]提出以来，语言模型规模呈持续扩张趋势，参数量由 BERT^[3]的亿级逐步发展到 GPT-2^[4]、T5^[5]的十亿级，并进一步扩展至 GPT-3^[6]、GPT-4^[7]以及 DeepSeek-V3^[29]、Qwen2.5^[30]等模型所代表的千亿级规模。在此基础上，MLLMs 通过引入视觉编码器并与语言模型进行跨模态对齐，形成了以 BLIP-2^[10]、InstructBLIP^[15]、LLaVA/LLaVA-1.5^[11, 16]、Flamingo^[12]、Qwen-VL^[17]、InternVL^[18]和 MiniGPT-4^[19]等为代表的技术谱系，其模型规模已覆盖数十亿至数千亿参数，训练数据规模也由百万级图文对扩展至十亿级样本（如 LAION-5B^[59]）。相关工作已从模型架构、跨模态对齐策略、数据构建与系统效率等维度对该领域进行了系统总结^[20, 60, 61]。单卡或单机在显存容量和计算吞吐上已难以支撑完整训练流

程^[35, 36]，并行与分布式训练因此成为大模型与 MLLMs 训练的基础技术路径^[38, 39]。

除了模型的参数规模，训练数据的规模同样持续扩大。图像方面，Open Images^[62]、COCO^[63] 等海量图片数据集需 TB 级存储，LAION-5B 等开放图文对达数十亿样本。文本方面，Common Crawl、The Pile^[64] 等为 LLM 预训练提供数百 GB 至数万亿词元量级的语料。视频方面，YouTube-8M^[65]、WebVid^[66] 等基准需 PB 级存储与高吞吐数据管线，多模态理解与推理则依赖 VQAv2^[22]、TextVQA^[67] 等数据集与长上下文基准^[37]。在单机无法存储完整数据集与模型的前提下，并行训练成为必然选择。

在并行策略方面，工业界与学术界在同构 GPU 集群上已形成以模型与数据切分为核心的多维混合并行，以及以状态分片为核心的 ZeRO 系列两大技术路线，并被 Megatron-LM、DeepSpeed 等并行训练框架广泛应用于千亿级参数 MLLMs 训练^[35, 38, 39]。多维混合并行通过在不同维度上划分模型或数据以降低单卡负载：数据并行^[40, 70] 在各设备组上复制完整模型、按批次划分样本，通信集中在梯度 AllReduce，实现简单且扩展性好；张量并行^[35, 36] 将单层内矩阵按行或列切分到多卡，降低单卡显存，是千亿级稠密模型的主流选择；流水线并行将模型按层划分为若干流水级并分配到不同设备，有效支持超大规模模型的并行训练，但需借助更精细的调度以缓解流水线气泡、提高 GPU 利用率。此外，序列并行^[35]、专家并行^[43] 等可与张量、流水线并行组合，以进一步扩展训练规模。NVIDIA 的 Megatron-LM^[35, 36] 联合使用张量、流水线与数据并行，已支撑在数千张同构 GPU 上训练千亿至万亿级参数的大语言模型。零冗余优化器 ZeRO^[38, 39] 在数据并行的基础上将优化器状态、梯度与参数分片存于各卡、按需聚合，显著降低单卡显存，但引入较多卡间通信；ZeRO-Infinity^[39] 进一步将部分状态卸载至 CPU 或 NVMe；ZeRO++^[72] 则通过量化与通信优化削减跨节点开销。

面向 MLLMs 的模块结构，DistMM^[80] 针对子模块差异组合张量并行与数据并行，DistTrain^[73] 通过解耦模态编码器、LLM 基座与模态生成器的训练流程分别分配资源且配置并行策略，在同构集群上提高了训练 MLLMs 时集群的利用率。在显存优化方面，Korthikanti 等^[71] 在 Megatron-LM 中提出序列并行与选择重计算，与张量并行协同以降低激活显存；Colossal-Auto^[45]、Mist^[117]、AdaPipe^[105] 等工作进一步将重计算与自动并行或流水线划分联合搜索。

然而，主流训练框架在设计时多假设同构 GPU 集群与相同 Transformer 层堆叠的语言模型，对 MLLMs 的模块异构（编码器、投影层、LLM、生成器结构各异）与分阶段冻结训练考虑不足。以 Megatron-LM 为例，其同构集群上训练 MLLMs 时，编码器与生成器的并行方式通常被动跟随 LLM 基座，缺乏面向各子模块算力与显存需求的细粒度映射，典型策略如图 1.3 所示^[73]。在同构集群上直

接套用上述策略已易出现负载不均与显存瓶颈；当集群内设备能力趋于一致而模型模块差异显著时，问题尤为突出。

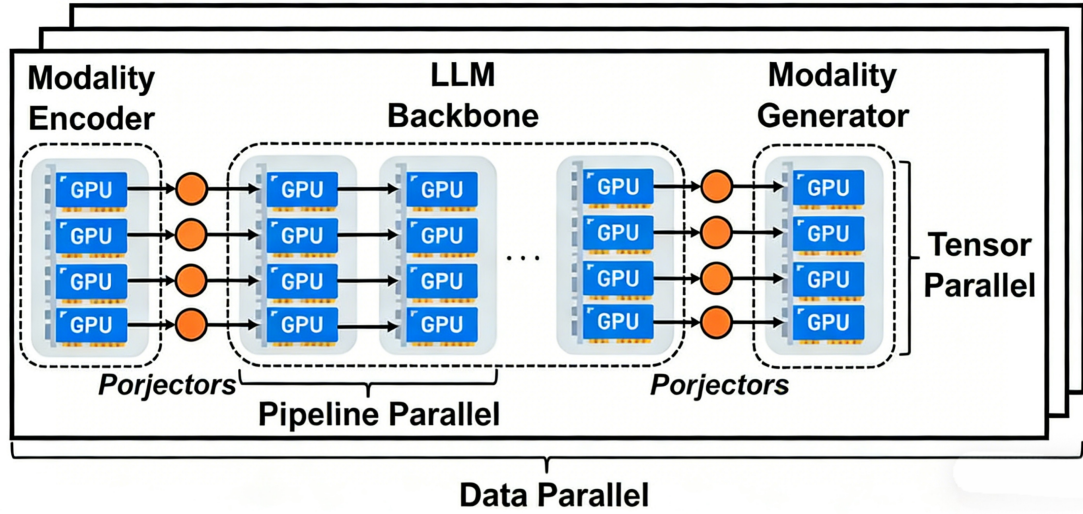


图 1.3 使用 Megatron-LM 框架在同构集群上训练多模态大语言模型的典型并行配置

1.2.2 异构 GPU 集群并行训练策略研究现状

受预算与采购周期限制，实际训练集群往往由多代、多型号 GPU 混合组成，呈现显著的显存与算力异构^[51, 52]。将上一小节所述面向同构集群的并行与重计算策略直接用于此类环境时，往往把不同显存容量、算力与互联带宽的设备当作同构设备统一配置，低显存或低算力节点易成为瓶颈，高端设备资源利用率不足，整体训练效率下降^[51, 52]。针对上述问题，学术界已开展异构 GPU 感知的并行规划与资源调度研究。

早期工作如 HetPipe^[52] 将流水线并行与数据并行结合，借助虚拟工作节点（VW）在异构设备间分配负载，但未显式建模设备排列与通信拓扑，也未针对 MLLMs 的模块异构与分阶段冻结开展联合设计。Whale^[51] 从显存与算力差异出发进行硬件感知负载均衡，按异构 GPU 算力比例划分模型与数据批次，但其划分主要围绕设备算力展开，对模型模块异构带来的计算与显存差异覆盖仍不充分。AMP^[74] 在异构环境下自动搜索并行策略，综合考虑模型异构与设备异构，通过开销模型与动态规划得到近似最优配置；然而其划分粒度偏粗，通信与显存估计精度不足，难以支撑更细粒度的高效映射。

近年来，面向异构 GPU 集群的分布式训练系统在“异构感知并行规划—资源编排—代价建模”等方向上持续演进：HetHub^[75] 统一规划通信器、性能预测器与自动并行以适配多厂商 GPU；HexiScale^[76] 将多种并行划分形式化，并通过层次图划分在异构环境中分配计算；Espresso^[77] 联合优化异构资源供给与流水级放置；

Metis^[78] 以异构感知搜索实现快速自动并行规划；HAP^[79] 将 SPMD 并行优化建模为程序合成问题，联合搜索张量分片与通信方式。总体而言，上述异构感知工作多数仍偏向通用语言模型，或对设备异构、模块异构、分阶段冻结仅做单一维度优化，尚未在同一框架内系统建模 MLLMs 训练中三者的耦合关系，也较少处理冻结阶段切换在异构流水级上引发的动态负载变化。

在异构流水级上，上一小节所述面向同构集群的全局统一重计算配置适用性进一步下降：显存紧张的流水级若配置偏保守，仍可能因中间激活过大而 OOM；显存充裕的流水级则被强制承担同等强度的重计算，引入冗余前向计算并延长迭代时间。在 MLLMs 模块异构与分阶段冻结并存的场景下，如何在与划分、调度协同的前提下按流水级差异化配置重计算，仍缺乏系统化研究。

除多维混合并行外，以 ZeRO 为代表的状态分片路线在异构 GPU 集群上训练 MLLMs 时亦面临局限：ZeRO^[38, 39] 引入大量 All-Gather/All-Reduce 通信，在多节点集群中易成为瓶颈；MiCS^[81] 难以结合设备算存差异与冻结阶段切换实现细粒度、动态的负载自适应；ZeRO++^[72] 的量化与通信优化可能带来精度与通用性代价；AMSP^[82] 等侧重跨节点通信削减，对集群内存储、算力与带宽的细粒度异构利用仍有限。因此，在异构 GPU 集群上高效训练 MLLMs，仍需在 Megatron-LM、DeepSpeed 等现有框架基础上，引入对设备能力、模型模块结构与训练阶段的联合感知，实现划分、调度与重计算的协同自适应优化。

1.2.3 研究问题与挑战

围绕异构 GPU 集群上的多模态大语言模型训练，本文聚焦的核心研究问题是：在设备显存、算力与互联带宽不一致的真实异构 GPU 集群环境下，如何针对 MLLMs 的模块异构结构与分阶段冻结训练特性，联合优化模型划分、流水线调度与重计算配置，在满足显存约束与收敛正确性的前提下最小化迭代时间，提升端到端训练速度。

为回答这一问题，本文面临以下三项关键挑战。第一，模块异构与设备异构耦合导致负载失衡。MLLMs 由编码器、投影层与 LLM Backbone 等多个模块构成，不同模块在计算量与显存占用上差异显著。当其与异构 GPU 的算存能力差异叠加时，传统按层均分或按单一硬件指标映射的方法容易形成瓶颈流水级，导致整体吞吐受限。第二，冻结阶段切换使最优并行策略动态变化。在不同冻结策略中，可训练参数集合动态变化，反向计算与梯度同步开销随之动态变化，静态划分与固定调度难以在全流程保持近似最优。第三，统一的重计算策略难以兼顾显存安全与训练效率。在使用更大批次与更长序列训练时，部分流水级存在显存瓶颈并可能发生 OOM。若对所有设备采用统一的重计算配置，则容易在显存充裕流水级

引入冗余重计算，延长关键路径，降低端到端训练的效率。

基于上述挑战，本文的研究目标是构建一套面向异构 GPU 集群上多模态大语言模型训练的联合优化框架：在模型划分层面考虑异构 GPU 设备与冻结导致的模型算存需求的变化，在调度层面利用冻结信息剪枝冗余反向与通信，在重计算策略层面按流水级配置差异化重计算策略，从而形成“划分—调度—重计算”多级优化框架。具体而言，挑战一主要对应框架的划分层，侧重模块异构与设备异构耦合下的负载均衡；挑战二则要求划分与调度随冻结状态协同调整，并消除冻结场景下的冗余反向与通信，对应框架的划分层与调度层，由第三章 MMoH 的并行策略规划器与提前终止调度器共同解决；挑战三主要对应框架的重计算层，由第四章 HR-MMoH 在 MMoH 基础之上按流水级进行差异化重计算配置，以支撑更大批次与更长序列训练并进一步压缩迭代时间。

1.3 研究内容与贡献

围绕异构 GPU 集群上多模态大语言模型的高效训练这一总体目标，本文将研究工作分解为两个层次递进、相互衔接的子问题。第一，划分与调度子问题：在设备显存与算力异构、且训练过程伴随分阶段冻结切换的条件下，如何自动生成面向多模态大语言模型的细粒度模型划分、设备映射与流水线调度方案，使各流水级计算负载与异构设备能力相匹配；如何随各训练阶段的冻结状态生成相应的划分与映射配置，并在流水线执行中剪枝连续冻结流水级的冗余反向计算与级间通信，以提升 GPU 利用率与训练吞吐。第二，重计算子问题：在第一个子问题所确定的并行执行骨架（设备拓扑、模型划分与冻结感知调度）之上，当训练规模向更大批次与更长序列扩展时，如何按流水级配置差异化的重计算策略，在各设备显存约束下尽可能降低重计算引入的额外计算开销，进一步缩短迭代时间。

针对上述子问题，本文依次提出 MMoH 与 HR-MMoH 两种方法，分别承担划分—调度与重计算两个层次的优化任务，研究内容与贡献如下。

1.3.1 MMoH：多模态大语言模型在异构 GPU 集群上的混合并行方法

针对异构 GPU 环境中显存与算力差异导致的设备利用率低、模块结构差异引发的模型划分困难，以及分阶段冻结训练造成的计算与显存需求动态变化等问题，本文提出一种面向异构 GPU 集群的多模态大语言模型混合并行训练方法。该方法通过为不同计算设备分配不同规模的模型分片，实现对计算资源与显存空间的协同利用，从而提升训练吞吐并缩短迭代时间。进一步地，为适应多阶段冻结训练过程，该方法能够感知各阶段冻结状态并与并行划分方案协同调整，同时在连续冻结的流水级上省略不必要的反向计算与级间通信，在保证模型收敛性的前提下

进一步提升异构设备的有效利用率与整体训练效率。MMoH 承担本文两级方法中的划分与调度层，其输出包括设备拓扑、模型映射及冻结感知的流水线调度策略，构成后续重计算优化的优化对象与约束条件。

1.3.2 HR-MMoH: 多模态大语言模型在异构 GPU 集群上的重计算方法

针对现有重计算策略多采用全局统一配置、未考虑异构 GPU 集群中设备显存与算力差异的问题，本文提出一种面向异构设备与多模态大语言模型的重计算优化方法。HR-MMoH 以 MMoH 的划分与调度结果为输入，将重计算决策粒度由全局统一下沉至流水级差异化：以各设备显存上限为约束，对显存受限流水级采用较强的重计算以降低激活驻留，对显存充裕流水级则减少冗余重算，从而在满足显存约束的前提下最小化重计算引入的额外计算开销。HR-MMoH 不改变 MMoH 的划分与调度策略，而是在其基础之上与之共同构成“划分—调度—重计算”多级优化链路，进一步提升异构 GPU 集群上 MLLMs 的端到端训练效率。

1.4 论文组织结构

本文共分为以下五个章节，具体结构如下：

第一章为绪论。首先从多模态大语言模型的训练需求与异构 GPU 集群的现状两个方面阐述研究背景与意义，随后分别从多模态大语言模型并行训练和异构 GPU 集群并行训练两个维度梳理国内外研究现状，最后介绍本文的研究内容与主要贡献。

第二章为相关工作。首先介绍多模态大语言模型的模型架构、训练过程以及数据并行、模型并行与 ZeRO 等主流并行训练技术，其次综述异构 GPU 集群上的并行训练策略与工业实践，最后阐述重计算技术的基本原理、分类及主流实现的局限性。

第三章介绍本文提出的 MMoH 框架，重点阐述需求驱动的设备拓扑识别、复合粒度模型抽象、冻结感知划分及提前终止流水线调度方法，并给出实验评估。

第四章介绍 HR-MMoH 异构重计算优化，详细说明如何在异构 GPU 集群与不同模型模块间实现重计算策略的细粒度自适应配置并进行实验验证。

第五章为总结与展望。概括本文主要创新点，归纳本文的研究局限，并对后续研究问题与思路作简要展望。

第二章 相关工作

本章围绕多模态大语言模型训练中的并行训练与显存优化问题，对相关工作进行综述，具体包括以下三个方面。（1）首先介绍多模态大语言模型的模型结构与分阶段冻结训练范式，并梳理数据并行、张量并行、流水线并行、序列并行、专家并行及 ZeRO 等主流并行训练方法；（2）其次综述异构 GPU 集群上的模型并行训练研究进展；（3）最后从重计算的基本思想出发，归纳重计算策略及典型框架实现，并分析同构集群下全局统一重计算配置的局限性。

2.1 多模态大语言模型及并行训练技术

2.1.1 多模态大语言模型的模型架构与训练数据

多模态大语言模型（MLLMs）将视觉、语言与生成能力（可选）统一到以大语言模型为核心的模型架构中，其典型结构如图 1.1 所示。

Modality Encoder 即模态编码器，负责将图像、视频、音频等非文本输入编码为高维特征表示，以便与语言模型的词嵌入空间统一。实践中多采用在大规模图片文本对或视频文本对等多模态数据集上预训练好的视觉模型作为编码器。CLIP^[13] 通过对比学习让模型可以理解视觉信息，被 LLaVA^[11]、MiniGPT-4^[19] 等广泛采用。ViT^[8] 及其更大规模的变体^[9] 则将图像切分为多个小的碎片（patch）后经 Transformer 编码，让其它模态信息的处理也使用了与语言模型相同的架构，在 Flamingo^[12] 等工作中用作视觉编码器。InternVL^[18] 将视觉基础模型规模扩展至 60 亿参数，并在多图、长文本与文档理解等任务上展现出极强的能力。Qwen-VL^[17] 是阿里巴巴提出的代表性多模态模型之一，支持百万像素级高分辨率与多轮对话，在 DocVQA^[83]、ChartQA^[84] 等基准上表现突出。模态编码器的输出通常为序列形式的特征，再经后续模块映射到大语言模型的语义空间，从而完成视觉等其它模态数据与语言模型对齐的第一步^[85]。

Input Projector 将编码器输出的多模态特征映射到与 LLMs 词嵌入空间维度对齐的表示，使 LLMs 能够将视觉等其它模态的信息与文本信息统一处理。常见实现包括线性层、轻量 Transformer 层以及 BLIP-2^[10] 提出的 Q-Former（通过可学习的 query 从编码器特征中抽取与任务相关的表示）。InstructBLIP^[15] 在此基础上引入指令感知的 Q-Former，根据用户指令从图像中抽取与任务更相关的特征，在 ScienceQA^[86] 等零样本任务上显著优于 BLIP-2。该模块参数量远小于 LLM，常与编码器一起在预训练或对齐阶段进行参数更新。

LLM Backbone 即基座大语言模型，承担主要的语言理解与生成任务，参数量

通常占整个 MLLM 的 70%–80%^[10, 11]。基座模型多选用已在海量文本上预训练好的 Transformer 模型，如 GPT 系列^[4, 6]、LLaMA 系列^[32] 等。在资源受限或强调高效微调时，常对 LLMs 进行冻结训练或仅微调部分层来减少对于计算资源的需求。

Output Projector 与 Modality Generator 用于更加复杂的多模态生成任务（如图生图、文生图、文生视频、文生音频等）：前者将 LLMs 的输出映射到模态生成器可接受的表示空间，后者根据该表示生成图像、视频或音频。生成器多采用扩散模型，如 Stable Diffusion^[87] 中基于 U-Net^[88] 的潜在扩散结构。Next-GPT^[89]、DreamLLM^[90] 等工作在此基础上实现了任意模态到任意模态的生成与转换，进一步拓展了多模态技术的应用领域。

在训练策略方面，BLIP-2^[10] 率先系统性地提出“冻结 Encoder + 冻结 LLM + 仅训练轻量 Projector”的范式，从而在显著降低训练成本的同时获得与全参数微调可比的效果，后续多数 MLLMs 的训练均沿用或在此基础上演进。该范式的关键问题在于 MLLMs 同时呈现模块异构与分阶段冻结的耦合效应。模块异构使得编码器、投影层、LLM Backbone 与生成器在算子类型、激活规模、参数量与反向计算特性上存在显著差异，而冻结策略则在训练过程中持续切换各模块的可训练集合，从机制上动态改变反向计算图与显存占用分布。因此，同一模型结构在不同训练阶段往往对应不同的计算 / 显存需求，这会使流水线并行中“阶段性负载变化”被转化为流水级间的不均衡负载，并进一步在异构 GPU 集群上放大这种错配。设备之间本就存在显存容量与算力吞吐差异，若并行划分与调度仍采用固定模型映射，将导致整个训练系统受限于瓶颈设备，降低端到端吞吐。综上所述，为实现高效并行与稳定收敛，面向 MLLMs 的划分与调度必须同时细粒度考虑“模块差异 + 冻结状态”，并让并行配置能够随训练阶段自适应调整，而不能简单沿用单一 Transformer 场景下的静态划分方式。

在数据与模型规模方面，Kaplan 与 Chinchilla 等^[25–27] 关于扩展定律与计算最优配比的讨论已在第一章给出，此处不再展开。大语言模型通常在海量文本语料（如 Common Crawl、The Pile^[64]、书籍与代码）上预训练，规模可达数万亿 token；而 MLLMs 在此基础上还需要大规模图文或视频—文本配对数据，以支撑对齐与生成等训练阶段。多模态评估基准（如 MMMU^[24]）在任务类型与数据形态上的持续扩展^[61]，也进一步推动了训练数据在模态与规模上的增长。

表 2.1 列举了代表性数据集的规模与用途。图像方面，Open Images^[62] 包含约 900 万张图像及框注，存储需求约 18TB。此外，LAION^[59] 等开放图文对数据集规模可达数亿至数十亿样本，常用于视觉语言模型的预训练。视频方面，YouTube-8M^[65] 等基准包含数百万视频及其标签，用于视频理解与生成的模型训练往往需要 PB 级存储与高吞吐的通信互联支持。文本方面，大语言模型常在由公

开网页与多源文本构建的海量语料上进行预训练，其中 The Pile^[64] 等规模化数据集可提供数万亿 token 的训练文本。

除数据规模增长外，当前 MLLMs 训练还普遍呈现批次大小与序列长度同步增大的趋势，进一步加剧单设备显存压力。一方面，为稳定梯度估计并提升分布式优化器收敛性，预训练与指令微调常采用较大的全局批次大小（global batch size），使激活值的显存开销随批次大小近似线性增长；另一方面，文档与图表理解、多图对话、高分辨率输入与视频帧建模等任务显著增加了碎片（patch）化后视觉词元（token）的数量，同时语言侧也持续向长上下文扩展^[37]，如 LLaVA-NeXT 等工作进一步增大了训练与推理中的上下文规模^[91]。视觉信息与文本信息拼接后，序列长度进一步拉长。

在 Transformer 架构下，激活值显存开销对批次大小与序列长度高度敏感，多模态信息叠加后，该开销会被进一步放大。因此，即便模型参数规模保持不变，更大的批次大小与更长的序列长度仍会与模块异构导致的不均匀显存分布相互耦合，使显存开销成为制约单卡可训练配置与端到端吞吐的关键瓶颈之一。综合数据规模、存储 I/O 供给与激活显存压力等因素，混合并行训练（数据并行、模型并行及其组合）与重计算等显存优化技术已成为 MLLMs 训练中的必要手段。

表 2.1 多模态大语言模型训练中常用的数据集类型与规模

类型	规模量级	存储量级	典型用途
图像文本对	数百万 – 数十亿	TB–PB	图文预训练、对齐（如 Open Images ^[62] ）
视频	数百万 – 数亿	百 TB–PB	视频理解、生成（如 YouTube-8M ^[65] ）
文本 / 语料	数百亿 – 数万亿 token	PB 级	LLM 预训练、指令数据

2.1.2 多模态大语言模型的训练过程

多模态大语言模型的训练通常采用多阶段流程，并通过分模块冻结控制各阶段的可训练参数规模，以在训练成本与模型性能之间取得平衡^[10, 11]。整体上，该流程遵循“先对齐、后增强”的路径，先完成跨模态表征对齐，再逐步强化指令遵循、对话与生成等能力。具体而言，训练流程一般可归纳为以下三个阶段：（1）模态对齐。在大规模图文对或视频—文本对等多模态数据集上主要训练 Input Projector，必要时联合更新部分 Encoder 或 LLM Backbone，使视觉等模态特征与 LLM 的语义空间对齐。（2）指令微调（Instruction Tuning）。在高质量指令—回答数据上继续训练，以提升指令遵循与多轮对话能力。该阶段通常冻结 Encoder，仅

更新 Projector 与 LLM Backbone 的部分层。(3) 多模态生成微调（若任务涉及文生图等生成能力）。在对应生成数据上训练 Output Projector 与 Modality Generator，同时对 LLM 采用冻结或低秩微调等轻量更新方式。

上述多阶段范式已被广泛采用：LLaVA^[11] 在约 60 万图文对上完成视觉—语言对齐与指令微调；InstructBLIP^[15] 在 13 个指令化数据集上开展微调；Qwen-VL^[17]、LLaVA-NeXT^[91] 等工作进一步扩展了指令数据规模与上下文长度；Gemini^[92]、PaLM^[93] 等闭源系统则依赖更大规模数据与算力，但其训练阶段的划分也遵循上述范式。

由于不同训练阶段对模块冻结的设置不同，相同的模型结构在训练过程中会呈现不同的计算负载与显存开销分布。换言之，模块异构不仅体现为静态架构差异，也表现为随训练阶段而变化的动态负载形态。从机制上看，冻结策略直接改变各模块的反向计算量与显存开销。冻结参数既不参与梯度计算，也无需在反向阶段为其保留完整激活与梯度，优化器状态（如 Adam 的一阶与二阶动量）同样仅在可训练参数上维护。因此，随着冻结比例升高，单步计算量与显存需求通常随之下降。在流水线并行场景下，上述机制会进一步转化为流水级间负载失衡。若某一流水级对应的大段模块被冻结，该级的反向传播计算量将显著小于仍需全量反传的流水级，导致整体流水线效率下降。当训练环境为异构 GPU 集群时，设备间本就不一致的显存容量与算力水平，会与上述阶段切换引起的动态负载变化相互叠加，使固定的并行划分与调度策略难以在各训练阶段均维持较高效率。因此，能够随训练阶段与冻结状态自适应调整划分与调度的方案，具有重要的工程意义。

图 2.1 示意了多阶段冻结训练中常见的模块冻结策略，有助于直观理解不同训练阶段在算力与显存需求上的动态变化。

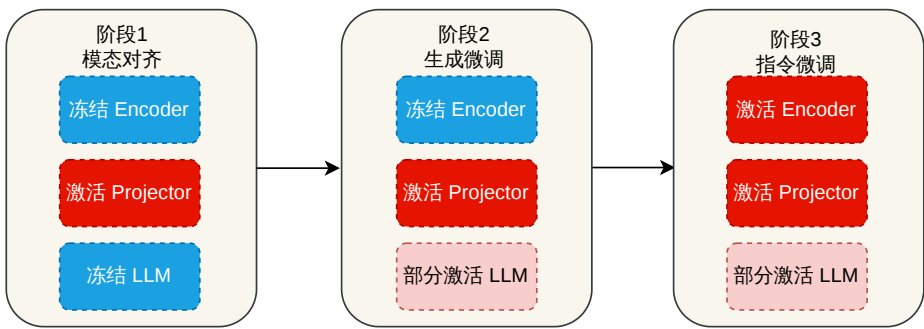


图 2.1 MLLM 多阶段训练中典型模块冻结

2.1.3 多维混合并行训练方法

数据并行将训练数据在各计算设备组间划分，每个设备组持有一份完整模型，每个设备组本地进行前反向计算后发起 All-Reduce 集合通信在各设备组之间同步

梯度数据^[40, 70, 94]。如图 2.2 所示，各设备独立计算所分配 mini-batch 的梯度，再在前向传播和反向传播都结束后聚合梯度并更新参数，所有的设备组保持参数一致。DP 实现简单、扩展性好，是 PyTorch DDP^[40]、Horovod^[70] 等框架的默认方式。PyTorch 的 FSDP^[95] 在数据并行基础上对参数、梯度与优化器状态进行分片，与 ZeRO-3 思路相近，可在单卡显存受限时支撑千亿级稠密模型训练。Parallax^[96] 针对稀疏与异构场景对参数服务器与 All-Reduce 做了灵活选择。数据并行的局限在于单卡需容纳完整模型与优化器状态（DP）或需额外通信聚合分片（FSDP），无法单独支撑超大参数规模，常与张量 / 流水线并行或 ZeRO 组合使用。近年来，EDiT^[97] 与 HALoS^[98] 分别从本地 SGD 与分层异步同步机制出发，面向异构或跨地域网络降低通信瓶颈。而去中心化训练方面，From Promise to Practice^[99] 在 Transformer 训练中通过去中心化 Adam 与通信 - 计算重叠实现了稳定加速。

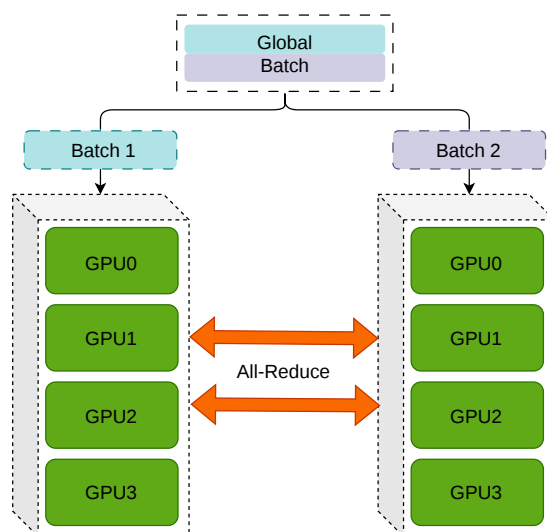


图 2.2 数据并行示意

张量并行在层内划分模型权重与计算，使多设备协同完成单个模型层的前向与反向，并通过通信保持数学等价性^[35, 36]。图 2.3 给出了张量并行示意图，左右两幅子图分别示意了两种使用张量并行的矩阵乘法形态（分别对应列并行与行并行下的权重分片及相应的集合通信操作），用于说明单层内如何拆分线性层并在设备间协同保证计算正确性。使用张量并行后，各设备仅持有部分权重与局部激活片段，依赖 All-Gather、All-Reduce 等高频、小粒度的集合通信在层内传播数据，这与数据并行主要在层外同步梯度的模式有本质区别。TP 通信密集且对带宽敏感，工业实践中多限于单节点内部或高带宽拓扑内，并与流水线、数据并行组合使用。

流水线并行在层间划分模型，每个设备持有若干连续模型层构成一个流水级。如图 2.4 所示，对于一个 12 层模型的三级流水线，每个设备持有 4 层，数据依次

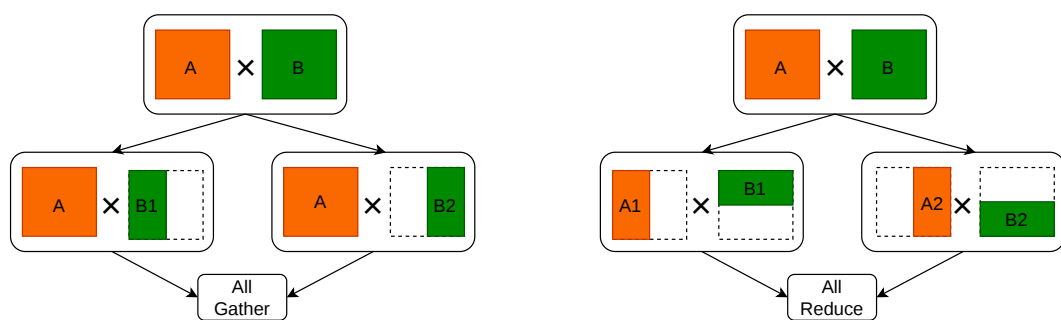


图 2.3 张量并行示意（左、右分别示意两类典型矩阵切分与通信模式；二者在注意力子层与前馈子层中均需组合使用）

流过各级流水线进行前向和反向计算。GPipe 使用多个 micro-batch 依次流过各流水级以重叠不同设备的计算，降低流水线气泡率^[41]。PipeDream^[42] 提出 1F1B（1 Forward 1 Backward）交错调度流水线技术。TeraPipe^[100] 在序列内做 token 级流水线，实现更细粒度的层间并行。Chimera^[101] 采用双向流水线减少气泡。Hanayo^[102] 等波式调度进一步优化多流水级下的计算重叠。近年来，ZeroBubble^[103] 在同步语义下将反向拆分为“对输入的梯度”与“对参数的梯度”两阶段，结合 1F1B1W 等调度与参数更新重叠，实现了近似零气泡，在相同显存下相较 1F1B 提升吞吐约 23%–31%。DualPipe^[104] 采用双向 micro-batch 流动与前后向重叠，在 DeepSeek 等超大规模训练中用于降低气泡占比。AdaPipe^[105] 联合优化自适应重计算粒度与流水线分割策略，在 GPT-3、Llama 2 等模型上取得显著加速；MEPipe^[106] 提出切片级流水线调度，在消费级 GPU（如 RTX 4090）上实现显存高效的大模型训练。WeiPipe^[107] 将传统“激活传递流水线”改为“权重传递流水线”，在长上下文训练中显著降低通信开销。PP 通信量小，仅在流水级间传递中间输出，适合跨节点扩展，常与节点内 TP、全局 DP 组合^[36]。

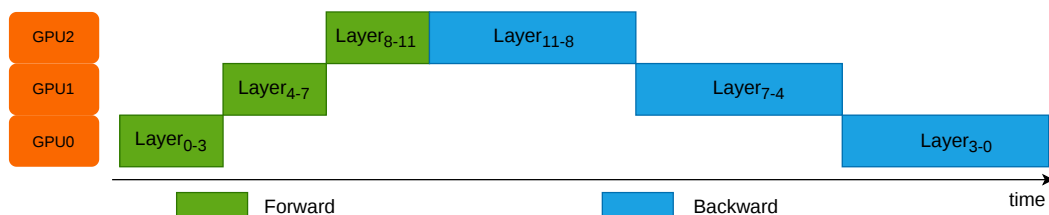


图 2.4 流水线并行示意

序列并行（SP）针对长序列场景，将输入沿序列维划分到多卡，每卡仅处理一段子序列，通过通信在注意力等层交换信息以保持语义一致^[71, 108]。DeepSpeed-Ulysses^[108] 将序列均匀分片，在注意力层采用双阶段 All-to-All 重分布，使通信量随序列长度与设备数成比例缩放，支持百万级 token 上下文长度的训练。Megatron

中的序列并行实现常与张量并行共用进程组，与 TP、PP、DP 组合构成多维混合并行^[71]。

上下文并行（CP）在如今序列长度越来越长的场景下得到了广泛应用。单卡显存难以容纳全长序列的激活时，将输入沿序列维度划分到多卡，在注意力层通过 All-to-All 集合通信进行分块计算保持注意力分数的完整性。与张量并行共用进程组的序列并行（Sequence Parallelism）不同，上下文并行把模型输入在序列维度做切分，不同的序列块使用不同的 GPU 进行计算，称为 Context Parallelism（CP）^[71]。CP 在 8K+ token 场景下显存收益显著，但会引入额外通信与拓扑约束，需与 Flash Attention、重计算等技术配合使用。

在 MoE（Mixture of Experts）架构中，模型由多个专家（experts）组成，门控网络会为每个 token 选择 Top- k 个专家，仅对被选中的专家执行计算，从而在控制激活成本的同时提升模型容量。基于这一执行机制，专家并行（EP）是一种面向 MoE 模型的并行策略，即将不同专家分片到不同设备，每设备仅持有部分专家的全量权重。前向阶段根据门控路由将 token 激活发送至对应专家所在设备计算，随后汇总得到输出^[43]。与张量并行相比，EP 的关键通信开销集中在 token 级路由与重分布上，其通信模式通常为 All-to-All 的 token dispatch/reduce，这使得它尤其适合专家数较多且单专家规模适中的 MoE 场景。GShard^[43] 将 MoE 与张量并行结合并做自动分片；DeepSeek-MoE^[109] 等在有限资源下通过 MoE 扩展语言模型规模。进一步地，DualPipe^[104] 等工作也在流水线并行中结合专家并行以支撑超大规模 MoE 训练。EP 的主要挑战是门控路由导致的专家负载不均（routing imbalance）以及路由带来的通信开销，通常需要高带宽互联与负载均衡设计。图 2.5 展示了专家并行中的典型路由过程：门控网络先为 token 选择 Top- k 专家，再通过 All-to-All 集合通信将 token 分发到对应 GPU 上的专家执行计算，最后汇总计算结果。

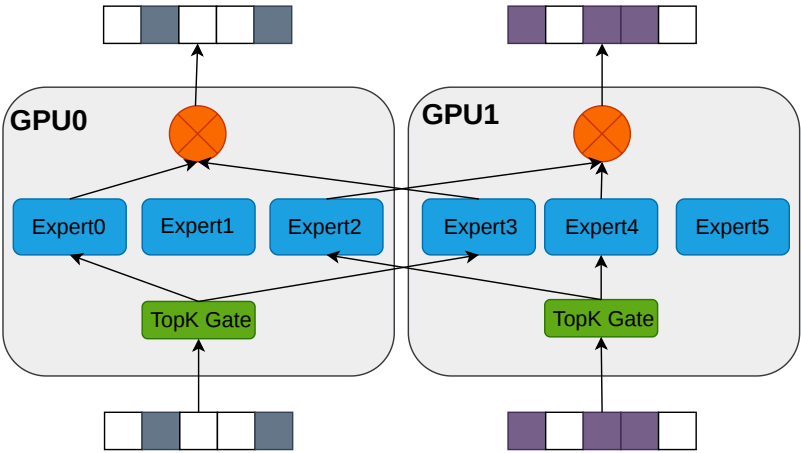


图 2.5 专家并行中的门控路由与专家分布示意

2.1.4 零冗余优化器（ZeRO）

ZeRO^[38, 39] 是在数据并行（DP）框架下的显存优化方法，其核心思想是将 DP 中冗余的优化器状态、梯度与参数分散存放到不同设备上，并在前反向阶段按需进行聚合。与传统 DP 相比，ZeRO 在不改变训练语义的前提下显著降低单卡显存占用，从而使大参数规模模型能够在有限显存条件下完成训练。

如图 2.6 所示，ZeRO-1、ZeRO-2、ZeRO-3 分别对优化器状态、梯度与参数进行分片，显存节省效果逐级增强，但通信与调度复杂度也同步提高。ZeRO-1 主要缓解优化器状态开销，ZeRO-2 进一步减少梯度冗余，ZeRO-3 则将参数本体也分片，单卡显存压力最低，但需要更频繁的参数聚合通信。针对“显存容量不足但带宽相对充足”的场景，ZeRO-3 通常更具吸引力，而在跨节点带宽受限场景下可能带来额外通信瓶颈。

在扩展方向上，ZeRO-Infinity^[39] 通过将部分状态卸载到 CPU/NVMe 进一步突破 GPU 显存墙。ZeRO++^[72] 通过量化与通信重算降低跨节点通信开销。MiCS^[81]、AMSP^[82] 等方法则面向云环境和统一划分空间进一步优化工程可用性。总体而言，ZeRO-3 与 TP/PP 的组合在工程上仍存在实现约束（如 DeepSpeed 与 Megatron 的 3D 并行常采用 ZeRO-1/2 + TP+PP），需要结合框架能力与集群拓扑进行配置^[38, 39, 110]。因此，实际部署需在显存节省、通信开销与系统复杂度之间权衡，ZeRO 也通常与 TP、PP、SP 等并行方式协同使用，构成当前 MLLMs 训练的主流技术路径。

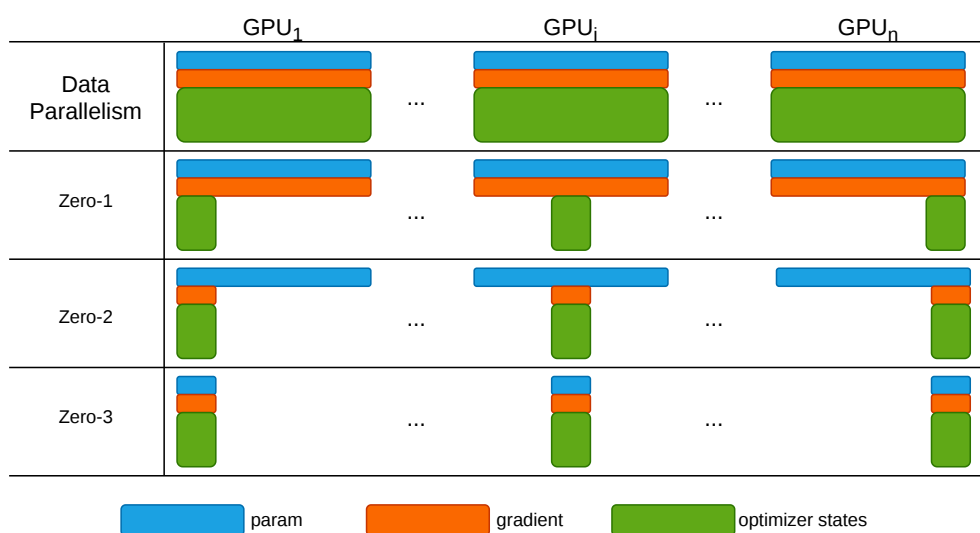


图 2.6 零冗余优化器 ZeRO 示意

2.2 异构 GPU 集群并行训练

第一章从实际部署出发，已说明常见情形：同一集群中往往混有多代 GPU，显存容量与算力并不一致。若仍按同构假设选用默认并行配置，容易出现负载不均衡以及单点设备制约整体效率等问题。本节不再展开现象层面论述，而沿文献脉络梳理近年面向异构 GPU 集群的并行训练工作，并说明其在训练多模态大语言模型时仍存在的不足。

2.2.1 异构 GPU 集群并行训练策略

围绕异构环境下的并行训练，已有若干代表性工作从不同角度展开了探索。

HetPipe^[52] 较早面向异构 GPU 设备提出并行训练方案。该方法将若干 GPU 组织为虚拟工作节点（VW），在节点内部采用流水线并行处理小批量，并在节点间叠加数据并行以提升训练吞吐。通过将不同能力的设备进行分组，HetPipe 在一定程度上缓解了低算力设备导致的流水线瓶颈。然而，该方法未对 VW 内部的拓扑结构、设备排列与通信开销进行细粒度建模。同时，其建模前提主要针对结构相对规整的层堆叠模型。对于编码器、投影层与语言模型模块差异显著的 MLLMs，以及训练过程中分阶段冻结策略切换等场景，该方法的适用性仍然受限。

Whale^[51] 针对设备间显存与算力差异，采用与设备能力相匹配的差异化批次大小和分片策略，在数据并行与张量并行维度进行负载分配。该方法为异构设备上的负载平衡提供了可实现路径，但其划分粒度主要停留在子图层级。随着集群规模扩大或异构维度增加，仅按照设备算力按比例切分的方式通常难以同时兼顾通信效率与显存约束。

AMP^[74] 将异构场景下并行策略配置问题进一步自动化。该方法将层间结构差异表征为“模型异构”，将设备显存、算力与链路特征表征为“设备异构”，在此基础上于 DP、TP、PP 组合空间构建开销模型，并采用动态规划求取近似最优配置。已有结果表明，该方法在异构场景下可获得优于若干基线的性能。然而，AMP 仍以全局粗粒度划分为主，通信与显存估计误差会传导至最终配置。若缺少显存上界等硬约束，实际部署中仍可能出现显存溢出（OOM），并需要额外超参数优化以维持训练稳定性。

Metis^[78] 针对异构 GPU 集群提出快速自动并行策略搜索，借助异构感知的开销模型与贪心搜索自动确定 DP/TP/PP 配置，在多代 GPU 混合的集群上取得显著加速。HAP^[79] 将异构环境下的 SPMD 并行优化建模为自动程序合成问题，通过 A* 搜索联合优化各层的张量分片方式与通信策略。

上述工作在缓解负载不均衡与显存压力方面提供了有益借鉴，但这些方法都

有一个共同前提：模型主体可近似为结构相近的 Transformer 层堆叠。当研究对象转向多模块拼合、各模块计算与显存开销差异显著的多模态大语言模型，并需应对训练过程中分阶段冻结策略的动态变化时，原有的共同前提不再充分成立，性能收益也随之下降。

2.2.2 异构 GPU 集群并行训练的工业实践

近年来，工业界研究更关注在模型与数据规模确定的前提下，异构 GPU 集群应如何配置不同的 GPU、流水级如何映射到异构设备，以及如何在成本与时延约束下满足训练性能需求。Espresso^[77] 针对上述需求构建了并行训练系统，其使用 Profiler 在各类 GPU 上测量迭代时间与显存占用，使用 Allocator 完成设备分配，Stage Placer 确定流水级映射，Performance Estimator 估计给定配置下的吞吐。用户输入模型、数据集、训练轮次与 SLO 等指标后，系统输出 GPU 选择方案以及模型映射策略，在成本与效率之间进行权衡。

多厂商设备的混合则进一步放大了上述需求。HetHub^[75] 针对华为昇腾、AMD、NVIDIA 等计算卡混合部署的异构集群，实现了统一通信、性能预测和自动并行规划。在 768 张异构计算卡上训练 Llama-140B，可达到理论性能上界约 97.49%，表明多厂商混部集群同样能够支撑超大模型训练，但对系统工程要求较高。HexiScale^[76] 则把 DP、PP、TP 的非对称划分写成约束优化，用层次图划分给各卡分计算，并提出了可复用的算法框架，在 7B–30B 规模上 MFU 与同构系统只差大约 3.5%。

Zorse^[111] 针对 LLM 训练中流水线与数据并行如何组合以降低通信与显存开销、流水级能否非对称划分、同一数据并行组内能否容纳能力不同的设备等问题进行求解。它将上述灵活性统一纳入实现，并配备自动规划器，实验表明相对既有工作可获得可观的训练加速。Helix^[112] 将异构 GPU 集群与非对称网络建模为最大流问题，通过全局优化流水级映射与请求调度，在混合多代 GPU 的集群上实现高效的模型部署。

在多模态训练的工业实践方面，DistMM^[80] 针对多模态模型训练中各子模块算力需求差异大的问题，提出以子模块为粒度的异构并行策略，分别为编码器、投影层与 LLM 选择最佳并行配置。DistTrain^[73] 进一步通过解耦多模态大语言模型各模块训练，为数据异构与模型异构提供统一的分离式训练框架，在大规模多模态训练中降低了端到端迭代时间。

ZeRO 系列的方法在异构 GPU 集群上亦不能完全照搬同构假设下的经验。原版 ZeRO^[38, 39] 中参数与梯度的 All-Gather 等集合通信，在跨节点且链路带宽参差不齐时易成为瓶颈。MiCS^[81] 在超大模型与复杂网络拓扑下对高带宽链路的利用

仍不充分。ZeRO++^[72] 借助量化操作降低通信流量，需在收益与精度、通用性之间权衡。PaRO^[113] 与 OptiReduce^[114] 分别从数据并行状态分片与 AllReduce 通信延迟优化角度提升了分布式训练效率。AMSP^[82] 等以削减跨节点通信为主，对单机内显存、算力与带宽细粒度错配的综合利用讨论相对较少。

综合上述工作可以看出，学术界与工业界正在通过自动并行和约束优化等方法，逐步降低训练系统对手工调参的依赖。但现有路线多数仍建立在通用 LLM 或“先近似同构、再扩展”的前提上。对于模块异构性显著且资源需求差异巨大的 MLLMs，训练过程中又伴随多阶段冻结切换，计算图与负载分布会随阶段变化，已有方法往往难以稳定适配，尚未形成面向该场景的系统化方案。结合本章对并行划分与异构扩展方法的综述可知：在模块异构与分阶段冻结并存的条件下，如何联合优化划分、调度与显存策略，仍是一个未充分解决的问题。

在工程落地层面，仍需解决异构环境下与 Megatron、DeepSpeed 等框架的集成问题，以及多阶段训练中并行划分与调度能否随阶段变化及时更新的问题。Frenzy^[53] 等内存感知的无服务器调度研究，为资源分配与阶段放置提供了有价值的思路；Espresso、Zorse 等系统则尝试将性能分析、资源配置与并行规划封装为更完整的工具链。然而，面向 MLLMs 的模块级划分、冻结切换下的自适应调度以及显存优化协同，当前仍缺乏成熟的一体化解决方案。

2.3 重计算技术

在模型训练过程中，前向传播产生的中间激活通常需要常驻显存直到反向传播阶段以支持梯度计算。在大批次和长序列情形下，若完整保存所有激活，显存占用会随批次大小与序列长度快速增长，并逐渐成为训练瓶颈。重计算（Recomputation），又称激活检查点（Activation Checkpointing），通过“以计算换显存”的方式缓解这一问题。本节主要从策略分类与主流实现两个方面综述重计算技术，并概括其同构假设下的适用边界，为第四章的异构重计算方法作铺垫。

2.3.1 重计算基本原理与分类

重计算的基本思想是在前向传播阶段仅保存部分中间激活，在反向传播阶段对丢弃的激活值重新执行局部前向计算，以恢复梯度计算所需的中间结果，如图 2.7 所示^[115]。该机制本质上是在显存占用与额外计算开销之间进行权衡。按照配置粒度与作用范围，重计算大致可分为全部重计算、选择重计算和细粒度重计算三类。

全部重计算（full recomputation）以层为单位保存模型层输入，而不长期保留中间激活，当反向传播需要这些激活时，再从最近的输入进行完整的前向计算来恢复中间激活。该策略实现相对直接，显存收益也最明显，但需要额外重算完整

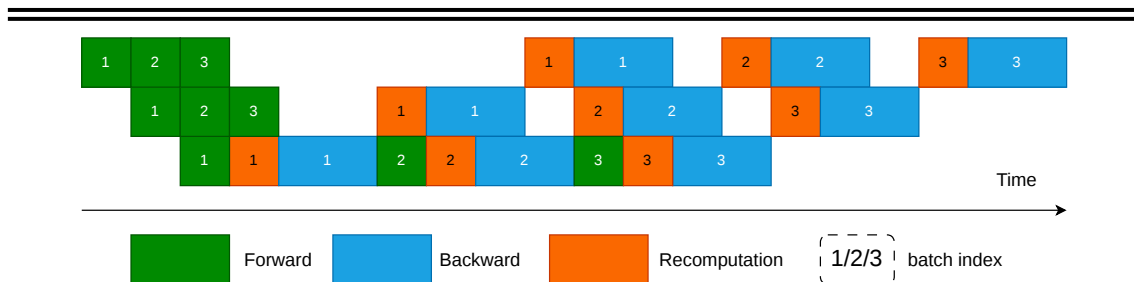


图 2.7 重计算示意图

层或层块，计算冗余较高。Chen 等^[115] 针对链式计算图给出了重计算最优策略分析，表明该类方法可以显著降低显存复杂度，但会带来额外训练时间开销。因此，全部重计算更适合显存极度受限而算力相对充裕的场景。

选择重计算（selective recomputation）不再对整个 Transformer 层统一执行完整重计算，而是仅对显存占用较大但重算代价相对可控的子模块进行重算，其余部分仍保留激活。以大规模 Transformer 为例，已有工作通常优先对注意力中的关键中间结果进行重算，因为这类张量的显存开销随序列长度增长速度较快，却往往只引入相对有限的额外计算开销^[71]。与整层重计算相比，选择重计算的显存收益略低，但通常能显著降低额外计算开销，因此更适合作为工程实践中常用的权衡型策略。

细粒度重计算（fine-grained recomputation）进一步下沉到算子级或张量级，更关注“如何在模块内部重算”。这类方法能够利用不同算子的显存占用、通信需求与 FLOPs 差异进行更细致的权衡；若再与通信或并行调度协同设计，还可能同时改善显存效率与训练吞吐^[116]。对于由 Encoder、Projector 和 LLM Backbone 等异构模块组成的 MLLMs，细粒度策略也为后续按模块、按设备差异化配置重计算提供了基础。

2.3.2 重计算的主流实现及其局限性

在 Megatron-LM、NeMo 等主流并行训练框架中，重计算通常被实现为可配置的多级机制，既支持按层或按层块的全层重计算，也支持按子模块的选择性重计算^[71]。这类机制使用户能够在显存预算与计算开销之间进行权衡，因此已广泛应用于大规模 Transformer 模型训练。总体而言，主流框架已经为重计算提供了较成熟的工程支撑，但其配置仍主要围绕通用 LLM 语言模型的场景展开。

尽管如此，现有重计算策略大多建立在同构设备假设之上，重计算层数、重计算范围和参与重算的子模块往往需要人工指定，并默认各设备的显存与算力相近。因此，这些方法通常难以根据异构集群中不同设备的资源差异进行在线自适应调节。近年来，Mist^[117]、AdaPipe^[105]、Colossal-Auto^[45] 等工作开始尝试将重计算与并行划分、流水线配置或自动搜索联合优化，为后续研究提供了参考。

但总体来看，现有重计算方法仍主要面向同构集群与通用 LLM。对于异构 GPU 集群上的 MLLMs 训练，尤其是在模块异构与分阶段冻结同时存在的情况下，如何联合考虑划分、调度与重计算配置，仍缺乏成熟的系统化方案。

2.4 本章小结

本章延续第一章的问题背景，分三部分归纳相关工作：多模态大语言模型（MLLMs）的架构、训练数据与冻结范式，以及数据并行、张量并行、流水线并行、序列并行、专家并行与 ZeRO 等基础并行技术；异构 GPU 集群上的并行训练与调度；重计算的策略、实现及其在同构假设下的局限。

总体来看，面向同构集群与语言模型的并行划分、调度与重计算已相对成熟，但对 MLLMs 的模块异构与分阶段冻结、以及集群设备异构的联合约束仍关注不足，难以同时兼顾负载均衡与训练速度，从而为后续章节讨论异构 GPU 集群上多模态大语言模型的混合并行与异构重计算提供问题导向。

第三章 多模态大语言模型在异构 GPU 集群上的高效混合并行技术

多模态大语言模型（MLLMs）在结构上由编码器、投影层与 LLM Backbone 等异构模块组成，训练时又广泛采用分阶段冻结的训练策略，致使不同模型层的计算量与显存需求随模块类型与参数训练状态而呈现显著差异。在异构 GPU 集群上若直接沿用面向同构集群、针对传统大语言模型所设计的并行策略，易产生负载不均衡等问题而导致资源利用率低下。针对上述问题，本章介绍 MMoH（multimodal large language model training on heterogeneous clusters）方法。该方法通过需求驱动的设备拓扑与复合粒度模型划分，在异构 GPU 集群上为 MLLMs 自动生成模型划分和设备映射，并引入冻结感知的提前终止调度器，在冻结场景下识别并消除冗余的反向计算与流水级间通信，在保证训练收敛性的前提下提升训练速度。本章首先给出问题背景与动机，随后概述 MMoH 的整体架构与各组件职能，最后给出实验结果。

3.1 引言

3.1.1 问题与动机

传统大语言模型通常由结构一致的 Transformer 层堆叠而成，在常见配置下各层的计算量（FLOPs）与激活显存占用在不同层之间差异较小，因而可近似视为“层间负载同构”。面向 LLMs 的异构 GPU 感知并行方法（如 Whale^[51]、Espresso^[77]、Metis^[78]）往往据此采用相对粗粒度的划分准则，即以设备算力或设备显存等单一或组合指标为约束，将连续模型层按比例映射到不同设备，从而获得近似均衡的流水级负载。在同构 GPU 集群上，上述策略通常具有较好的可解释性与工程有效性。若各设备算力与显存容量相近，则只需使各流水级承载的模型层数大致相当，即可在流水线并行中获得较均衡的负载与较高的设备利用率。

然而，多模态大语言模型在架构上与传统大语言模型存在较大差异。如图 3.1 所示，最常见的 MLLMs 结构可归纳为三个核心模块。编码器（Encoder），负责将图像、视频等多模态输入编码为特征表示，多采用视觉 Transformer 或轻量卷积网络。投影层（Projector），将编码特征映射至与 LLM 词嵌入空间对齐的语义空间，常见形式为线性层或 Q-Former^[10]。LLM Backbone 承担主要的语言理解与生成计算，由参数量巨大的 Transformer 层组成^[10, 11]。编码器单层的参数量与序列长度通常远小于 LLM Backbone 单层的参数量与序列长度，投影层则为小型模块，因此 MLLMs 各层在计算量与显存开销上差异显著，在某些模型配置下甚至可达数

十倍。在实际部署中，研究者往往需要在由多代 GPU 混合构成的异构集群上训练这类模型，不同计算设备之间的算力和显存差异显著，“模块异构 + 设备异构”叠加导致了严重的负载失衡问题。

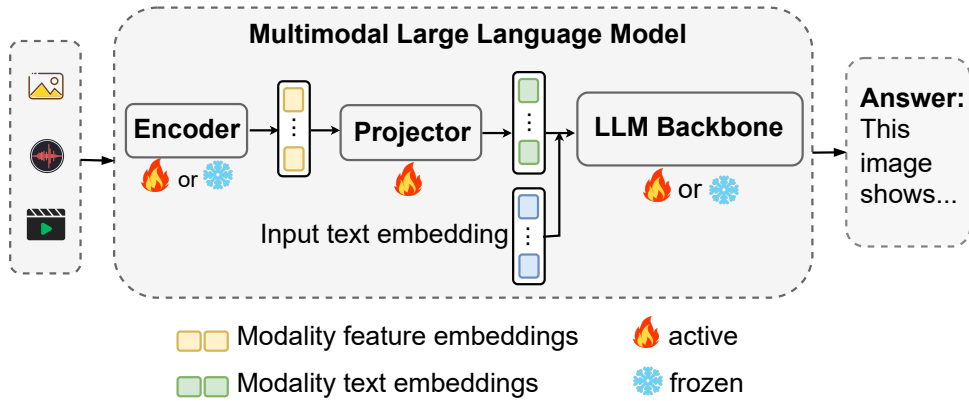


图 3.1 主流 MLLMs 的编码器—投影层—LLM 结构及冻结方式。

若仍采用与 LLM 相同的单一层粒度进行模型划分，将导致部分流水级过载而其他流水级空闲，无法达到负载均衡。由于整体迭代时间由瓶颈流水级主导，故异构 GPU 设备的算力与显存无法得到有效利用。图 3.2 对比了 LLM (LLaMA-3B) 与 MLLM-3B (见表 3.2) 在各层计算开销与显存占用上的变化趋势。LLM 各层较为均匀，而 MLLMs 随模块类型与冻结场景 (NF 表示不冻结，FB 表示编码器与 LLM 均冻结) 呈现显著差异。从图中可以观察到，在编码器段，单层计算量 FLOPs 与显存曲线整体处于较低水平，而在进入 LLM Backbone 后，相邻两层之间的计算量跃升明显，显存占用也显著增加。若简单地以层数均分作为模型划分的方式，则很容易出现流水级负载不均衡的情况，导致少数高负载流水级成为流水线瓶颈限制吞吐。

进一步地，异构 GPU 环境下设备能力也存在显著差异。例如在由 RTX 3090、RTX 2080 Ti 与 RTX 2080 组成的集群中，不同型号 GPU 在算力与显存上的差异可达 2-3 倍以上。若不区分各模块对算力和显存的需求特征，而仅按照“设备显存从大到小”或“FLOPS 从高到低”的简单规则对异构设备进行排序并顺序映射流水级，则无法保证算存（计算与显存）需求高的模型层被分配到算存能力强的设备。在某些情况下，甚至可能出现高负载的 LLM 层被映射至低性能 GPU 上，进一步加剧负载不均衡。

另一方面，MLLMs 训练普遍采用冻结 (freeze) 技术^[10]。在预训练或模态对齐的训练阶段，模型仅更新投影层或部分 LLM 层，而编码器与 LLM Backbone 保持冻结，以利用已有的预训练权重、降低过拟合并节省优化器状态与显存。被冻结后的参数不参与梯度计算与参数更新，其反向计算量与梯度同步开销显著降低，其计算和显存需求与可训练参数截然不同。现有异构并行策略大多假设所

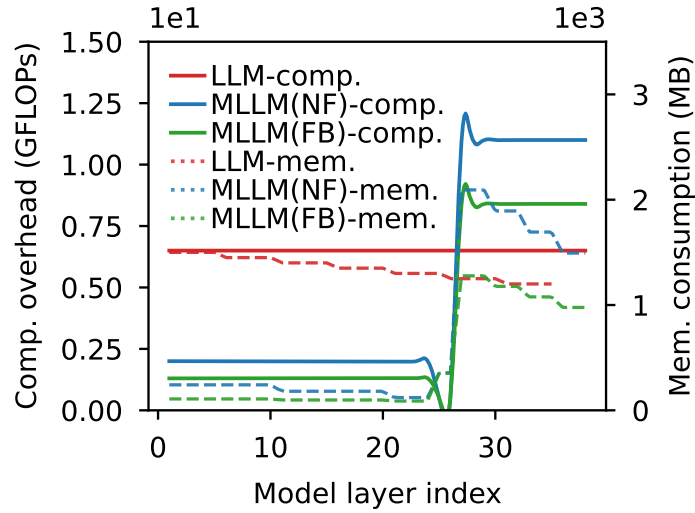


图 3.2 LLM 与 MLLM-3B 各层计算开销与显存占用的变化趋势（NF 为不冻结，FB 为 freeze-both）。

有参数在整个训练过程中均处于激活状态，无法随参数训练状态（冻结与激活）的动态变化而调整划分与调度。在“仅冻结编码器”（freeze-encoder）、“仅冻结 LLM”（freeze-llm）或“同时冻结编码器与 LLM”（freeze-both）等场景下，若继续采用为“全参数激活”场景设计的策略，则不能达到最优性能。

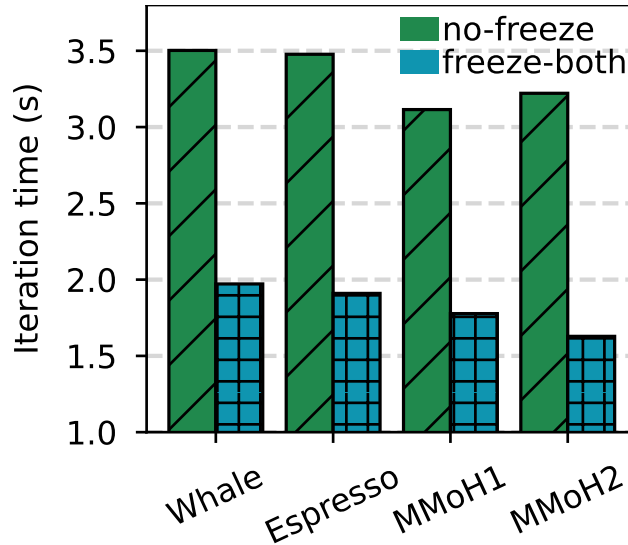


图 3.3 不同并行策略在两种冻结场景下训练 MLLM-3B 的迭代时间，MMoH1/MMoH2 为 MMoH 规划器针对各场景得到的策略。

如图 3.3 中不同并行策略在两种冻结场景下的迭代时间对比所示，相较于针对异构 GPU 集群下为传统大语言模型所设计的并行策略，MMoH 为多模态大语言模型设计的策略可加速约 13%–21%。与此同时，根据实验结果可知，同一组设备拓扑与模型划分方案很难在所有冻结场景下都保持最优性能，在不同的冻结场景下，最优的模型划分策略会有显著差异。直观理解，在“仅训练投影层”的阶

段，编码器与大部分 LLM 层几乎不产生梯度同步与参数更新，若仍然为其预留与 no-freeze 阶段相同的显存与计算预算，就会造成大量资源浪费。而在激活更多模块时，若不重新规划模型划分与设备映射，又会重新暴露出负载瓶颈。综上所述，当前为异构 GPU 集群、针对大语言模型所设计的并行策略难以适配多模态大语言模型的训练。

上述不匹配在工程层面往往进一步体现为两类现象。其一，面向异构集群的划分与映射配置空间较大，开发者通常需要在不同 batch size 与序列长度组合下反复调整划分与设备映射，以在策略可行性与迭代时间之间进行权衡，试错成本随集群规模与模型复杂度显著上升。其二，出于实现复杂度与训练稳定性等考虑，并行策略在训练过程中较少随阶段切换而在线调整，从而在冻结场景发生变化后难以及时重新均衡负载，GPU 利用率可能出现阶段性下降。

从形式化角度，可将上述挑战归纳为两类相互耦合的约束。第一类约束刻画异构设备条件下的流水线负载失衡。设流水线并行度为 P ，各流水级完成一次迭代的用时为 $t_1, \dots, t_P$ ，在同步流水线语义下一次迭代的完成时间由 $\max_{1 \leq j \leq P} t_j$ 决定。若划分与映射未充分匹配各流水级的计算与显存需求，则瓶颈流水级将主导迭代时间，其余设备在同步点等待，异构集群的整体资源利用率随之下降。第二类约束刻画分阶段冻结训练引起的有效负载与通信模式变化。记冻结流水级集合为 $\mathcal{F}$ ，则对 $j \in \mathcal{F}$ 的流水级而言，其可训练参数集合收缩，反向计算与梯度同步需求显著降低；对 $j \notin \mathcal{F}$ 的流水级仍需维持完整的反向传播与梯度聚合。若划分与调度未显式考虑冻结引起的变化，则一方面可能在连续冻结流水级上保留冗余的反向与级间通信，另一方面也难以在阶段切换后重新分配各流水级的有效工作量，从而使整体吞吐难以保持稳定。

因此，在异构 GPU 集群上高效训练 MLLMs 需要同时应对两类相互关联的挑战。第一类挑战来自 MLLMs 的混合模块结构，其层间计算量与显存需求呈显著非均匀分布。第二类挑战来自参数训练状态随训练阶段切换而发生变化，从而使最优的模型划分与调度策略随之改变。为此，MMoH 引入并行策略规划器与提前终止调度器两类核心组件。规划器在给定异构集群规格与 MLLMs 配置的前提下，自动生成面向各训练阶段的近似最优模型划分与设备映射。提前终止调度器在冻结场景中识别并消除不必要的反向计算与流水级间通信，从而在不影响模型收敛性的前提下提升训练效率。

3.1.2 MMoH 框架概览

MMoH 由两大核心组件构成，分别为 *MMoH* 并行策略规划器（MMoH parallel strategy planner）与提前终止调度器（early-termination scheduler）。二者协同工作，

分别负责离线的策略生成与在线训练过程中的调度优化。图 3.4 给出了 MMoH 的整体架构。规划器以 MLLMs 模型配置与异构集群规格为输入，通过需求驱动的设备拓扑识别、复合粒度抽象与冻结感知模型划分生成并行策略。调度器在训练过程中利用冻结特性减少冗余计算与通信。规划器的输入主要包括异构集群规格（各 GPU 的算力、显存及互联带宽）、MLLMs 的模型结构与参数量，以及当前训练阶段对应的冻结状态（如编码器与 LLM 是否冻结）。其输出包括设备拓扑（各流水级与 GPU 的对应关系）、模型划分策略（每个流水级包含哪些模型层）以及 DP/TP/PP 配置。调度器根据模型当前训练阶段的冻结状态，优化流水线调度策略，从而在冻结场景下消除不必要的反向计算与流水级间通信。与 Whale^[51]、Espresso^[77] 等仅按显存或算力比例做层划分、且不区分模块类型与冻结状态的方法相比，MMoH 显式建模 MLLMs 的模块异构与冻结状态，从而在不同冻结场景下获得更高的训练吞吐量。

并行策略规划器以异构集群规格（各 GPU 的算力、显存及互联拓扑）与 MLLMs 配置（模型结构、当前冻结状态）为输入，依次执行三个步骤。第一步为需求驱动的设备拓扑识别，即根据 MLLMs 各模块的计算与显存需求的变化趋势，与异构 GPU 设备组内各设备的算力显存供给的变化趋势进行匹配，筛选出若干候选设备拓扑（即设备组内各流水级与 GPU 的对应关系），使需求与供给在趋势上一致。第二步为复合粒度抽象，即针对编码器、投影层与 LLM Backbone 的结构差异，采用多级粒度将模型重组为模型块（model chunk）列表。第三步为冻结感知的模型划分，即在候选拓扑与模型块列表基础上，将“把模型块分配到各流水级”的问题形式化为整数线性规划（Integer Linear Programming, ILP），在满足显存等约束的前提下，最小化单次训练的迭代时间（其中通信时间依赖各流水级的冻结状态），求解得到各流水级所包含的模型块及与异构 GPU 设备的映射。训练时，每个设备组内采用流水线并行（PP），组间采用数据并行（DP）与张量并行（TP）以扩展规模与批量大小。

提前终止调度器针对冻结技术带来的计算与通信冗余进行剪枝。在流水线并行中，若某流水级及其之前所有流水级均为冻结流水级，则该流水级无需再计算对输出的梯度，也无需接收后续流水级的梯度或向后续流水级发送激活，相应的 P2P 通信操作可安全省略，且不改变梯度反向传播的数学等价性。调度器在 1F1B（One-Forward-One-Backward）流水线调度基础上，识别从流水线首端开始的连续冻结流水级并剪枝冗余的反向计算与通信操作，从而在部分冻结场景下进一步缩短迭代时间。

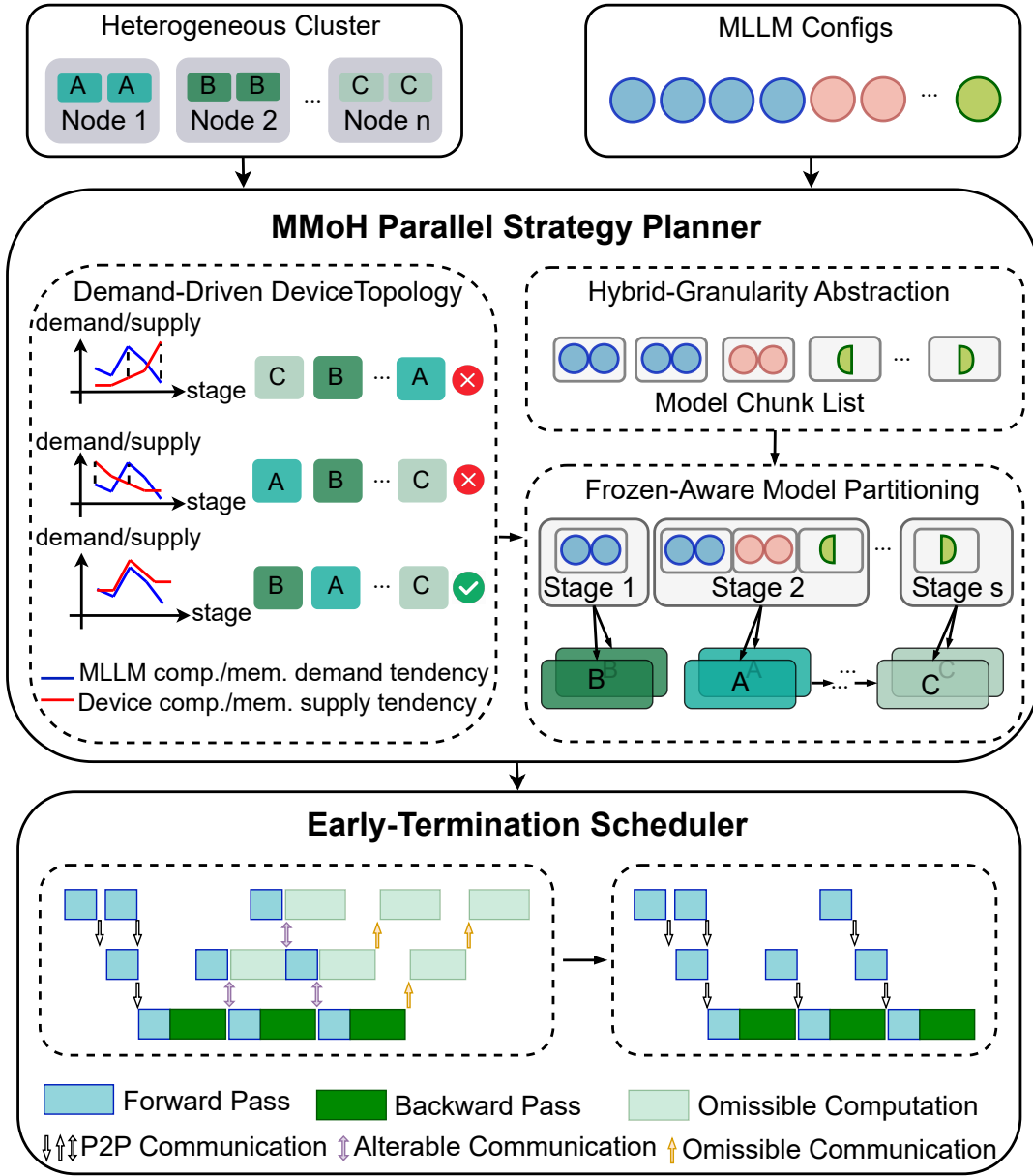


图 3.4 MMoH 框架总览。

3.2 MMoH 并行策略规划器

本节围绕 MMoH 并行策略规划器展开，形式化描述其三个核心步骤：需求驱动的设备拓扑识别、复合粒度模型抽象与冻结感知的模型划分，并给出相应的数学模型与约束条件，整体流程可以概括为以下三步。首先根据模型的计算与显存需求的变化趋势，从所有可能的异构设备排列中筛选出多组候选设备拓扑。随后，将原始模型使用复合粒度抽象为由若干模型块组成的线性序列，使之既能准确反

映模块异构性，又能在划分复杂度与负载均衡之间进行权衡。最后，在给定拓扑与块列表的前提下，将“模型块到流水级的分配”形式化为一个以迭代时间为目标的整数线性规划问题并求解其最优解。

3.2.1 需求驱动的设备拓扑识别

在异构设备组内，不同 GPU 的算力与显存容量存在显著差异。若将算力与显存较强的设备优先分配给计算量与显存需求较高的模型层，则可以更好地对齐供需关系，从而降低显存溢出的风险。相反，若高负载模型块被映射至低性能设备，不仅该流水级将成为全局瓶颈，其余设备也会因等待而长期空闲，导致集群利用率严重下降。

为避免这种供需错配，MMoH 规划器首先通过剖析（profiling）获取 MLLMs 各模型层的计算量（以 FLOPs 计）与显存占用。由于 FLOPs 与显存占用主要由模型参数决定，与具体设备型号关系不大，每类层仅需剖析一次，剖析结果可在不同设备上复用。在获得各层的计算与显存需求后，规划器枚举设备组内所有可能的 GPU 排列（即设备拓扑），并对每种排列评估其“需求侧”与“供给侧”的匹配程度，从而筛选出与当前模型需求相适配的候选拓扑。具体来说，规划器通过计算各拓扑的“需求趋势”与“供给趋势”之间的匹配度（Matching Degree, MD）来量化供需对齐程度，并据此对候选拓扑进行排序与筛选。

设流水线并行度为 P ，即设备组内包含 P 张 GPU。若这 P 张 GPU 分属 k 种异构型号，各型号对应的设备数量分别为 $n_1, \dots, n_k$ ，并满足 $n_1 + \dots + n_k = P$ ，则在同型号设备不可区分的意义下，不同的设备线性排列（即可行的设备拓扑数目）为多重集合排列数：

$$\frac{P!}{n_1! \cdots n_k!}.$$

对每一种可行排列，规划器计算其匹配度 MD 并完成一次候选评估。随后按 MD 从高到低进行排序，并保留排序序列的前一半拓扑作为后续整数线性规划的候选集合，从而在匹配质量与求解开销之间进行权衡。

记 MLLMs 总层数为 l （即编码器、投影层与 LLM Backbone 等模块中所有可单独计层的单元之和），第 i 层的计算量与显存占用分别为 L_i 、 $\hat{T}_i$ （ $i = 0, 1, \dots, l-1$ ）。设备组内流水线并行度（即流水级数）记为 P ，在某一给定设备拓扑下，第 j 个流水级所对应设备的算力与显存容量记为 C_j 、 Q_j （ $j = 0, 1, \dots, P-1$ ）。首先，按“计算均衡”原则将 l 层划分为 P 个连续流水级，使各流水级所分配层的总 FLOPs 与各设备算力成比例，得到各流水级的总 FLOPs 与显存占用 W_j 、 T_j 。

定义模型需求趋势为各流水级“单位显存下的计算量”之比构成的序列，用于刻画各流水级对计算—显存比的需求相对关系：

$$\text{Demand} = \left\{ \frac{W_1/T_1}{W_0/T_0}, \frac{W_2/T_2}{W_1/T_1}, \dots, \frac{W_{P-1}/T_{P-1}}{W_{P-2}/T_{P-2}} \right\}. \quad (3.1)$$

类似地，定义设备供给趋势为各设备计算显存比的比值构成的序列：

$$\text{Supply} = \left\{ \frac{C_1/Q_1}{C_0/Q_0}, \frac{C_2/Q_2}{C_1/Q_1}, \dots, \frac{C_{P-1}/Q_{P-1}}{C_{P-2}/Q_{P-2}} \right\}. \quad (3.2)$$

为量化某一设备拓扑下模型需求与设备供给的匹配程度，定义匹配度（Matching Degree, MD）为两趋势序列的负指数平方误差：

$$\text{MD}(\text{Demand}, \text{Supply}) = \exp \left(-\frac{1}{P-1} \sum_{j=1}^{P-1} (x_j - y_j)^2 \right), \quad (3.3)$$

其中 x_j 、 y_j 分别为 Demand 与 Supply 的第 j 个分量。MD 取值于 $(0, 1]$ ，越大表示该设备拓扑下各流水级的计算与显存需求与各设备能力在趋势上越一致。规划器对所有候选拓扑按 MD 降序排序，保留 MD 较高的前一半作为后续冻结感知的模型划分的候选拓扑，以在保证匹配质量的前提下控制求解规模。

直观上，模型需求趋势刻画的是“从流水级 0 到 $P-1$ ，单位显存下所需算力的相对变化”。若某个流水级对应 LLM 的注意力块与 FFN 块，则 W_j/T_j 与 LLM 的参数设置相关；若对应到编码器层，则与编码器层的模型参数相关。设备供给趋势则刻画“从设备 0 到 $P-1$ ，计算与显存比的相对变化”。将具有较高 C_j/Q_j （算力—显存比）的设备优先映射至具有较高 W_j/T_j （单位显存下的计算需求）的流水级，有助于使各流水级的计算强度与对应设备的算存供给能力相匹配，从而缓解瓶颈流水级对迭代时间的支配效应，并降低显存溢出（OOM）风险。与直接对 W_j 、 T_j 和 C_j 、 Q_j 本身做匹配相比，利用“相邻比值”构成的趋势序列，可以在不依赖绝对尺度的前提下捕捉模型在不同模块下的算存需求变化趋势以及变化幅度。

在实现层面，规划器对所有设备拓扑计算 MD 并按降序排序，仅保留 MD 较高的前一半作为后续冻结感知的模型划分的候选拓扑。这既确保了这些拓扑在趋势层面与模型需求相容，又把整数线性规划的求解规模控制在可接受范围内，避免在匹配度较低的拓扑上浪费时间。整体而言，该步骤使设备拓扑的选取由 MLLMs 的实际计算—显存需求驱动，而非仅按显存或算力单一维度排序，从而更好地适应多模态大语言模型的非均匀结构以及冻结带来的需求变化。

3.2.2 复合粒度模型抽象

若对整个模型采用统一的“Transformer 层”粒度进行划分，将面临两难的权衡问题。编码器单层的计算开销与显存开销都较小，按层划分会导致层数过多、划分搜索空间过大且对负载均衡帮助有限，而 LLM 单层的计算与显存开销都很大，按层划分在异构 GPU 设备上易造成单流水级过载或 OOM。简单地采用“encoder Transformer 层粒度”或“LLM Transformer 层粒度”都不适合多模态大语言模型的异构结构：前者不会改善负载均衡但是会增大模型划分的搜索空间，后者则使得划分粒度太粗，以至于即使采用优化的 GPU 拓扑，仍难以获得良好的负载均衡。因此 MMoH 提出复合粒度模型抽象，将模型重组为若干模型块（model chunk）组成的列表，不同模块采用不同的划分粒度，在划分精度与搜索效率之间取得平衡，如图 3.5 所示。

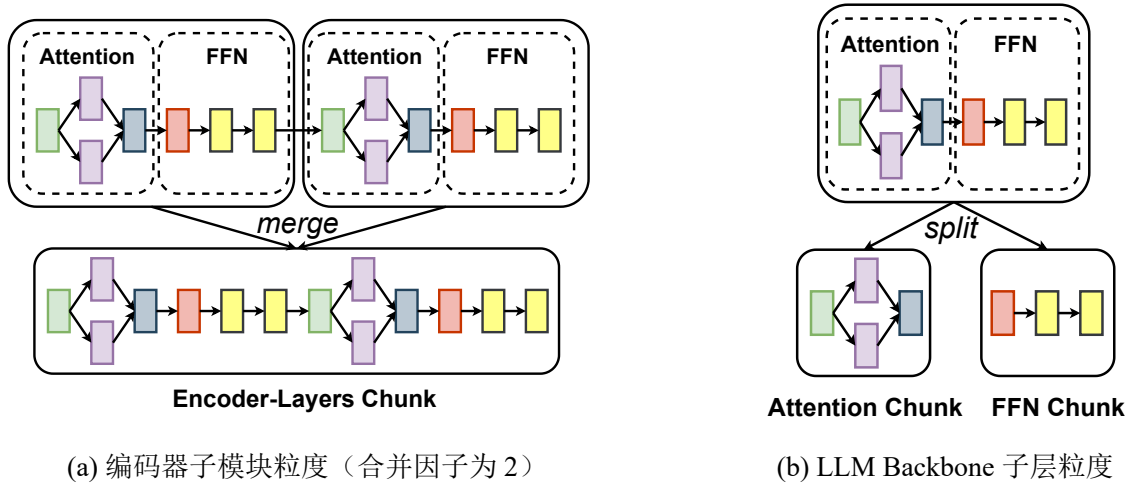


图 3.5 复合粒度抽象示意。(a) 编码器：将连续多层合并为编码器层块；(b) LLM Backbone：将单层拆分为注意力块与 FFN 块，不增加流水级间通信量。

编码器内 Transformer 层的参数量与序列长度相对较小，单层计算需求与显存开销较低，本文采用子模块级粒度，即将连续多层合并为一个“编码器层块”（encoder-layers chunk），如图 3.5(a) 所示。合并因子由剖析结果确定，使得每个编码器层块的计算量与显存占用均不超过 LLM 中单个注意力块或 FFN 块，从而在划分时不会造成较大的负载失衡。该粒度减少了块的数量、降低了后续 ILP 的空间规模，避免了编码器部分划分过细带来的冗余搜索。

LLM Backbone 单层参数量大、序列长，将整层作为一个模型块进行模型划分容易导致某个流水级显存或算力瓶颈，适合采用子层粒度，即将每一个 Transformer 层拆分为“注意力块”（Attention chunk）与“FFN 块”（FFN chunk），划分时以块为单位分配到流水级，如图 3.5(b) 所示。这种拆分不改变层间传播张量的形状，因

此不会改变流水级间通信量（块边界与层边界一致），同时使划分粒度更细，便于在异构 GPU 设备间更精细地平衡负载，并有效缓解低显存设备上的 OOM 风险。

嵌入层、输出层与投影层等非 Transformer 层的结构各异，计算开销与显存占用的差异较大，比如嵌入层的计算开销非常小，但是显存开销巨大，适合采用层级粒度，各自作为独立模型块参与模型划分（embedding chunk、output chunk、projector chunk），以准确反映其资源需求。

经复合粒度抽象后，某个特定的 MLLM 会被表示为一个长度为 N 的模型块列表， N 及各块的大小随具体模型的参数设置而发生变化，为后续冻结感知的模型划分提供合适粒度的决策单元。从优化视角看，模型块相当于整数线性规划中的原子单位，块越小，划分的可调维度越多，但变量数量也随之上升；块越大，变量数量减少，但可搜索空间被压缩。复合粒度的设计旨在在上述两项因素之间进行权衡。以本文实验中的 MLLM-3B 为例：编码器约 24 层，按合并因子 3 得到 8 个编码器层块，投影层为 1 个独立模型块，LLM Backbone 12 层，每层拆为注意力块与 FFN 块，共 24 块，加上嵌入块与输出块等，总块数 N 为 35。如果使用单一层粒度，MLLM-3B 会被视为 $1+24+2+1+12+1=41$ 层的模型进行划分；如果使用单一子层粒度，MLLM-3B 会被视为 $1+48+2+1+24+1=77$ 层的模型进行划分。相较按照单一层粒度或者单一子层粒度进行模型划分，使用复合粒度进行模型划分，既避免块数过多导致 ILP 决策空间过大，又避免块过粗导致划分无法细粒度平衡负载。

3.2.3 冻结感知的模型划分

在得到候选设备拓扑与模型块的列表后，规划器将“把块分配到各流水级”形式化为一个整数线性规划（ILP）问题，目标是最小化单次训练迭代时间，并显式考虑冻结对反向计算开销与梯度同步时间的影响。这一阶段的输入包括：（1）上一小节筛选出的异构 GPU 设备拓扑；（2）由复合粒度模型抽象得到的模型块序列以及每个块在不同类型设备上的执行时间、显存开销。在此基础上，规划器对每个候选拓扑分别构建并求解一个整数线性规划实例，最终选取迭代时间估计值 T^* 最小的拓扑与划分方案，作为该训练阶段的并行策略。

记块列表长度为 N ，设备组内流水级数为 P 。划分方案表示为序列 $R = (r_1, r_2, \dots, r_P)$ ，其中 r_i 为第 i 个流水级所分配的块数，满足 $\sum_{i=1}^P r_i = N$ 。对给定拓扑与 R ，可估计各流水级前向时间列表 $FT = (F_1, F_2, \dots, F_P)$ 与反向时间列表 $BT = (B_1, B_2, \dots, B_P)$ （由各块在对应设备上的执行时间累加得到）。在 1F1B 调度下，单次迭代时间 T^* 可写为前向计算、反向计算与通信时间之和，其中通信时间与各流水级是否需要梯度同步有关。若某流水级内所有块的模型参数均为冻结状态，则该流水级不产生梯度同步开销。规划器将通信时间建模为各流水级不可

被隐藏的同步时间开销的最大值，且冻结流水级对应的同步开销为 0，从而在成本模型中显式考虑冻结带来的通信开销变化。形式化地，记主流水级（所有流水级中前反向计算时间最长的流水级，主导训练的迭代时间）的索引为 K 、micro-batch 数为 M ，则迭代时间可计算为

$$T^* = \sum_{i=1}^{K-1} (F_i + B_i) + 2 \sum_{i=K+1}^P (F_i + B_i) + (M - (P - K))(F_K + B_K) + \text{COMM}, \quad (3.4)$$

其中第一项为 warm-up 阶段前 $K - 1$ 个流水级的前向与反向时间之和，第二项为 cool-down 阶段后 $P - K$ 个流水级各执行两次前向与反向的耗时，第三项为主流水级在稳定阶段对 $M - (P - K)$ 个 micro-batch 的计算耗时，COMM 为通信时间。通信时间为各流水级不可与前反向重叠的梯度同步开销的最大值：

$$\text{COMM} = \max_{i=1, \dots, P} \left(AR_i - \sum_{j=1}^{i-1} B_j \right), \quad (3.5)$$

其中 AR_i 为第 i 个流水级的梯度同步开销，可由该流水级内可训练参数的数量以及设备的通信带宽估计，本文实际中使用 NVIDIA 推出的工具 nccl-test 来进行通信时间的估计。若第 i 个流水级内所有块均为冻结，则 $AR_i = 0$ 。式中“减去前缀 $\sum_{j=1}^{i-1} B_j$ ”刻画了梯度同步与反向计算的可重叠部分。若某流水级的反向计算时间足够长，则其之后流水级的梯度同步开销可被隐藏在反向计算中，对总迭代时间不再产生影响。由此得到 T^* 关于 R 的隐式表达式（含冻结感知的通信项），规划器以最小化 T^* 为目标在约束下求解 R ，即：

$$\begin{aligned} & \underset{R=(r_1, \dots, r_P)}{\text{argmin}} && T^*(R) \\ & \text{s.t.} && \sum_{i=1}^P r_i = N, \\ & && F_i + B_i \leq F_K + B_K, \quad \forall i \neq K, \\ & && M_i^{\text{state}} + (P - i + 1) M_i^{\text{act}} \leq Q_i, \quad \forall i = 1, \dots, P. \end{aligned} \quad (3.6)$$

其中，第 i 个流水级上的模型状态显存（参数、梯度与优化器状态等，冻结参数不计入梯度与优化器状态）记为 M_i^{state} ，单个 micro-batch 在该流水级上的激活显存需求记为 M_i^{act} ，第 i 个流水级所映射设备的可用显存上界记为 Q_i 。完整性约束要求每个模型块恰好被分配一次，保证了模型被完整分配。主流水级一致性约束用于保证索引为 K 的主流水级在计算时间上支配瓶颈，从而使式 (3.4) 与 1F1B 成本模型一致。主流水级 K 的选取方式为：在给定 R 与拓扑下，计算各流水级的 $F_i + B_i$ ，取使 $F_i + B_i$ 最大的流水级索引作为 K ，以保证 1F1B 下成本模

型的正确性。显存约束限制各流水级的峰值显存占用，避免出现 OOM。规划器使用 CPLEX 整数线性规划求解器在候选拓扑上分别求解式 (3.6)，并取所有候选拓扑中使 T^* 最小的划分与拓扑作为最终并行策略。

显存估计中，各流水级占用包含模型参数与优化器状态（仅对可训练参数）、激活（与 micro-batch 大小及序列长度相关）、以及梯度与临时缓冲区。冻结流水级不存储参数的优化器状态与梯度，也不需要数据并行组之间进行梯度同步通信，故通信项中其同步开销置零。在实验环境中，单次 ILP 求解在候选拓扑上的耗时通常在毫秒级，整体规划时间在秒级可接受。若某拓扑下无可行解（如显存约束无法满足），规划器可尝试更大 TP 或更小的批次大小后重新求解。

通过将冻结状态纳入成本模型与通信项，同一 MLLM 在不同训练阶段（如 no-freeze、freeze-encoder、freeze-both）会得到不同的最优划分与设备拓扑，从而随训练进程动态适配，提升异构 GPU 集群上的训练效率。相关研究已表明，将线性流水线应用映射到异构平台并优化其时延（与流水线级映射密切相关的组合优化）在一般情况下属于 NP-hard 问题^[118]。本节采用整数线性规划在给定拓扑与块列表下求解近似最优划分。

3.3 提前终止流水线调度策略

上一节从划分与映射角度刻画了模型异构和设备异构对最优并行策略的影响，而在给定某一划分与设备拓扑后，实际训练过程仍受具体流水线调度方式的制约。流水线并行调度策略在工业界最常用的是 1F1B（One-Forward-One-Backward）：各流水级在稳定阶段交替执行前向与反向，对多个 micro-batch 形成流水线。流水线并行的一个关键特点是反向传播阶段不仅需要完成本流水级对参数的梯度计算，还必须依赖来自后续流水级的输出梯度，以继续向前序流水级回传梯度。

当 MLLMs 中部分模块被冻结时，冻结参数不参与梯度更新，但仍需为前序层计算“对输入的梯度”，从而保证未冻结参数接收到正确的反向信号。直观地看，这会削减冻结流水级的反向计算量，却不会改变“反向必须逐级串行传播”的依赖结构。因此，若仅在成本模型中简单考虑冻结层的反向 FLOPs，而不改变调度逻辑，仍然无法充分挖掘冻结带来的潜在加速空间。

MMoH 在此基础上提出提前终止流水线调度，在不改变模型正确性的前提下，识别并剪枝反向阶段中冻结前缀相关的冗余反向计算与 P2P 通信，进一步缩短迭代时间。本节将详细说明冗余产生的原因、调度器的判定规则与执行流程，以及其对成本模型与划分策略的影响。

3.3.1 冗余识别与剪枝规则

在标准反向传播中，每个流水级需完成两类计算，即对参数的梯度（用于更新该流水级的训练参数）与对输入的梯度（用于向上一流水级传递梯度）。对于冻结流水级而言，对参数的梯度不再需要计算，但在一般设置下仍须计算对输入的梯度，以便驱动前序流水级执行反向传播。然而，在部分冻结情况下，上述惯用实现会引入冗余计算和通信。

设流水级的下标依次为从 0 到 $P - 1$ ，若流水级 i 及其之前所有流水级 $(0, \dots, i - 1)$ 均为冻结流水级，则可以注意到：

- 流水级 i 的输出仅被后续流水级的前向与反向使用，而不会再被任何可训练参数使用；
- 损失函数 $\mathcal{L}$ 对冻结层中任一参数 θ_f 的梯度 $\partial\mathcal{L}/\partial\theta_f$ 始终为零，因为不需要进行参数更新；
- 对输入的梯度 $\partial\mathcal{L}/\partial x_f$ 仅用于驱动更早的冻结流水级执行反向，而这些反向本身也不会产生非零的参数梯度。

由此可以得到一个关键结论：若某流水级及其前缀均为冻结流水级，则该流水级在反向阶段所参与的“输出梯度计算与回传”对任何可训练参数的最终梯度均无影响，仅产生无效的计算和通信开销。换言之，冻结前缀中的反向传播不会再对可训练参数的更新产生影响，其梯度信息既不参与参数更新，也不影响后续非冻结流水级的梯度正确性。因此，在数学上可以安全省略这部分计算与通信，而不改变未冻结参数的梯度和更新过程。据此，MMoH 将冗余识别与剪枝规则概括为：

若存在整数 D ，使得流水级 $0, \dots, D - 1$ 均为冻结流水级，且流水级 D 为第一个包含可训练参数的流水级，则对任意 micro-batch，流水级 $0, \dots, D - 1$ 在完成前向计算后，无需再执行该 micro-batch 的反向计算及与梯度回传路径相关的流水级间 P2P 通信。

上述规则直接导致两类性能收益：一是计算剪枝，即对冻结前缀不再调度反向算子，减少了 kernel 启动与算子执行时间；二是通信剪枝，即省略对应流水级之间的通信，避免启动不必要的通信算子。由于冻结前缀常覆盖编码器与部分早期 LLM 层，在典型的 MLLMs 训练阶段（如 freeze-encoder、freeze-both）中，这两类收益对端到端迭代时间具有显著影响。

3.3.2 调度流程与实现细节

在具体实现上，提前终止调度器并不修改 1F1B 的整体组织方式，而是对每个时间步中“需要执行反向与通信的流水级集合”做动态裁剪。其执行流程可概括为三个步骤：

1. **冻结前缀检测。**在每个训练阶段开始时（例如切换冻结策略后），调度器从下标为 0 的流水级起顺序扫描，依次检查每个流水级中可训练参数的数量（即 `tensor.requires_grad` 为 True 的张量的参数数量），直至遇到第一个包含可训练参数的流水级，将其索引记为 D 。若首流水级即为可训练流水级，则 $D = 0$ ，退化为标准 1F1B 调度。
2. **时间步级调度裁剪。**对于每个时间步 t 与 micro-batch m ，调度器首先按照标准 1F1B 规则计算“应当执行前向计算与反向计算的流水级的集合”。在此基础上，对所有索引 $i < D$ 的流水级上的反向传播都直接省略，仅保留其前向计算。对应的 P2P 接收与发送操作也一并省略，避免在运行时创建多余的通信操作。
3. **资源复用与同步。**由于冻结前缀在反向传播阶段不再参与某些 micro-batch 的计算，其 GPU 资源可以更早地被下一轮 micro-batch 的前向阶段复用。调度器在实现时通过调整 CUDA stream 与 NCCL group 的启动顺序，确保这种调度方式不会破坏 1F1B 对有依赖张量的先后约束。

图 3.6 给出了提前终止调度的示例图：在 4 流水级、4 个 micro-batch 的场景下，当前两个流水级为冻结流水级时，子图 (a)、(b)、(c) 分别代表标准 1F1B 流水线调度、冻结流水级反向耗时缩短但调度策略仍为 1F1B 的情形、以及采用提前终止调度后跳过冻结前缀上对输入的梯度计算及相应 P2P 通信的情形。相较 (a) 和 (b)，(c) 由于省略了与冻结前缀相关的反向计算与通信操作，从而缩短了训练的迭代时间。

从接口层面看，MMoH 在 Megatron-LM 框架之上增添了一层轻量级调度控制，调度器即可在运行时据此自动完成冻结前缀检测与剪枝，无需用户显式编写新的反向传播算子。

3.3.3 对成本模型与划分策略的影响

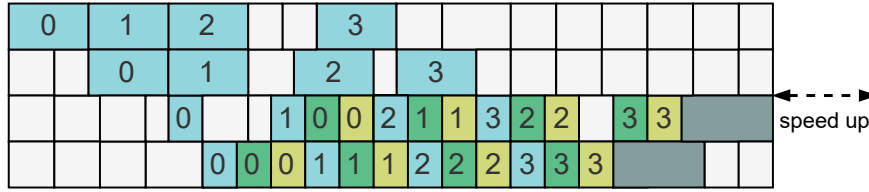
在存在连续冻结前缀（前 D 个流水级）的情况下，迭代时间 T^* 的表达式需与前述调度行为保持一致。采用提前终止调度后，前 D 个流水级在单轮迭代中仅



(a) 1F1B pipeline scheduling when no stage is frozen



(b) 1F1B pipeline scheduling when the first two stages are frozen



(c) early-termination scheduling when the first two stages are frozen



图 3.6 提前终止调度器效果示意。

有前向传播的计算，其反向与梯度同步开销视为零，因此 T^* 可改写为

$$T^* = \sum_{i=1}^D F_i + \sum_{i=D+1}^{K-1} (F_i + B_i) + 2 \sum_{i=K+1}^P (F_i + B_i) + (M - (P - K))(F_K + B_K) + \text{COMM}. \quad (3.7)$$

其中 D 为流水线首端起连续冻结的流水级的个数， K 为非冻结流水级中计算时间 $F_i + B_i$ 最大的流水级索引。COMM 仍取各流水级不可重叠的梯度同步开销的最大值，且对前 D 个流水级有 $AR_i = 0$ ，以便在 1F1B 下正确估计可重叠程度。

与未引入提前终止调度的情形相比，上式在两个方面影响了整数线性规划阶段的最优解形态。一方面，冻结前缀上的反向计算与通信被去除，使得更多计算量转移到这部分流水级不会显著增加迭代时间，从而在求解过程中自然倾向于使用性能更差的 GPU 设备进行冻结前缀流水级的计算，以减轻后续非冻结流水级的负载。另一方面，冻结前缀中的流水级不需要进行梯度同步，使得主瓶颈更有可能出现在包含大量可训练参数的非冻结流水级上，这与实际训练中“非冻结流水

级成为性能瓶颈”的经验观察是一致的。

从全局角度看，提前终止调度器与冻结感知的模型划分形成了一个闭环：划分阶段根据冻结状态与模型划分预估各流水级的前反向计算时间与通信时间，调度阶段在运行时据此执行提前终止调度以剪枝模型中冗余的计算和通信开销。反过来，调度规则的改变又会影响成本模型的具体形式，使得后续阶段的划分能够更准确地利用冻结前缀带来的冗余空间。

需要强调的是，提前终止思想并不局限于 1F1B 流水线调度本身，而是与调度策略正交。对于 Chimera^[101]、Hanayo^[102] 等近年来提出的双向或波式的新型流水线调度机制，只要其反向阶段仍遵循“从后向前逐级传播梯度”的基本模式，就可以在对应的时间步上应用同样的冻结前缀检测与剪枝逻辑。图 3.7 给出了在 Chimera（双向流水线）与 Hanayo（波式流水线）下使用提前终止调度的示意图：在前段连续冻结流水级上，同样可以提前终止反向与通信，以进一步缩短迭代时间。

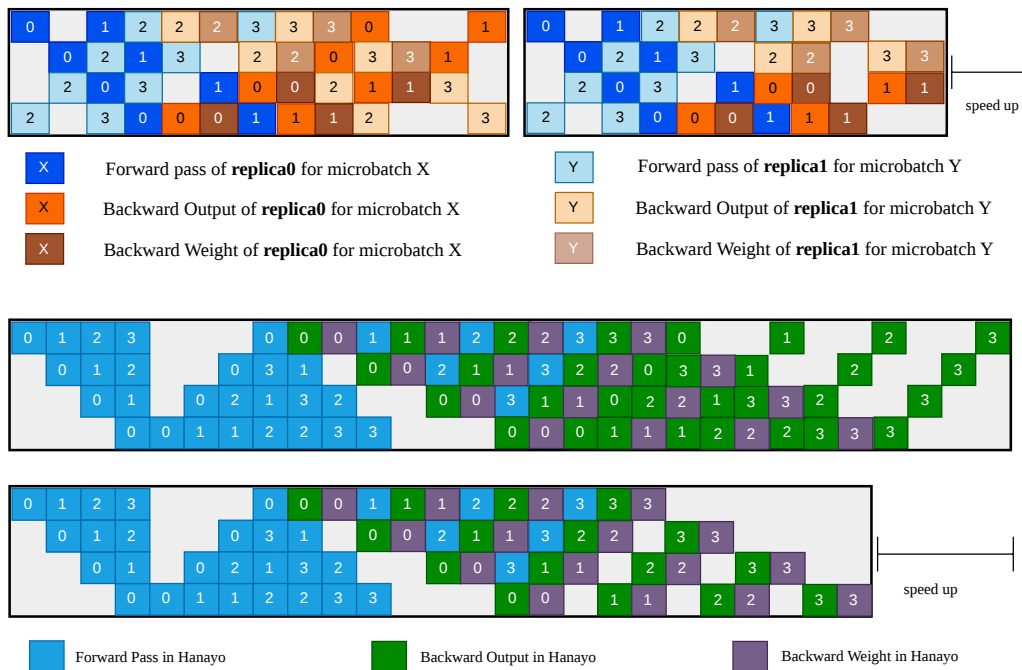


图 3.7 提前终止在其它流水线调度机制上的适用性。上：Chimera（双向流水线）；下：Hanayo（波式流水线）。

3.3.4 实现要点

MMoH 的规划器和调度器与 Megatron-LM^[36] 及 PyTorch 分布式后端高度集成，规划器在给定集群与模型配置下离线输出设备拓扑与模型映射，训练脚本据此构建各种并行策略的通信组与各设备上的子模块，调度器在运行时根据各流水

级的参数冻结状态决定是否调度反向与 P2P 通信。Profiling 在代表设备上以“块”为粒度测量前向 / 反向 FLOPs 与显存占用，结果保存在配置文件中供规划器重复使用。整数线性规划采用 CPLEX 求解器进行求解。若某拓扑下无可行解，可减小批次大小或增大 TP 后重新求解。

3.4 实验验证

本节从实验设置、实验结果与实验分析三方面对 MMoH 进行验证，并与 Megatron-LM、Whale、Espresso 等基线方法进行对比。

3.4.1 实验设置

实验在两种异构 GPU 集群上进行。**Cluster-S** 包含 4 个节点、共 16 张 GPU：1 个节点配备 4 张 NVIDIA RTX 3090，2 个节点各配备 4 张 RTX 2080 Ti，1 个节点配备 4 张 RTX 2080，集群总显存为 216 GB。**Cluster-L** 包含 8 个节点、共 32 张 GPU：4 个节点为 RTX 3090、2 个节点为 RTX 2080 Ti、2 个节点为 RTX 2080，集群总显存为 536 GB。RTX 3090 节点内 GPU 通过 PCIe 4.0 互联，RTX 2080/2080 Ti 节点内通过 PCIe 3.0 互联，节点间通过 100 Gbps InfiniBand 互联。各节点均为双路 Intel Xeon 4310@2.10 GHz、128 GB DDR4 内存。软件环境为 Ubuntu 18.04、CUDA 11.8、cuDNN 8.7.0、NCCL 2.19.3 与 PyTorch 2.2，与集群运维环境一致。表 3.1 总结了两种异构集群的节点与显存配置。

表 3.1 实验用异构 GPU 集群配置

集群类型 (GPU 数)	节点编号	GPU 型号 (每节点张数)	总显存 (GB)
Cluster-S (16)	1	RTX 3090 (4)	216
	2, 3	RTX 2080 Ti (4)	
	4	RTX 2080 (4)	
Cluster-L (32)	1-4	RTX 3090 (4)	536
	5, 6	RTX 2080 Ti (4)	
	7, 8	RTX 2080 (4)	

为验证 MMoH 在不同集群规模与模型结构下的表现，本文构建两个 MLLMs。**MLLM-3B** 由 ViT 编码器（约 0.2B 参数、24 层）、MLP-S 投影层（约 73M 参数）与 Mistral 2.8B Backbone（12 层）组成；**MLLM-12B** 由 InternViT 编码器（约 6B 参数、45 层）、MLP-L 投影层（约 149M 参数）与 Yi 6B Backbone（20 层）组成。MLLM-3B 在 Cluster-S 上训练，MLLM-12B 在 Cluster-L 上训练，以与集群总显存规模相匹配。训练数据采用 LLaVA Visual Instruct 数据集（约 60 万条图文指令数

据)，用于模态对齐与指令微调，训练时采用与 LLaVA 类似的采样与数据混合策略^[11]。关键超参数方面，MLLM-3B 的全局批次大小设置为 64、最大序列长度设置为 1024；MLLM-12B 的 global batchsize 设置为 32、最大序列长度设置为 1024。表 3.2 汇总两种 MLLM 各模块的参数量与层数配置。

表 3.2 实验用 MLLM 模块配置（模型参数）

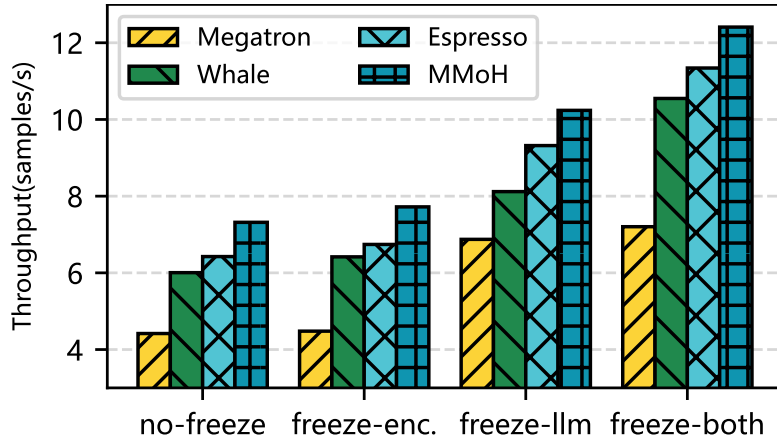
MLLM	模块	类型	参数量	层数
MLLM-3B	ViT	编码器	0.2B	24
	MLP-S	投影层	73M	2
	Mistral	LLM Backbone	2.8B	12
MLLM-12B	InternViT	编码器	6B	45
	MLP-L	投影层	149M	2
	Yi	LLM Backbone	6B	20

对比方法包括：（1）**Megatron-LM**^[36]，面向同构集群的并行训练框架，为缓解异构环境下的 OOM，本文在其基础上采用与 **Whale** 一致的设备拓扑组织方式；（2）**Whale**^[51]，按显存容量组织设备并按算力对模型做层级划分；（3）**Espresso**^[77]，引入计算—显存比（CMR）度量，在满足显存约束的前提下联合优化流水级放置与通信开销。在设备组内采用流水线并行（PP），在组间使用数据并行（DP）与张量并行（TP）。端到端对比时，每种冻结场景均在可行范围内搜索并行配置，初始设定为 DP=4、TP=1。若发生 OOM，则增大 TP、相应减小 DP。当单节点允许的最大张量并行度（本文为 TP=4）下仍无法容纳训练状态时，该配置记为无效并在图中标注 OOM。评价指标为训练吞吐（samples/seconds）。每种配置执行 10 次迭代预热，随后在 50 次稳定计时迭代上统计平均吞吐并作为报告值。

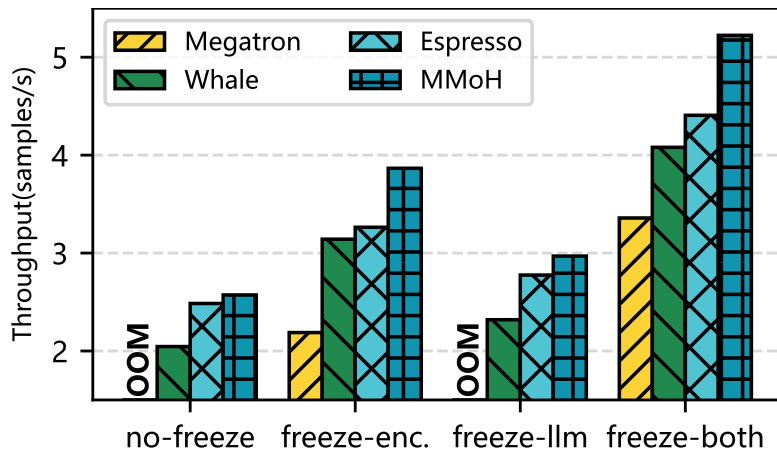
3.4.2 实验结果

在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种冻结场景下，MMoH 在两种集群与两种模型上均取得最高吞吐，且在不同场景下均能自动调整设备拓扑与划分策略。图 3.8 给出了两种集群与两种 MLLMs 下的端到端吞吐对比，柱长表示吞吐（samples/seconds），OOM 表示该配置下发生显存溢出无法训练。

在小规模异构集群 Cluster-S 上对 MLLM-3B 的测试中，Megatron-LM 于四种冻结设定下均处于吞吐下界。其原因在于默认划分仍以同构集群为隐含前提，难以使各流水级的有效工作量与异构设备的算力—显存供给相匹配；即便改用 **Whale** 的设备排列以降低 OOM 风险，负载失衡现象依旧显著。为满足显存约束，部分场景不得不将张量并行度提升至 TP=4，由此带来的高频集合通信与额外同步使得端到端样本吞吐进一步受限。相较之下，**Whale** 通过显存容量主导的拓扑组织



(a) MLLM-3B 在 Cluster-S 上



(b) MLLM-12B 在 Cluster-L 上

图 3.8 四种冻结场景下 MMoH 与基线方法在两种异构集群上的训练吞吐对比 (samples/seconds)。柱越长表示吞吐量越高，OOM 表示显存溢出。

并结合算力信息进行层级划分，Espresso 进而引入计算—显存比 (CMR) 以联合权衡流水级映射与通信开销，二者均较 Megatron-LM 取得明显提升。MMoH 在此基础上显式纳入需求驱动的拓扑筛选与复合粒度模型划分，进一步取得性能提升。freeze-encoder 时 MMoH 相比 Espresso 提升约 $1.15\times$ ；freeze-llm 时相比 Whale 提升约 $1.26\times$ ；freeze-both 时相比全部基线提升约 $1.08\times$ – $1.72\times$ 。整体而言，冻结范围越大，冻结前缀上可被识别的冗余反向计算与流水级间通信越多，MMoH 经由冻结感知划分与提前终止调度所转化的端到端收益亦更为显著。

在更大规模的异构集群 Cluster-L 上训练 MLLM-12B 时，no-freeze 与 freeze-llm 两种设置下 Megatron-LM 均会因粗粒度的模型划分导致显存溢出，即使流水线最前端的流水级的设备配备较大显存，调整 TP 与 DP 后仍无法避免 OOM。Whale 将编码器拆分到多个流水级以避免首流水级过载，从而在四种冻结场景下均可训练；Espresso 进一步联合优化负载与通信。MMoH 在所有冻结场景下保持最高吞

吐：no-freeze 下相对 Espresso 与 Whale 分别约 $1.04\times$ 与 $1.26\times$ ；freeze-llm 下相对 Whale 与 Espresso 分别约 $1.26\times$ 与 $1.11\times$ ；在 freeze-encoder 与 freeze-both 下，得益于提前终止调度器与冻结感知划分，相对 Megatron-LM、Whale、Espresso 最高分别可达约 $1.77\times$ 、 $1.28\times$ 与 $1.19\times$ 。上述结果说明，MMoH 的复合粒度划分与冻结感知策略能够在不同规模与冻结阶段下稳定提升异构集群上的训练效率。

3.4.3 实验分析

Whale 与 Espresso 在扩展至多模态大语言模型时仍沿用统一层粒度来进行模型划分，MMoH 则采用复合粒度。具体而言，对 MLLM-3B，编码器侧多层 Transformer 合并为编码器层块，LLM Backbone 每层再拆分为注意力块与 FFN 块，嵌入层、投影层与输出层等亦各自独立成块；对 MLLM-12B，InternViT 编码器单层算存占用显著增大，故采用按层分块，其余模块的处理方式与 MLLM-3B 类似。因而 MMoH 在表 3.3 中给出的各流水级块数总和与 Whale、Espresso 的按层划分的总和并不相同，MMoH 反映了复合粒度下更细的调度单元划分。

表 3.3 对比了三种方法在两种模型与四种冻结场景下的设备拓扑、划分结果及相对 Whale 的吞吐加速比（Megatron-LM 在部分场景 OOM，故表中 Baseline 取 Whale）。表中 A、B、C 分别表示 NVIDIA RTX 3090、RTX 2080 Ti 与 RTX 2080 GPU。在不同冻结场景下，MMoH 得到的设备拓扑与模型划分策略随冻结状态变化。freeze-encoder 时模型前段的计算与显存需求下降，MMoH 会将性能较差的 GPU 置于初始的流水级，当训练状态切换为 freeze-llm 时，模型后段的算存需求下降，设备顺序相应调整，来匹配模型在不同冻结场景下的算存需求。该动态调整能力是现有方法所不具备的，也是 MMoH 在多种场景下均能取得较优性能的重要原因。

图 3.9 给出了 MLLM-12B 在 no-freeze 与 freeze-llm 两种冻结情况下各流水级的单次迭代计算时间分布。在 no-freeze 设置下，Whale 将高性能 GPU 优先映射至流水线前段的策略，与多模态模型在各流水级上的峰值负载时空分布并不总是对齐，表现为部分流水级耗时偏高，负载均衡差；Espresso 与 MMoH 则更倚重将大显存设备与后段高负载相匹配，并在映射上兼顾通信与算存比。MMoH 进一步在中后段流水级引入子层粒度的细粒度分块，使各流水级用时分布更为集中、极值更低，迭代时间受单一瓶颈支配的程度随之减弱。在 freeze-llm 设置下，Whale 需要 TP=4 才能避免 OOM，而 Espresso 与 MMoH 在 TP=2 下即可正常训练。MMoH 则通过将相对较低规格设备置于初始流水级、前移更多编码器计算等方式，压低后段流水级的计算开销峰值，获得更好的负载均衡效果，从而在端到端迭代时间上更优于 Espresso。

表 3.3 不同冻结场景下的模型划分对比（加速比 Baseline 为 Whale）

冻结模式	Whale（层粒度）		Espresso（层粒度）		MMoH（复合粒度）	
	MLLM-3B	MLLM-12B	MLLM-3B	MLLM-12B	MLLM-3B	MLLM-12B
no-freeze	[A,B,B,C]	[A,A,A,A,B,B,C,C]	[B,B,C,A]	[B,B,C,C,A,A,A,A]	[B,A,B,C]	[B,B,C,C,A,A,A,A]
	[30,3,3,2]	[15,15,17,9,3,3,3,2]	[26,3,2,7]	[7,7,6,6,15,11,8,7]	[11,14,6,4]	[7,7,6,6,18,16,13,15]
	1.00×	1.00×	1.07×	1.22×	1.22×	1.26×
freeze-encoder	[A,B,B,C]	[A,A,A,A,B,B,C,C]	[B,B,C,A]	[B,B,C,C,A,A,A,A]	[C,A,B,B]	[C,C,B,B,A,A,A,A]
	[31,3,3,1]	[38,11,5,5,2,3,2,1]	[28,3,2,5]	[8,8,6,6,22,13,13,12]	[12,13,6,4]	[12,11,11,11,11,11,10]
	1.00×	1.00×	1.05×	1.04×	1.20×	1.23×
freeze-llm	[A,B,B,C]	[A,A,A,A,B,B,C,C]	[B,B,C,A]	[B,B,C,C,A,A,A,A]	[B,B,A,C]	[C,C,B,B,A,A,A,A]
	[27,4,4,3]	[13,13,13,13,4,4,4,3]	[14,14,3,7]	[3,3,3,3,13,13,17,12]	[7,8,16,4]	[3,3,5,5,13,14,23,22]
	1.00×	1.00×	1.15×	1.20×	1.26×	1.33×
freeze-both	[A,B,B,C]	[A,A,A,A,B,B,C,C]	[B,B,C,A]	[B,B,C,C,A,A,A,A]	[B,A,B,C]	[A,B,B,C,C,A,A,A]
	[30,3,3,2]	[38,9,5,5,3,3,2,2]	[28,2,2,6]	[18,5,4,4,16,7,7,6]	[14,14,4,3]	[36,4,5,3,4,14,12,10]
	1.00×	1.00×	1.08×	1.20×	1.18×	1.28×

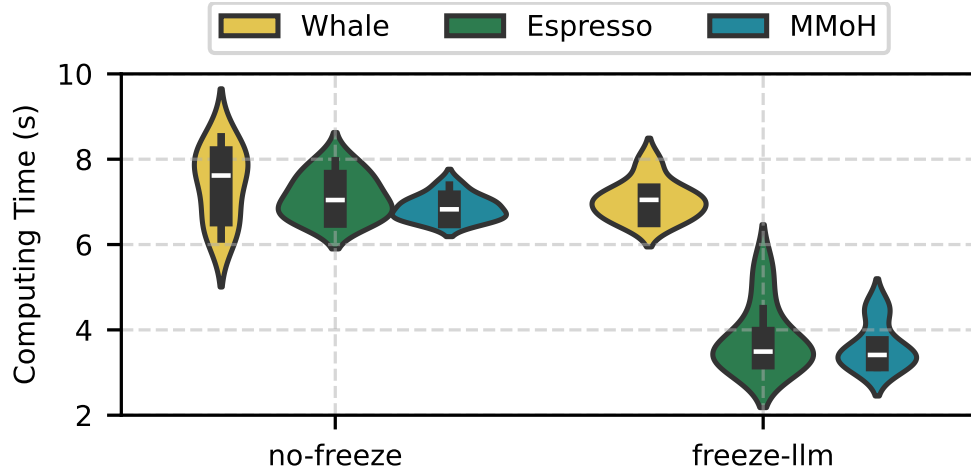


图 3.9 MMoH 与 Whale、Espresso 在 MLLM-12B 上、两种冻结设置下的负载均衡对比。分布越集中表示划分越均衡，高度越低表示迭代时间越短。

进一步地，为验证提前终止调度器的有效性，本文将 MMoH 的调度器接入 Espresso（记为 Espresso+），或从 MMoH 中移除该调度器（记为 MMoH-），在 freeze-encoder、freeze-both 两种冻结场景下进行消融实验（Megatron-LM 与 Whale 在该对比中性能不佳，故主要与 Espresso 系列进行对照）。图 3.10(a) 报告端到端吞吐，(b) 以 freeze-encoder 为例给出各流水级计算开销分布。在 freeze-encoder 下，原始 Espresso 吞吐最低，MMoH- 相对 Espresso 约提升 6.4%，表明仅凭冻结感知规划器即可带来增益。Espresso+ 相对 Espresso 约提升 11.3%，说明调度器单独接入异构框架亦有效。完整 MMoH 相对 Espresso 约提升 19.2%，相对 MMoH- 约提升 11.4%，体现规划器与调度器的叠加收益。在 freeze-both 下，仅原始 Espresso 仍需 TP=4，其余配置均在 TP=2 下训练。完整 MMoH 相对 MMoH- 与 Espresso+ 分别约提升 8.3% 与 10.0%，相对原始 Espresso 吞吐量提升约 19.0%。结合图 3.10(b)

可见，MMoH 将更多计算负载分配至流水线前段连续冻结流水级，这些流水级在反向阶段可被提前终止，从而减轻后段可训练流水级的负担并缩短整体迭代时间。上述结果表明，提前终止调度器能够在不改变未冻结参数梯度语义的前提下削减冗余计算与通信，且与冻结感知规划器协同具有显著增益与较好的可移植性。

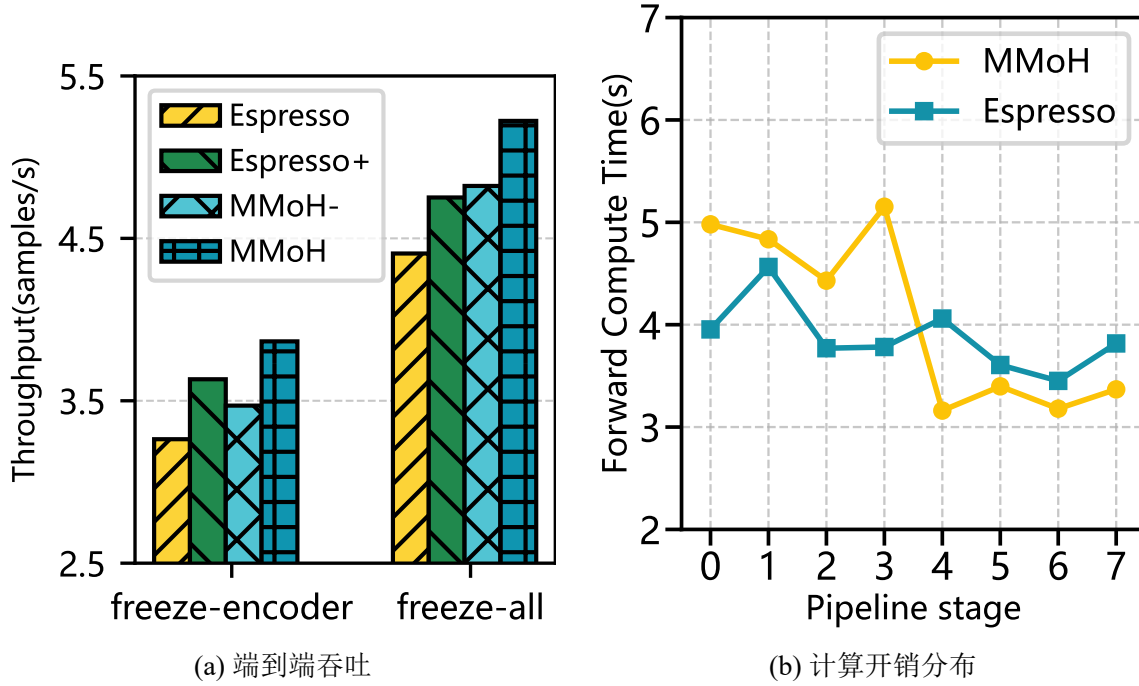


图 3.10 提前终止调度器消融。Espresso+ 为接入该调度器的 Espresso，MMoH- 为去掉该调度器的 MMoH。

3.5 本章小结

本章提出了面向异构 GPU 集群的多模态大语言模型高效混合并行框架 MMoH。针对 MLLMs 混合模块结构与分阶段冻结训练共同引起的计算与显存需求非均匀性及其动态变化的特点，本章将训练过程形式化为由瓶颈流水级主导的迭代时间优化问题，并显式考虑冻结技术对于反向计算与通信开销的影响。在此基础上，MMoH 通过规划器与提前终止调度器的协同实现训练阶段感知的划分与调度：前者结合需求驱动的设备拓扑识别、复合粒度模型抽象与冻结感知的整数线性规划划分，在给定集群与模型配置下自动生成与当前训练阶段匹配的混合并行策略；后者在连续冻结前缀流水级跳过不必要的反向计算与 P2P 通信，在保持模型收敛性的前提下进一步提升系统吞吐。在两类异构集群（Cluster-S、Cluster-L）与两种规模 MLLMs（MLLM-3B、MLLM-12B）上的实验结果表明，相较于 Megatron-LM、Whale 与 Espresso 等基线方法，MMoH 可获得最高约 $1.77\times$ 的加速，并在不同冻结场景下自适应调整设备拓扑与划分策略；负载分布对比与调

度器消融进一步表明，复合粒度划分有助于缓解瓶颈流水级对迭代时间的支配效应，提前终止机制则可在不改变未冻结参数梯度语义的前提下显著削减冗余计算与通信。

第四章 多模态大语言模型在异构 GPU 集群上的重计算技术

异构 GPU 设备的显存容量存在显著差异，仅依赖模型划分与流水线调度策略，仍难以在各流水级同时满足大批次与长序列带来的激活值显存需求，因此显存仍是制约端到端可训练配置的关键因素。重计算技术缓解了一个问题，但是为同构集群设计重计算策略往往难以充分适配异构 GPU 集群中非均匀的算力与显存约束。针对上述问题，本章提出 HR-MMoH (heterogeneous recomputation for training multimodal large language models on heterogeneous GPU clusters)。该方法在 MMoH 的基础上引入流水级差异化的异构重计算配置，与第三章共同构成“划分—调度—重计算多级优化”链路。本章首先阐述异构重计算的研究背景与问题动机，随后介绍 HR-MMoH 框架的总体结构及具体实现，最后介绍实验结果并进行实验分析。

4.1 引言

4.1.1 问题与动机

异构 GPU 集群内设备显存大小并不相同，显存较小的 GPU 设备仍难以支持更大批次大小和更长序列长度的模型训练，显存仍是异构 GPU 集群训练的主要瓶颈。重计算技术通过在前向传播时丢弃部分中间激活、在反向传播时按需重算，以计算开销换取显存空间，缓解了一个问题。

然而现有的重计算策略通常建立在同构集群的前提下，即在一次训练中令所有 GPU 设备共用同一套重计算配置。虽然 AdaPipe^[105] 和 Mist^[117] 已尝试将重计算配置与流水线并行联合优化，但其方案仍主要面向同构集群设计，尚未充分利用异构 GPU 设备间的显存与算力差异。这种为同构 GPU 集群设计重计算配置在异构 GPU 集群中的适用性显著下降。对显存不足的流水级而言，若不采用重计算或者采用较少的重计算，则中间激活仍会占据大量显存空间，导致显存溢出 (OOM) 的风险上升；对于显存压力小的流水级，若被强制使用统一的重计算配置，则会在显存充足的情形下仍引入冗余重计算，进而造成计算资源浪费，导致迭代时间延长。

针对上述问题，本章在 MMoH 已给出的模型划分与流水线调度方案之上提出 HR-MMoH，为各流水级分别配置重计算策略。显存压力大的流水级采用更激进的重计算策略以减少驻留的激活值，显存压力较小的流水级则尽量减少不必要的重计算，从而在显存溢出风险与额外计算开销之间实现更细粒度的权衡。

4.1.2 HR-MMoH 框架概览

HR-MMoH 以第三章 MMoH 的并行策略规划器与提前终止调度器的输出为前置条件：在确定的设备拓扑、流水级映射与冻结感知调度规则下，进一步为每个流水级求解一组重计算配置，使各流水级在自身显存约束下尽可能减少冗余重计算。

图 4.1 从系统层次上刻画了 HR-MMoH 与 MMoH 的衔接关系与端到端数据流：输入侧由 MLLM 结构及训练阶段配置与异构集群资源描述共同构成。MMoH 据此求解并输出设备拓扑、模型划分以及冻结感知的流水线调度等并行执行方案。在上述执行骨架已确定的前提下，HR-MMoH 进一步在训练框架的运行路径中，按流水级注入差异化的重计算配置，使重计算强度与各级显存余量及算力供给相匹配。与 Megatron-LM 默认的全局统一重计算策略相比，HR-MMoH 显式利用各流水级显存余量与算力差异，避免部分设备显存受限而部分设备重计算冗余的矛盾。与同构假设下的重计算配置相比，该框架将流水级维度的重计算配置与 MMoH 的规划结果对齐，便于在异构 GPU 集群上实现与扩展。

本章其余章节展开如下：第 4.2 节给出重计算策略的形式化描述、定量分析与本文实现要点，第 4.3 节报告实验设置、实验结果与实验分析，第 4.4 节对本章工作进行总结。

4.2 异构 GPU 重计算技术

4.2.1 现有的重计算策略与定量分析

Megatron-LM^[35, 36, 71] 是大模型分布式训练广泛使用的框架之一，其对重计算技术进行了全面的支持，用户可以在训练脚本中通过 `recompute-granularity`、`recompute-method`、`recompute-num-layers`、`recompute-modules` 等参数进行重计算配置。表 4.1 归纳了与重计算配置直接相关的主要可配置参数及其含义。

整层重计算（full） 以 Transformer 模型层为重计算单元，前向传播后只在每个模型层边界保留输入，反向传播时需要重新进行一遍完整的前向计算来恢复中间激活值。整层重计算下由 `recompute_method` 与 `recompute_num_layers` 共同规定“隔多少层留一个激活”。`recompute_method` 有两种可配置项：

- **uniform**：将当前流水级内的所有层均匀分成多个模型块，每个模型块包含 `recompute_num_layers` 层。每个模型块的输入激活值被保存，模型块反向时整个模型块先重计算再反传。模型块的个数少则显存更省，但单次重计算层数较多。
- **block**：在当前流水级内，仅对前 `recompute_num_layers` 层逐层重计算；其后各层不重计算，中间激活照常保留。

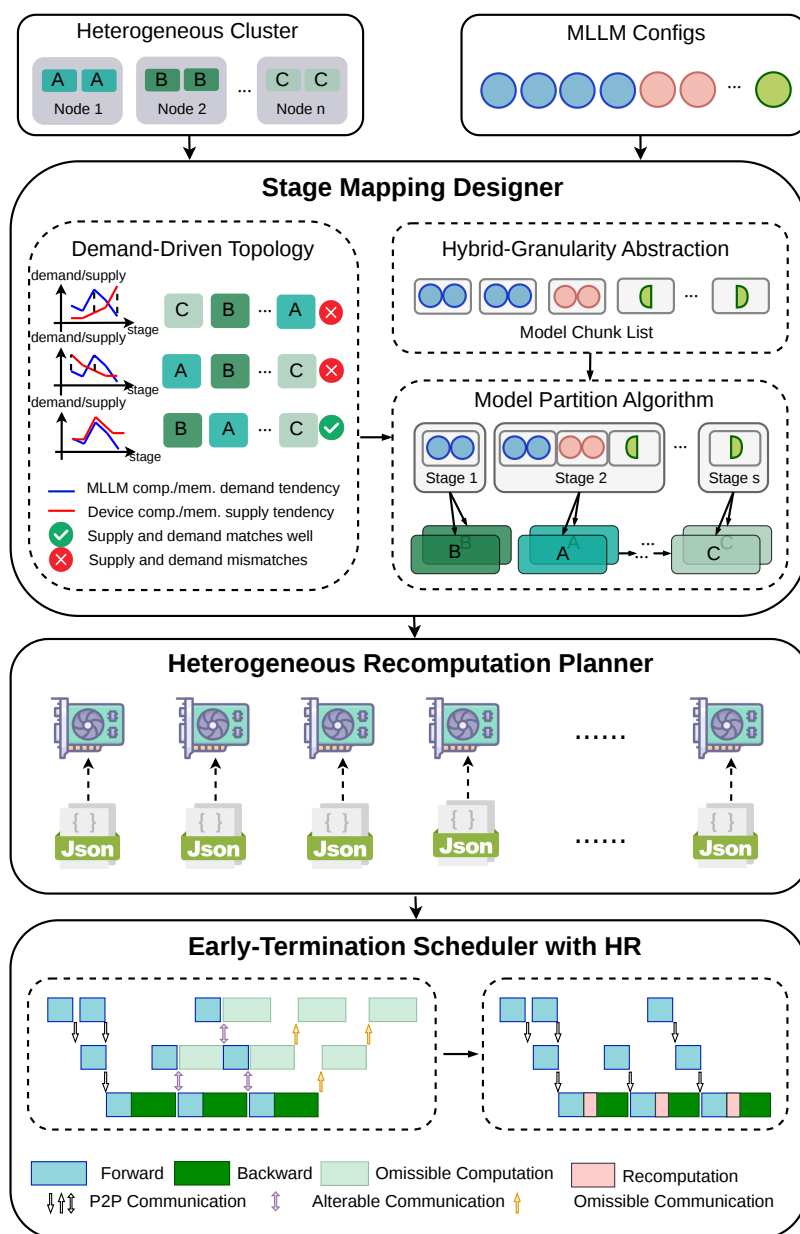


图 4.1 HR-MMoH 框架总览及其与 MMoH 的衔接关系。MMoH 负责划分与调度；HR-MMoH 在各流水级上配置差异化的重计算策略，以进一步适配异构 GPU 设备的显存与算力。

选择重计算 (selective) 在 Transformer 模型层内以子模块为粒度执行重计算，即仅对指定的子模块在反向阶段进行重计算，其余模块仍按常规方式保留激活，具体需要重计算的子模块由 `recompute_modules` 指定。采用选择重计算时，`recompute_method` 与 `recompute_num_layers` 均应设为 `None` (Python 空值，表示该参数不适用)。值得注意的是，选择重计算的配置将统一作用于所有流水级中的各层，即所有流水级中的各层都使用相同的子模块列表与重计算策略。

表 4.1 Megatron-LM 重计算相关配置参数

参数	类型 / 取值	说明
<code>recompute_granularity</code>	<code>none</code> <code>full</code> <code>selective</code>	重计算粒度。 <code>none</code> 表示不重计算； <code>full</code> 对整层做重计算； <code>selective</code> 仅对 <code>recompute_modules</code> 中列出的子模块重计算。
<code>recompute_method</code>	<code>None</code> <code>uniform</code> <code>block</code>	仅在 <code>full</code> 粒度时有效：如何划分“哪些层”做重计算。 <code>uniform</code> 为均匀分块， <code>block</code> 为仅对每流水级前若干层做重计算。 <code>selective</code> 粒度时必须为 <code>None</code> 。
<code>recompute_num_layers</code>	<code>None</code> 正整数	仅在 <code>full</code> 粒度时有效。 <code>uniform</code> 时表示每个 <code>chunk</code> 的层数； <code>block</code> 时表示每个 <code>stage</code> 内做重计算的层数。 <code>selective</code> 粒度时必须为 <code>None</code> 。
<code>distribute_saved_activations</code>	布尔	为 <code>True</code> 时，将重计算保存的中间激活在张量并行组内分片存储，仅在 <code>full</code> 粒度且非序列并行时有效。
<code>recompute_modules</code>	字符串列表	仅用于 <code>selective</code> 粒度。要重计算的子模块名列表（见下文）。默认 <code>["core_attn"]</code> 。

子模块可选项由训练脚本中的 `recompute_modules` 参数给出，为 Python 中的列表格式，未显式配置时默认使用 `[core_attn]`。稠密模型里常见的“节约显存”的组合是 `[core_attn, mlp, layernorm]`。表 4.2 列出选择重计算中常用模块名以及代表的具体含义。

表 4.2 Megatron-LM 选择重计算下可配置的子模块

模块名	含义
<code>core_attn</code>	核心注意力（QKV→Attention→输出）
<code>mlp</code>	稠密 MLP（非 MoE）
<code>layernorm</code>	输入 LayerNorm 与 MLP 前 LayerNorm
<code>moe</code>	MoE 模型层的整块 MoE 层（router + experts + combine）
<code>shared_experts</code>	MoE 的共享专家
<code>moe_act</code>	MoE 专家内部激活（如 SiLU）与专家权重

现有策略在同一训练任务中对各流水级采用统一的重计算粒度、方法、层数与模块，实质上默认了使用同构 GPU 设备。在异构 GPU 场景下，这一假设会带来两个问题。首先，显存受限的流水级仍可能 OOM，而显存充裕的流水级则承担不必要的重计算开销，导致计算资源浪费。其次，各流水级的计算开销增幅近似一致，计算能力较弱的 GPU 更易成为关键路径上的性能瓶颈，进而限制整体的训练吞吐量。

为比较各种重计算策略下激活值占用的显存空间大小，本文在固定输入形状与 BF16 数据类型的前提下，对单层稠密 Transformer 层反向计算所需中间激活的显存开销作逐项定量分析。设 Transformer 模型层的输入张量 X 形状为 $[B, S, H]$ (B 为批次大小， S 为序列长度， H 为隐藏维大小)，激活值使用 BF16 格式，每元素占用 2 字节，层归一化方式使用最常用的 Pre-LayerNorm。输入张量 X 经 LayerNorm 后进入注意力子层，残差连接后再经过 LayerNorm 进入 MLP 子层（线性层 $\rightarrow$ GELU 激活函数 $\rightarrow$ 线性层 $\rightarrow$ Dropout 层），再残差连接得到层输出。其中，注意力头数记为 A ，MLP 中间维度大小取普遍使用的 $4H$ 。

第三章中流水线并行度（流水级数）记为 P ，本章中流水级数亦记为 P ，与第三章保持一致。表 4.3 汇总本章其余主要符号。

(1) **不重计算 (none)**，即所有的中间激活值都需要被保留。

在注意力子层中， $Q/K/V$ 共享输入为 $2BSH$ ，计算得到的 $Q/K/V$ 大小均为 $2BSH$ ，共计 $6BSH$ 。计算 QK^T 后进行 softmax 的中间结果为 $2BAS^2$ ，Dropout 层的 mask 矩阵形状为 BAS^2 ，Dropout 层的输出为 $2BAS^2$ 。最后与 V 相乘的中间结果为 $2BSH$ ，最后的 Dropout 的 mask 矩阵为 BSH ，合计显存开销为

$$N_{\text{attn}} = 11BSH + 5BAS^2. \quad (4.1)$$

其中 $5BAS^2$ 来自 QK^T 、softmax 与 dropout 等，随 S 增大而关于序列长度二次增长，长序列下往往占主导。图 4.2 示意重计算策略为 none 时注意力子层需要存储的中间激活值以及每个激活值占据的显存空间大小。

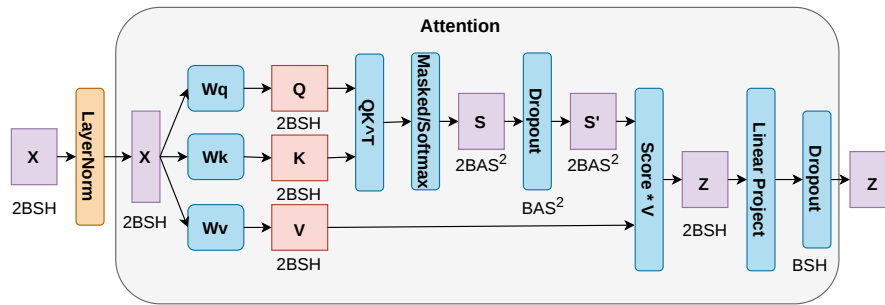


图 4.2 注意力子层中间激活示意图

表 4.3 本章主要符号说明

符号	含义
B	批次大小 <code>batchsize</code>
S	输入序列长度 <code>sequence length</code>
H	模型隐藏维度 <code>hidden size</code>
A	注意力头数。 <code>head num</code>
X	输入张量，形状为 $[B, S, H]$ 。
$\mathbf{h}_1$	注意力子层与残差连接后的中间张量，形状为 $[B, S, H]$ 。
N_{attn}	单个注意力子层为反向传播保留的中间激活总量。
N_{mlp}	单个 MLP 子层为反向传播保留的中间激活总量。
N_{ln}	单个 LayerNorm 子层为反向传播保留的中间激活总量。
N_{none}	不采用重计算 (<code>none</code>) 时，单层 Transformer 需保留的中间激活总量。
$N_{\text{selective}}^{\text{core_attn}}$	仅对 <code>core_attn</code> 使用 <code>selective</code> 重计算时，单层需保留的中间激活总量。
$N_{\text{selective}}^{\text{core_attn+mlp}}$	对 <code>core_attn</code> 与 <code>mlp</code> 使用 <code>selective</code> 重计算时，单层需保留的中间激活总量。
$N_{\text{selective}}^{\text{core_attn+mlp+ln}}$	对 <code>core_attn</code> 、 <code>mlp</code> 与 <code>layernorm</code> 使用 <code>selective</code> 重计算时，单层需保留的中间激活总量。
N_{full}	采用 <code>full</code> 重计算时，单层需保留的中间激活总量。

在 MLP 子层中，中间维度 $4H$ ，第一个线性层的输入为 $2BSH$ ，需要保存的输出为 $8BSH$ ，经过激活函数 GELU 后第二个线性层的输入为 $8BSH$ ，最后的 Dropout 层的 mask 矩阵为 BSH ，合计显存开销为

$$N_{\text{mlp}} = 19BSH. \quad (4.2)$$

图 4.3 对应给出 MLP 子层在重计算策略为 `none` 时的激活值分解图。

两个 LayerNorm 层（注意力子层前、MLP 前各一个）各保存输入 $2BSH$ ，合计显存开销为

$$N_{\text{ln}} = 4BSH. \quad (4.3)$$

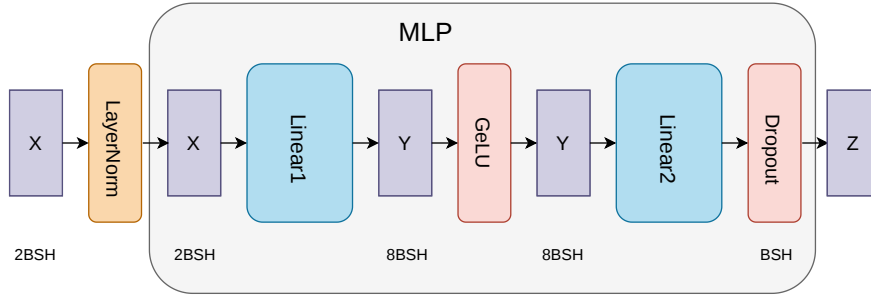


图 4.3 MLP 子层中间激活示意

综上所述，重计算策略为 **none** 时单个 Transformer 层需保存的中间激活总量为

$$N_{\text{none}} = N_{\text{attn}} + N_{\text{mlp}} + N_{\text{ln}} = 34BSH + 5BAS^2. \quad (4.4)$$

当 S 较大时， $5BAS^2$ 这一项通常显著超过与 BSH 线性相关的部分，从而主导单层需常驻的中间激活显存规模。

(2) 选择重计算 (selective)。 在该模式下，可对所选子模块只保留其输入激活值，反向传播时先在子模块内重新进行前向计算，再进行反向传播计算梯度。未列入 `recompute_modules` 的子模块，其行为与 **none** 一致，仍全量保存中间激活。

考虑仅对 `core_attn` 启用重计算， $QK^\top$ 、`softmax` 与 `mask` 等注意力内部激活不再常驻显存，但仍需保留子模块边界输入。此时单层常驻激活由 $34BSH + 5BAS^2$ 降至约 $23BSH$ 。若仅对 MLP 启用重计算，MLP 内部约 $19BSH$ 的中间激活推迟到反向阶段重算，则常驻激活为 $15BSH + 5BAS^2$ 。若仅对 MLP 与 LayerNorm 同时启用重计算，则进一步释放两处 LayerNorm 输出，常驻激活为 $11BSH + 5BAS^2$ 。若在 `core_attn` 的基础上叠加 LayerNorm 重计算（即 `ln+attn`）时，常驻激活为 $19BSH$ 。若继续同时对 MLP 启用重计算，则常驻激活降至 $6BSH$ ；再将 LayerNorm 纳入后，显存开销进一步降至 $4BSH$ 。

相对于 **none** 情形，记

$$N_{\text{selective}}^{\text{mlp}} = 15BSH + 5BAS^2, \quad (4.5)$$

$$N_{\text{selective}}^{\text{mlp+ln}} = 11BSH + 5BAS^2, \quad (4.6)$$

$$N_{\text{selective}}^{\text{core_attn}} = 23BSH, \quad (4.7)$$

$$N_{\text{selective}}^{\text{core_attn+ln}} = 19BSH, \quad (4.8)$$

$$N_{\text{selective}}^{\text{core_attn+mlp}} = 6BSH, \quad (4.9)$$

$$N_{\text{selective}}^{\text{core_attn+mlp+ln}} = 4BSH. \quad (4.10)$$

表示使用选择重计算策略时不同模块组合下常驻激活规模。相对无重计算策略 **none**，**selective** 可以选择性丢弃指定模块的中间激活，只保留子模块边界输入，可

以对显存占用进行更精细的控制。

(3) **整层重计算 (full)**。以整层为单元，前向只保留模型层的输入 X ，反向传播时先整层重新前向计算后再反向传播。于是

$$N_{\text{full}} = 2BSH. \quad (4.11)$$

常驻显存最少，但是反向传播时会进行完整的前向计算，引入的重计算开销最大。

表 4.4 汇总了不采用重计算、选择重计算不同子模块与整层重计算时单层 Transformer 需保存的中间激活总量。

表 4.4 单层 Transformer 中间激活量

策略	需保存的激活 (元素个数)	说明
none	$34BSH + 5BAS^2$	整层所有中间激活均保留
selective [core_attn]	$23BSH$	仅注意力内部重计算, MLP 与 LN 仍保留
selective [mlp]	$15BSH + 5BAS^2$	仅 MLP 内部重计算, 注意力与 LN 仍保留
selective [mlp, layernorm]	$11BSH + 5BAS^2$	MLP 与 LN 重计算, 注意力内部激活仍保留
selective [core_attn, layernorm]	$19BSH$	注意力与 LN 重计算, MLP 内部激活仍保留
selective [core_attn, mlp]	$6BSH$	注意力与 MLP 内部重计算, 仅边界与 LN 保留
selective [core_attn, mlp, layernorm]	$4BSH$	三者均重计算, 仅两个子模块边界的输入保留
full	$2BSH$	仅保存层输入, 反向时整层重计算

4.2.2 异构 GPU 重计算策略设计

HR-MMoH 将 Megatron-LM 的重计算配置抽象为可按流水级独立配置的离散决策，在保持各流水级前向反向数学性质不变的前提下，通过调节重计算粒度与子模块集合，在算力与显存之间进行权衡。本节依次给出显存分解与优化形式、基于剖析的可行配置生成方法、结合冻结状态与流水级位置的启发式配置选择方法、多路分支搜索伪代码，以及与 MMoH 框架的结合方式。

本节与算法 4.1 中， P 表示流水线并行度（即流水级个数，与第三章一致）， p 表示流水级的编号。各流水级可用的显存上界记为 Q_p ，候选重计算策略个数记为 G （本文实现中取 $G=7$ ，实际组合方式不限于本文使用的 7 种）。

在 MMoH 规划器已完成模型划分与设备映射的前提下，流水级 p 映射至设备 d_p ，并由若干模型块组成。单次训练迭代处于稳定调度阶段时，设备 d_p 上的峰值显存可写为四项之和：（1）模型参数；（2）梯度；（3）优化器状态（如 Adam 优化器需要 FP32 格式的动量与方差）；（4）中间激活（含流水线并行在 1F1B 等调度下需暂存的激活，其峰值与 micro-batch 配置、流水级在流水线中的位置及重计算策略均相关）。在给定划分、优化器类型、数据类型及是否启用张量并行等训练配置后，（1）–（3）不随重计算配置 θ_p 改变，冻结参数不用存储梯度与优化器状态，显存开销进一步降低。决策变量 θ_p 在 Megatron-LM 配置空间中取值： $\text{recompute_granularity} \in \{\text{none}, \text{full}, \text{selective}\}$ ；若为 selective，则由 recompute_modules 指定子模块列表。记流水级 p 在 θ_p 下的四项显存分别为 $\text{Mem}_p^{\text{param}}$ 、 $\text{Mem}_p^{\text{grad}}$ 、 $\text{Mem}_p^{\text{opt}}$ 与 $\text{Mem}_p^{\text{act}}(\theta_p)$ ； Q_p 为 d_p 上可用于训练的显存上界，并已在实践中扣除 CUDA 驱动、缓存与显存碎片等余量（在本文的实验中设置为 1.5 GB）。则可行配置需满足对任意流水级 p ，四项显存占用之和不超过其可用的显存上界 Q_p 。

迭代时间方面，流水级 p 的无重计算时前向与反向本地耗时分别记为 F_p 、 B_p 。在模型划分、设备映射及 micro-batch 等确定配置下， F_p 、 B_p 均与 θ_p 无关，由重计算引入的、仅为恢复中间激活而增加的本地前向计算开销，其耗时记为 $R_p(\theta_p)$ 。则问题可定义为在显存约束下，最小化每个流水级由重计算引入的本地前向计算开销 $R_p(\theta_p)$ ，即

$$\begin{aligned} \min_{\theta_p} \quad & R_p(\theta_p) \\ \text{s.t.} \quad & \text{Mem}_p^{\text{param}} + \text{Mem}_p^{\text{grad}} + \text{Mem}_p^{\text{opt}} + \text{Mem}_p^{\text{act}}(\theta_p) \leq Q_p. \end{aligned} \tag{4.12}$$

式 (4.12) 对每个流水级 p 各给出一个相互独立的离散优化问题。 θ_p 仅影响 $R_p(\theta_p)$ 与 $\text{Mem}_p^{\text{act}}(\theta_p)$ ，不因联立各级而组合爆炸。 R_p 及显存峰值受算子融合等实现因素

影响，本文实践中以实测剖析结果为主。

重计算引入的计算开销 $R_p(\theta)$ 由重计算额外引入的 FLOPs 占该流水级前向计算 FLOPs 的百分比估计。先统计该占比，再结合与第三章的剖析结果，将该比例折算为相对前向的时间侧开销。根据模型配置参数和第4.2.1节的理论分析，计算不同重计算策略下的 $\text{Mem}_p^{\text{act}}(\theta)$ ，与固定大小的 $\text{Mem}_p^{\text{param}}$ 、 $\text{Mem}_p^{\text{grad}}$ 、 $\text{Mem}_p^{\text{opt}}$ 相加得 $\text{TotalMem}(p, \theta)$ 。

在具体配置上，本文不联合优化 $\{\theta_p\}_{p=1}^P$ 的全局时间指标，而是对显存压力较大的流水级优先采用更节省显存的重计算策略以降低 $\text{Mem}_p^{\text{act}}$ ，对余量较大的流水级在满足约束的前提下尽量选取较轻的 θ 以抑制 R_p ，具体由下一小节的多路分支搜索实现。据此可将典型情形粗分为两类：（1）显存相对不足：宜采用 `full` 或子模块覆盖较全的 `selective`（如 `[core_attn, mlp, layernorm]`），以降低 $\text{Mem}_p^{\text{act}}$ ；（2）显存相对充足：宜优先使用 `none`、仅对 `[core_attn]` 启用 `selective` 以控制 $R_p(\theta_p)$ 不必要增大。

结合第三章冻结训练与提前终止调度，可在为各流水级独立求解式 (4.12) 时引入启发式先验，以压缩搜索空间并增强配置结果的可解释性。

（1）冻结流水级。冻结流水级通常不执行完整反向传播与优化器更新，梯度与优化器状态所占显存空间显著下降，峰值显存主要由模型参数与流水线并行下暂存的中间激活决定。此时若采用激进的重计算策略，额外开销主要体现在重复前向计算，在最坏情形下近似等于在该流水级增加完整的前向传播。相对于冻结流水级单次前向为主的计算特征，该比例可能显著偏高。因此默认优先选取 `none` 策略或仅覆盖少量子模块的 `selective`，除非剖析结果表明 $\text{Mem}_p^{\text{act}}$ 仍接近 Q_p 。

（2）非冻结流水级。非冻结流水级需执行完整反向传播并维护优化器状态， $\text{Mem}_p^{\text{grad}}$ 与 $\text{Mem}_p^{\text{opt}}$ 占据较大的显存空间。在反向传播计算开销约为前向传播开销的 2 倍的粗略假设下^[103]，整层重计算可近似视为额外引入与前向相当的计算，其相对于原前向与反向总工作量的占比约为三分之一。非冻结流水级通常较冻结流水级更能承受此类计算增量，故在可行集非空时更易选取 `full` 或 `recompute_modules` 覆盖更全的 `selective`。

（3）流水级在流水线中的位置。在 1F1B 等调度下，靠近输入端的流水级在稳定阶段往往需要同时保存更多 `micro-batch` 的中间激活值，其峰值显存通常高于靠近输出端的流水级（也取决于流水线深度、`micro-batch` 数目及是否采用交错调度等）。基于此观察，搜索时可令前半段流水级从较保守的配置 θ 出发，在显存约束下沿候选重计算方案逐步提高重计算强度，后半段流水级在显存允许时从较激进的 θ 出发，再沿候选重计算方案回退以降低不必要的重计算。该位置启发与（1）（2）相互独立，并与冻结配置一并体现在算法 4.1 中。

据此构造长度有限的候选策略集 $\Theta = (\theta^1, \dots, \theta^7)$ ：在 Megatron-LM 语义下自保守端（激活常驻规模较大、重计算较少）至激进端（激活常驻规模较小、重计算较多）排列。下文（1）–（7）的编号仅用于说明本文在工程实现中对各候选配置的含义，实际的重计算配置根据模型结构会有更多候选，工程实现中应依据实际模型参数得到的 $\text{Mem}_p^{\text{act}}(\theta)$ 对 Θ 重新排序，使其满足 $\text{Mem}_p^{\text{act}}(\theta^i) \geq \text{Mem}_p^{\text{act}}(\theta^{i+1})$ 。本文配置的 Θ 各候选项含义如下：（1）**none**：不启用重计算；（2）**Selective-LayerNorm**：仅对 LayerNorm 子模块启用重计算；（3）**Selective-MLP**：仅对 MLP 子模块启用重计算；（4）**Selective-Attn**：采用默认 **selective** 粒度，即仅对注意力子模块（**core_attn**）启用重计算；（5）**Selective-Attn+MLP**：对注意力与 MLP 子模块启用重计算；（6）**Selective-LayerNorm+Attn+MLP**：对注意力、LayerNorm 与 MLP 子模块启用重计算；（7）**full**：采用 **full** 粒度，对整层进行重计算。

记非冻结流水级集合为 $\mathcal{U}$ ，并令 $\text{mid} = \lfloor P/2 \rfloor$ 作为前后半段分界。定义 $\mathcal{U}_{\text{front}} = \{p \in \mathcal{U} \mid p \leq \text{mid}\}$ 、 $\mathcal{U}_{\text{back}} = \mathcal{U} \setminus \mathcal{U}_{\text{front}}$ ，分别对应靠近输入侧与靠近输出侧的非冻结流水级；再将 $\mathcal{U}_{\text{front}}$ 中编号按升序排成序列 W_{front} ，将 $\mathcal{U}_{\text{back}}$ 中编号按降序排成序列 W_{back} ，二者即算法 4.1 赋值 θ_p 时的访问顺序。冻结流水级事先固定为 θ^1 (**none**)。对 W_{front} 中各 p ，自 Θ 的较小下标（较保守配置）起逐个尝试，直至满足式 (4.12)；对 W_{back} 中各 p ，则自较大下标（较激进配置）选取可行 θ_p ，并在显存仍有余量时沿下标递减方向回调，以降低冗余重计算。若某流水级在离线搜索或在线运行时仍发生 OOM，则将其在 Θ 中的下标向增大方向推进并重试，直至满足显存约束或触发 MMoH 重新规划（见算法 4.1 分支）。算法 4.1 汇总上述流程；其中 TotalMem 与式 (4.12) 约束左端一致，冻结流水级的梯度与优化器状态按第三章置零或依据剖析表取值。

MMoH 给出模型划分和设备映射后，HR-MMoH 的重计算规划器为每个流水级生成一个重计算配置，所有流水级的重计算配置组织为一个 JSON 文件，由流水线并行组中各 **rank** 在进程初始化时读入。同一流水级内多卡（如张量并行）须共享同一 θ_p ，以保证重计算边界与张量并行通信组一致。不同流水级之间则可取不同 θ_p ，从而实现流水级维度的异构重计算。除重计算范围按 θ_p 区分外，单卡计算图与梯度语义与标准 Megatron-LM 一致，从而在更大批量或更长序列下仍满足式 (4.12) 中的显存约束。

HR-MMoH 与 MMoH 在模型划分、设备映射及提前终止调度上的关系可概括为：先由 MMoH 确定模型划分与设备映射，再在此基础上配置各流水级重计算参数 θ_p 。当显存仍溢出时将信息反馈至 MMoH 以调整模型划分或设备映射策略，并迭代上述过程。最后在调度时使用提前终止调度器进行调度优化。其中，模型划分与设备映射决定模型参数与计算负载分布，调度策略决定各流水级前向一反向

算法 4.1 HR-MMoH 多路分支重计算配置搜索

已知: 流水线并行度 P ; 冻结标记 $\text{frozen}[p]$; 可用显存上界 $\{Q_p\}$

已知: 全序候选 $\Theta = (\theta^1, \dots, \theta^G)$ (本文 $G=7$, $\text{Mem}_p^{\text{act}}(\theta^i)$ 随 i 增大非增) ; 剖析表 $\text{Mem}_p^{\text{param}}, \text{Mem}_p^{\text{grad}}, \text{Mem}_p^{\text{opt}}, \text{Mem}_p^{\text{act}}(\theta^i)$

求: 各流水级 θ_p

```
1:  $\text{TotalMem}(p, \theta^i) \leftarrow \text{Mem}_p^{\text{param}} + \text{Mem}_p^{\text{grad}} + \text{Mem}_p^{\text{opt}} + \text{Mem}_p^{\text{act}}(\theta^i)$ 
2:  $\text{mid} \leftarrow \lfloor P/2 \rfloor$ ;  $\forall p$ , 若  $\text{frozen}[p]$  则  $\theta_p \leftarrow \theta^1$ 
3:  $\mathcal{U} \leftarrow \{p \mid \text{frozen}[p] = \text{false}\}$ ;  $\mathcal{U}_{\text{front}} \leftarrow \{p \in \mathcal{U} \mid p \leq \text{mid}\}$ ;  $\mathcal{U}_{\text{back}} \leftarrow \mathcal{U} \setminus \mathcal{U}_{\text{front}}$ 
4:  $W_{\text{front}} \leftarrow \text{sort}_{\text{asc}}(\mathcal{U}_{\text{front}})$ ;  $W_{\text{back}} \leftarrow \text{sort}_{\text{desc}}(\mathcal{U}_{\text{back}})$ 
5: for all  $p \in W_{\text{front}}$  do
6:    $j \leftarrow 1$ 
7:   while  $j \leq G$  and  $\text{TotalMem}(p, \theta^j) > Q_p$  do
8:      $j \leftarrow j + 1$ 
9:   end while
10:  if  $j > G$  then
11:    触发 MMoH Planner
12:  else
13:     $\theta_p \leftarrow \theta^j$ 
14:  end if
15: end for
16: for all  $p \in W_{\text{back}}$  do
17:    $j \leftarrow G$ 
18:   if  $\text{TotalMem}(p, \theta^G) > Q_p$  then
19:     触发 MMoH Planner
20:   else
21:     while  $j > 1$  and  $\text{TotalMem}(p, \theta^{j-1}) \leq Q_p$  do
22:        $j \leftarrow j - 1$ 
23:     end while
24:      $\theta_p \leftarrow \theta^j$ 
25:   end if
26: end for
```

执行路径及相关通信。HR-MMoH 在不改变单卡数学语义的前提下调节 θ_p , 以减少异构 GPU 集群训练时的冗余重计算。若已采用最激进候选 θ^G 仍发生 OOM, 表明在当前划分与映射下仅依靠重计算仍无法满足训练条件, 应将显存溢出信息反馈至 MMoH 规划器 (例如调整该流水级在优化目标中的显存相关项、减少该流水级所含模型块数量等), 并在新映射上重新剖析后执行算法 4.1。映射更新后, 亦可相应将部分 θ_p 回调至较保守配置, 以避免长期承担过量的重计算开销。提前终止调度在冻结流水级上省略部分反向计算与阶段间通信, 与按流水级独立选取的 θ_p 相容。冻结流水级通常使用较轻量的重计算策略, 非冻结流水级仍依据式 (4.12)

与多路分支搜索中的位置启发式独立配置。

4.3 实验验证

4.3.1 实验设置

HR-MMoH 的实验环境与第三章 MMoH 实验一致，采用更具有代表性的异构集群（Cluster-L）及两个模型中更大的 MLLM（MLLM-12B）。为了让实验结果尽可能贴近常见大规模训练的配置，本文使用更大的批次大小，编码器和 LLM 的序列长度也相应扩大，都变成了第三章中实验配置的 2 倍。具体来说，本文在实验设置中把批次大小变为原来的 2 倍，编码器的序列长度从 576 变为 1152，LLM 的序列长度从 1024 变为 2048，从而尽量符合目前工业界训练时常用的参数配置。本章未在 Cluster-S（16 卡）与 MLLM-3B 上重复 HR-MMoH 实验，原因在于 Cluster-S 总显存仅约 216 GB，在上述更大批次与更长序列的配置下，16 张消费级 GPU 的实验意义有限，而若仍沿用第三章中较小的批次与序列设定，则显存压力本身不足以触发差异化重计算的需求，难以有效体现 HR-MMoH 的设计动机。此外，在工业实践中，小规模集群上以短序列、小批次训练大模型的场景本身较少出现，重计算的研究价值主要体现在显存紧张的大规模训练配置下。因此，本章聚焦于更具代表性的 Cluster-L + MLLM-12B + 大批次长序列组合，以验证 HR-MMoH 在实际显存瓶颈场景下的有效性。

为验证按流水级差异化配置的有效性，在 MMoH 已确定的划分与映射下，本文对 MLLM-12B 的每个流水级 p 在其映射设备 d_p 上，按第 4.2.2 节构造的离散候选集 $\Theta = (\theta^1, \dots, \theta^7)$ 逐项开展剖析。对每个候选 θ^i ，测定峰值激活显存 $\text{Mem}_p^{\text{act}}(\theta^i)$ ，并结合 $\text{Mem}_p^{\text{param}}$ 、 $\text{Mem}_p^{\text{grad}}$ 、 $\text{Mem}_p^{\text{opt}}$ 得到 $\text{TotalMem}(p, \theta^i)$ （与式 (4.12) 及算法 4.1 使用的符号表述一致）。上述剖析表用于在给定显存上界 Q_p 下判定各级可行候选，并支撑 HR-MMoH 搜索 θ_p 。

异构重计算的对比基线为全局统一重计算配置：所有流水级不采用重计算，记为 none；所有流水级采用相同的 full recomputation，即在前向传播中丢弃所有的中间激活值，记为 full；所有流水级采用相同的默认 selective，即只重计算 attention 模块。模型训练的冻结情况与第三章一致，使用全部的 4 种冻结情况：no-freeze, freeze-encoder, freeze-llm, freeze-both。分别使用 Megatron-LM、Whale、Espresso 和 MMoH 作为模型划分策略，对于所有策略使用之前提到的 3 种全局统一重计算配置。对于 MMoH，本文还使用异构的重计算配置 HR-MMoH。端到端对比指标为迭代时间（seconds/iteration）。每种配置均执行 10 次迭代预热，随后在 50 次稳定计时迭代上统计平均时间并作为报告值。

4.3.2 实验结果

本小节首先对比 Megatron-LM、Whale、Espresso 和 MMoH 在不同冻结策略和不同统一重计算配置下的训练速度，如图 4.4 所示。

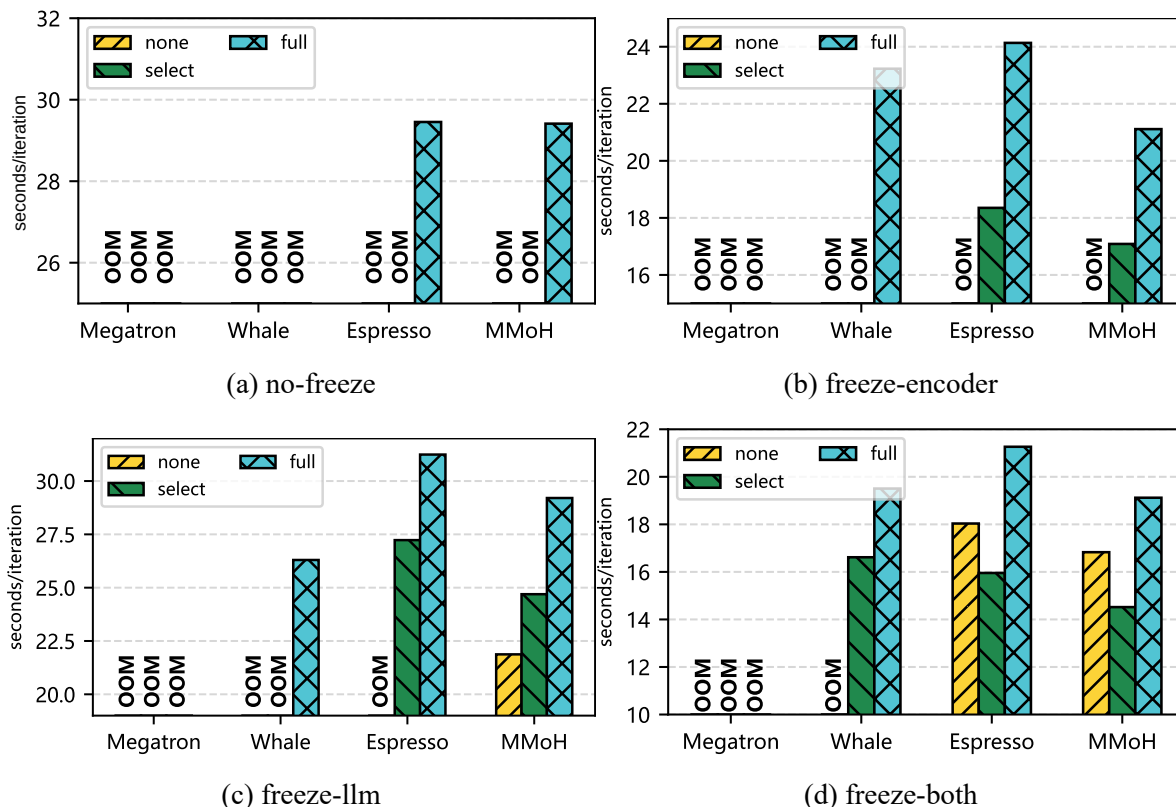


图 4.4 不同冻结策略下各框架迭代时间对比（自上至下、自左至右依次为 no-freeze、freeze-encoder、freeze-llm、freeze-both）。

图中按冻结策略拆分为四个子图，自左至右、自上而下依次为 (a) no-freeze、(b) freeze-encoder、(c) freeze-llm 与 (d) freeze-both。每个子图在同一冻结条件下对比各框架在全局统一的 none、full 与默认 selective 三种重计算配置下的迭代时间。由于数据的批次大小更大，encoder 和 llm 的序列长度更长，模型训练时需要的显存开销剧增。Megatron-LM 在所有冻结策略下都出现了 OOM，即使使用了全部重计算并且使用最大的 TP size，仍然无法进行训练，因为其同构假设导致负载不均。Whale 和 Espresso 通过异构感知的拓扑与划分避免了部分情况下的 OOM，所以两者表现优于 Megatron-LM，可以适应更多的冻结情况。在 no-freeze 的情况下，Espresso 在开启全部重计算后就可以正常训练，但是 Whale 在 no-freeze 的情况下无法正常训练，即使使用最大的 TP size 和最激进的全部重计算。总体上 Espresso 在更多场景下表现更好。MMoH 的负载均衡显著优于其它方法，在不开启任何重计算且冻结 LLM 的情况下，MMoH 是唯一可以正常训练的方法，而其余方法要

么彻底无法训练，要么需要使用重计算策略。在所有的冻结策略以及默认的重计算方式下，MMoH 的迭代时间都低于其余三者，这印证了本文在第三章中提出的方法的有效性。

就 Whale 与 Espresso 而言，在各自可运行的参数组合内，全局重计算策略在 none、full 与默认 selective 之间切换时，单次迭代时间往往出现显著差异。none 不在反向阶段为恢复激活而追加完整前向重算，在显存可行时通常对应较小的每步计算开销；full 与 selective 则通过缩小需常驻的激活规模、并以反向阶段的额外前向计算为代价换取峰值显存下降，因而在可比且均可训练的条件下，单次迭代时间通常高于 none，但在显存逼近设备上限时可用于获得可行配置。沿子图 (a)–(d) 观察，冻结范围逐步扩大使得参与更新的参数规模下降，相应的反向计算与流水级间梯度通信负荷随之减弱，各方法的迭代时间整体下移，不同重计算配置之间的相对优劣次序亦随之改变。由于划分粒度与设备拓扑构造不同，Whale 与 Espresso 在不同冻结场景及不同重计算配置下各有优劣。相较之下，MMoH 在上述四个冻结场景与三种统一重计算配置的组合中均保持最短或接近最短的迭代时间，表明在大批量与长序列所带来的显存受限设定下，复合粒度划分与冻结感知调度仍能提供可观的端到端性能收益。

在前一小节对全局统一重计算配置的基础上，本节进一步在各框架内部对 none、full、默认 selective 以及 HR-MMoH 的按流水级异构重计算配置分别搜索可行解，并在每种冻结策略下取各框架所能达到的最短迭代时间（seconds/iteration）进行并列对比，如图 4.5 所示。这样设置的意义在于：统一配置对比揭示了全局统一重计算配置在异构流水线中的局限性，而“每框架各自最优”对比则更客观地评估重计算优化后仍能否进一步加快训练速度。对 HR-MMoH 而言，最优配置由 4.2.2 节所述剖析驱动的候选枚举与约束判定给出，在 MMoH 已提供的模型划分与设备映射上为各流水级独立选择 `recompute_granularity`、`recompute_modules` 等参数，使显存压力大的流水级必要时更激进地释放激活、显存压力小的流水级尽量少做多余重计算，从而在满足各卡显存上限的前提下降低瓶颈流水级耗时。

图 4.5 的实验结果显示，在四种冻结条件下，启用异构重计算的 HR-MMoH 均取得最短迭代时间，说明在更大的批次大小与序列长度的训练设置下，按流水级差异化的重计算仍能系统性优于各框架在全局 none / full / selective 下所能达到的最优效果。

具体到数值，HR-MMoH 在 no-freeze、freeze-encoder、freeze-llm、freeze-both 下的迭代时间分别为 27.7714、16.5245、23.7048 与 12.9649 seconds/iteration。Espresso 同样覆盖上述四种冻结，其各自最短迭代时间为 29.4536、18.3514、27.2301 与 15.9547 seconds/iteration；与之相比，HR-MMoH 的迭代时间分别缩短 6.1%、11.0%、

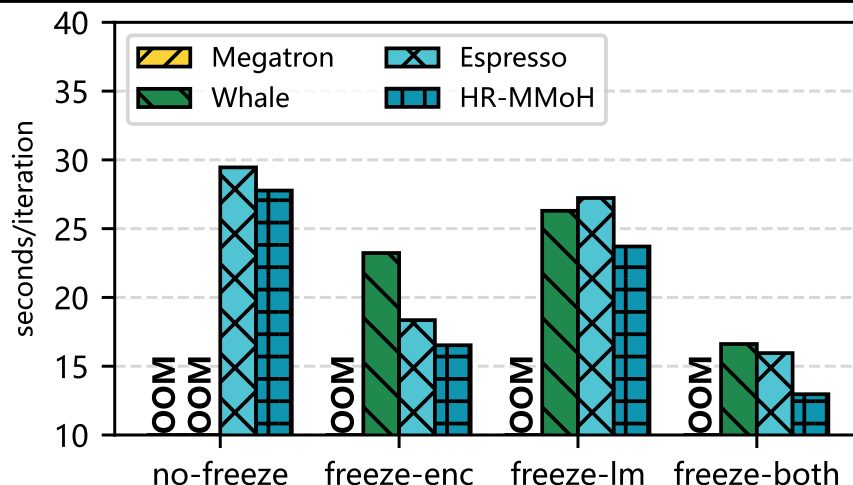


图 4.5 不同冻结策略下各框架在 none、full、selective 及 HR-MMoH 下的最短迭代时间对比。

14.9% 与 23.1%。可见相对这一强异构划分基准，HR-MMoH 的优势随冻结范围扩大而更加显著。freeze-both 场景下流水级间算力与显存需求的差异巨大，统一重计算配置更容易在非瓶颈级产生“过度重计算”，而 HR-MMoH 通过配置不同的重计算策略，更有效利用了冻结后释放的计算资源。

Whale 在本设定下仅能在 freeze-encoder、freeze-llm 与 freeze-both 三种冻结策略上完成训练（no-freeze 仍不可行），其对应最短迭代时间为 23.2292、26.2986 与 16.6158 seconds/iteration。HR-MMoH 相对 Whale 分别缩短 40.6%、11.0% 与 28.2%。其中 freeze-encoder 一列差距最大，与 Whale 在该场景下仍需依赖较重全局重计算、流水级负载与显存峰值更难同时平衡有关。HR-MMoH 则在剖析结果指导下将重计算开销主要分配给显存真正紧张的流水级，从而更明显降低端到端迭代时间。Megatron-LM 因同构假设下的负载分配失衡，在本实验的四种冻结策略与所试重计算、TP 配置下均触发 OOM，无法给出有效迭代时间，故不参与上图中的迭代时间对比。

综合而言，在每种冻结策略下将 HR-MMoH 与当前可运行的异构划分方法中较优者（Espresso 和 Whale）相比，端到端加速比落在约 $1.06\times$ – $1.41\times$ 区间：相对 Espresso 的提升相对较小（ $1.06\times$ – $1.23\times$ ），相对 Whale 在可训练子集上则出现更大幅度的迭代时间下降（ $1.11\times$ – $1.41\times$ ）。这表明 HR-MMoH 并非单一“更重的全局重计算”，而是在 MMoH 划分之上对计算显存占用进行进一步优化，可与第三章的划分与调度形成互补。

为了验证在同一划分与调度之上引入按流水级异构重计算能否进一步带来端到端收益，本节还对比了 MMoH 与 HR-MMoH 在不同冻结策略下最优训练速度，结果如图 4.6 所示。

实验显示，在 no-freeze、freeze-encoder、freeze-both 三种冻结配置下，启用

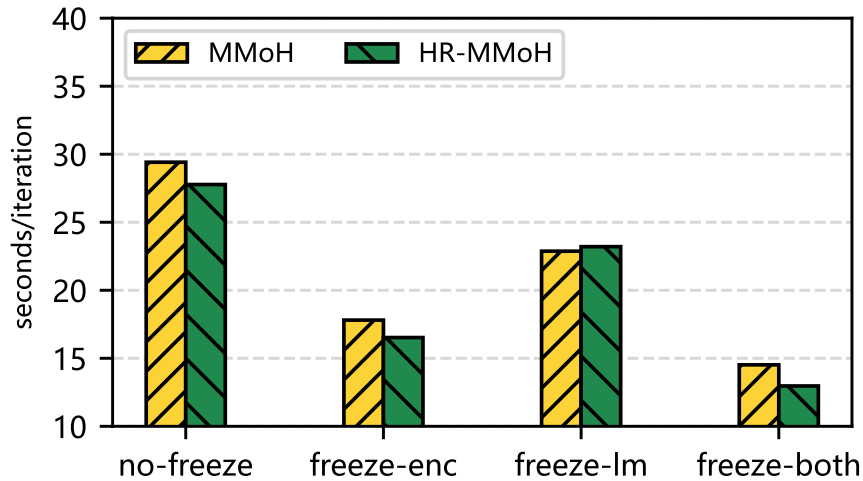


图 4.6 不同冻结策略下 MMoH 与 HR-MMoH 的迭代时间对比

异构重计算的 HR-MMoH 均获得不错的性能提升，说明在更大的批次大小和更长的序列长度设定下，按流水级差异化重计算仍能系统性优于 MMoH 的全局 none/full/selective 配置。具体来说，HR-MMoH 在 no-freeze、freeze-encoder、freeze-both 下的迭代时间加速比分别为 $1.06\times$ 、 $1.04\times$ 、 $1.12\times$ 。值得注意的是，在 freeze-llm 的场景下，HR-MMoH 的最优配置和 MMoH 的配置相同，因此两者迭代时间差异不显著。这是因为在 freeze-llm 的场景下，两者使用了相同的重计算策略，即均为 none。但是在 freeze-both 的场景下，HR-MMoH 的迭代时间加速比达到了 $1.12\times$ ，按设备差异化重计算带来了可观的性能提升。具体来看，MMoH 在使用默认的全局 selective 重计算策略时，迭代时间为最短，相比全局 none 重计算策略，TP 可以从 4 减少为 2，大幅减少了通信开销，而 HR-MMoH 在使用了异构重计算策略后，有了进一步的加速。

前述实验以迭代时间为主要指标，本段进一步验证 MMoH 的调度方式与 HR-MMoH 的重计算策略是否影响模型训练质量。从原理上看，MMoH 提前终止调度仅省略冻结前缀流水级中对可训练参数梯度无贡献的反向计算与通信，不改变任何可训练参数的梯度值；HR-MMoH 异构重计算则是在反向阶段重新执行前向以恢复中间激活，并不改变梯度计算的数学语义。因此，在相同随机种子、相同超参数与相同数据顺序下，上述优化不应引入训练精度偏差。

为实证上述分析，本文在 MLLM-12B + Cluster-L 上选取 freeze-both 这一冻结范围最大的场景，设计如下三组对照实验，均采用相同的超参数：（a）Espresso 划分 + 全局 selective 重计算（基线）；（b）MMoH 划分 + 提前终止调度 + 全局 selective 重计算；（c）完整的 HR-MMoH，即 MMoH 划分 + 提前终止调度 + 按流水级异构重计算。选择 Espresso 而非 Megatron-LM 作为基线，是因为后者在当前大批次长序列设定下会发生显存溢出（参见图 4.4），而 Espresso 是可正常运行的异

构划分方法中性能较优者。三组配置分别进行 400 步训练并记录每步的训练损失。

实验结果如图 4.7 所示，三条损失曲线在训练过程中有微小差异，但整体下降趋势与收敛速率一致，在训练后期趋于相同水平。配置 (b) 与基线 (a) 的损失轨迹高度接近，表明 MMoH 的提前终止调度不影响模型收敛质量；配置 (c) 与 (b) 同样未出现发散或系统性偏移，说明按流水级差异化的异构重计算同样不改变训练收敛行为。上述结果从实验层面证实了理论分析的正确性。

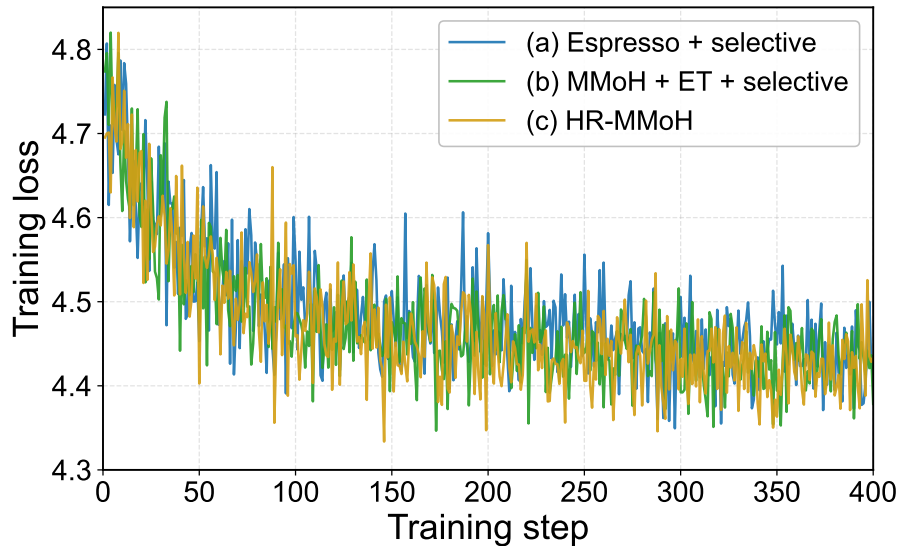


图 4.7 训练收敛性验证：三种配置下 MLLM-12B 在 Cluster-L freeze-both 场景的训练损失对比。(a) Espresso + selective（基线），(b) MMoH + 提前终止 + selective，(c) HR-MMoH（提前终止 + 异构重计算）。

4.3.3 实验分析

异构 GPU 集群上各流水级显存与算力分布不均，为防止 OOM 而采用统一且偏激进的重计算时，往往使重计算过多，瓶颈流水级计算负担进一步加大。按流水级差异化配置后，可在满足各流水级显存约束的前提下，根据各流水级使用的硬件配置使用不同的重计算策略，从而在系统层面降低训练迭代时间。

MMoH 的复合粒度与冻结感知划分已使各流水级负载趋于均衡，HR-MMoH 在此基础上做细粒度调节，二者结合可形成“划分—调度—重计算多级优化”的闭环。未来工作可将 HR-MMoH 的配置搜索与 MMoH 的整数规划联合建模，在进行模型划分和设备映射时即考虑重计算给迭代时间和显存开销带来的影响。

4.4 本章小结

本章在 MMoH 的异构流水线划分与提前终止调度的基础上，针对显存受限流水级仍可能 OOM、而现有重计算策略多采用全局统一配置难以兼顾显存与效率的

问题，提出了 HR-MMoH 的异构重计算技术。首先本章系统梳理了现有的重计算配置的粒度，并指出其在同构假设下的局限，随后给出了 HR-MMoH 的设计要点，即以各设备显存、算力为约束，为各流水级独立配置重计算粒度与子模块集合，通过基于剖析的配置策略在满足显存不溢出的前提下抑制冗余重计算，并与 MMoH 划分及提前终止调度器协同。实验结果表明，在不同冻结场景下 HR-MMoH 取得了显著加速，相比现有方法最高达到约 $1.41\times$ 的加速比。HR-MMoH 与第三章的 MMoH 共同构成“划分—调度—重计算多级优化”框架，为异构 GPU 集群上 MLLMs 的高效训练提供了有效的技术补充。

第五章 总结与展望

本章对全文工作进行总结，概括主要创新点，归纳研究局限，并从若干研究问题展望未来工作。

5.1 全文工作总结

本文围绕“面向异构 GPU 集群的多模态大语言模型并行训练技术”这一主题，针对实际环境中普遍存在的设备异构与 MLLM 的模块异构、分阶段冻结训练等特性，从模型划分与流水线调度以及重计算优化两个层面开展了研究，提出了 MMoH 与 HR-MMoH 两套方法，并在异构集群上进行了实验验证。下文概括主要创新点。

5.1.1 主要创新点

本文的主要创新点可归纳为以下两方面。

(1) MMoH: 面向异构 GPU 集群的多模态大语言模型高效混合并行训练框架。

针对异构集群上 MLLM 训练时存在的负载不均衡、资源利用率低以及冻结场景下冗余计算与通信等问题，本文提出了 MMoH 框架。其创新点包括：需求驱动的设备拓扑识别，根据 MLLM 各模块的计算与显存需求趋势与异构设备的算力—显存供给进行匹配，筛选候选设备拓扑，使需求与供给在趋势上一致。复合粒度模型抽象，针对编码器、投影层与 LLM 基座的结构差异，采用多级粒度将模型重组为模型块列表（编码器按子模块合并、LLM 按注意力/FFN 子层拆分），在划分复杂度与负载均衡之间进行权衡。冻结感知的模型划分，将块到阶段的分配形式化为整数规划问题，在满足显存约束与主流水级约束下最小化迭代时间，且目标函数显式依赖各阶段的冻结状态，从而随训练阶段动态调整划分策略。提前终止调度器，在 1F1B 等流水线调度基础上，识别从首端开始的连续冻结流水级并剪枝其反向计算与流水级间通信，在不影响模型收敛性的前提下提升吞吐。在两种异构集群（Cluster-S、Cluster-L）与两种规模 MLLM（MLLM-3B、MLLM-12B）上的实验表明，MMoH 在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种场景下相对 Megatron-LM、Whale、Espresso 均可取得最高吞吐量，最高可达约 1.77× 加速，且提前终止调度器在冻结场景下可带来约 8%–19% 的额外增益，并具有良好的可移植性。

(2) HR-MMoH: 面向异构 GPU 集群的多模态大语言模型重计算策略优化。

针对现有重计算策略在同构假设下采用全局统一配置、难以在异构集群上

同时兼顾显存安全与计算效率的问题，本文在 MMoH 的划分与调度基础上，提出并实现了 HR-MMoH 的异构重计算方法。HR-MMoH 使各流水级根据自身显存与算力条件采用差异化的重计算粒度（如 full/selective）与子模块选择，在降低 OOM 风险的同时减少多余重算、平衡流水级负载，与 MMoH 形成“划分—调度—重计算多级优化”。第四章通过在更大批次大小与更长序列长度的实验下验证，HR-MMoH 在 no-freeze、freeze-encoder、freeze-llm、freeze-both 四种冻结场景中均取得最短迭代时间，相对 Espresso 的迭代时间缩短约 6.1%–23.1%，在 Whale 可训练的三种冻结场景中缩短约 11.0%–40.6%。这说明按流水级差异化配置能够系统性优于全局统一策略，并验证了与 MMoH 协同优化的有效性。

综上，本文在异构 GPU 集群上 MLLMs 并行训练这一方向上，从划分与调度与重计算优化两条线给出了系统性的方法设计与实验支撑，为实际部署中的混合集群与分阶段训练场景提供了可参考的技术路径。

5.2 局限性及未来工作展望

结合前文全文总结，本节先归纳研究局限，再围绕后续研究的核心问题与思路作简要展望。

5.2.1 研究局限性

本研究仍存在以下局限，亦是后续工作的重要出发点。

规划器中的整数规划求解在模型层数与设备数较大时可能面临组合爆炸，当前采用启发式或现有求解器在有限时间内求近似解，最优性保证与可扩展性仍有提升空间；实验仅在两种规模的 MLLM（3B、12B）与两种异构集群配置上进行，对更大规模模型（如 70B+）、更多代际与型号混合的集群以及更长序列、更多模态的 MLLMs 的泛化能力有待进一步验证。

尽管第四章已完成端到端实验并验证了按流水级异构重计算的收益，当前策略搜索仍以启发式与剖析驱动为主，候选配置空间、搜索顺序与回退机制依赖经验设计，尚未给出更强的最优性保证与复杂度分析；实验主要在 Cluster-L 与 MLLM-12B 上开展，对更大规模模型、更复杂模型结构（如 MoE 模型）及更长上下文设定下的鲁棒性仍需补充；此外，HR-MMoH 与 MMoH 的联合优化目前仍采用“先划分、后配置重计算”的分层流程，尚未形成统一目标下的联合求解框架。

当前工作主要针对视觉—语言类 MLLM 与 NVIDIA GPU 生态，对音频、视频等多模态输入，以及对 AMD、国产 GPU 等异构硬件与软件栈的适配与评估尚未涉及，与专家并行、上下文并行等已有技术的协同优化仍有扩展空间。

5.2.2 未来研究工作

基于上述局限，后续研究可沿以下两条线索推进。

(1) 划分与重计算的联合求解。本文采用“先由 MMoH 求解划分与设备拓扑，再由 HR-MMoH 为各流水级搜索重计算配置”的分层流程。该流程存在循环依赖：划分决定各流水级的层分配与显存余量，进而影响可选的重计算配置与各流水级耗时；而重计算配置改变后各流水级的实际耗时又反过来改变最优划分。当前分层求解无法捕捉这种耦合，可能遗漏“不同划分 + 不同重计算组合”下迭代时间更低的方案。一种可行思路是将重计算配置作为决策变量引入第三章的整数线性规划，把重计算带来的额外前向开销纳入迭代时间目标，使划分与重计算在同一目标下联合求解；或采用交替优化——固定重计算求划分，再固定划分更新重计算——并分析其收敛性。此方向的核心在于量化分层流程的次优性来源及其量级，为判断是否值得引入联合求解提供依据。

(2) 提前终止调度向新型流水线机制的扩展。第三章的提前终止调度器与迭代时间的推导均建立在 1F1B 调度之上。近年提出的 ZeroBubble^[103] 将反向拆分为对输入的梯度与对参数的梯度两个独立阶段，通过 1F1B1W 调度将后者与下一 micro-batch 的前向重叠以压缩气泡；DualPipe^[104] 则采用双向 micro-batch 流动实现前反向交织。这两类调度改变了流水级间的依赖结构：冻结流水级不需要计算对参数的梯度，但对输入的梯度能否被提前终止策略剪枝，取决于后续流水级在新调度下的依赖关系是否与 1F1B 一致。在上述新调度框架下重新推导冻结前缀的可剪枝条件与迭代时间收益，并验证其与 HR-MMoH 按流水级差异化重算的兼容性，可作为下一步研究工作的切入点。若成功推广，可在进一步压缩气泡的同时保留冻结前缀剪枝的收益，使 MMoH 框架适用于更广泛的工程实践。

综上所述，本文在面向异构 GPU 集群的 MLLMs 并行训练策略方面取得了阶段性成果；后续宜在划分与重计算的联合求解以及提前终止向新型流水线调度的扩展两个方向上继续推进，为异构 GPU 集群环境下 MLLMs 训练的进一步优化提供方法与理论支撑。

致 谢

参考文献

- [1] Miao X, Wang Y, Jiang Y, et al. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism [J]. Proceedings of the VLDB Endowment. 2022, 16 (3): 470–479.
- [2] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need [C]. Advances in Neural Information Processing Systems. 2017: 5998–6008.
- [3] Devlin J, Chang M-W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [C]. Proceedings of NAACL-HLT. 2019: 4171–4186.
- [4] Radford A, Wu J, Child R, et al. Language Models are Unsupervised Multitask Learners [J]. OpenAI blog. 2019, 1 (8): 9.
- [5] Raffel C, Shazeer N, Roberts A, et al. Exploring the Limits of Transfer Learning with a Unified Text-to-text Transformer [J]. Journal of Machine Learning Research. 2020, 21 (1): 5485–5551.
- [6] Brown T, Mann B, Ryder N, et al. Language Models are Few-shot Learners [J]. Advances in Neural Information Processing Systems. 2020, 33: 1877–1901.
- [7] OpenAI. GPT-4 Technical Report [J]. arXiv.org. 2023. 2023-03-15.
- [8] Dosovitskiy A, Beyer L, Kolesnikov A, et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale [C]. International Conference on Learning Representations. 2021: 1–22.
- [9] Zhai X, Kolesnikov A, Houlsby N, et al. Scaling Vision Transformers [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2022: 1204–1213.
- [10] Li J, Li D, Savarese S, et al. BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models [C]. Proceedings of the 40th International Conference on Machine Learning (ICML). 2023: 19730–19742.
- [11] Liu H, Li C, Wu Q, et al. Visual Instruction Tuning [C]. Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023. 2023: 34892–34916.
- [12] Alayrac J-B, Donahue J, Luc P, et al. Flamingo: a Visual Language Model for Few-shot Learning [J]. Advances in Neural Information Processing Systems. 2022, 35:

[13] Radford A, Kim J W, Hallacy C, et al. Learning Transferable Visual Models from Natural Language Supervision [C]. International Conference on Machine Learning. 2021: 8748–8763.

[14] Ramesh A, Pavlov M, Goh G, et al. Zero-shot Text-to-image Generation [C]. International Conference on Machine Learning. 2021: 8821–8831.

[15] Dai W, Li J, Li D, et al. InstructBLIP: Towards General-purpose Vision-Language Models with Instruction Tuning [C]. Advances in Neural Information Processing Systems (NeurIPS). 2023: 49250–49267.

[16] Liu H, Li C, Wu Q, et al. Improved Baselines with Visual Instruction Tuning [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024: 26286–26296. LLaVA-1.5.

[17] Bai J, Bai S, Chu Y, et al. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond [J]. arXiv preprint arXiv:2308.12966. 2023.

[18] Chen Z, Wu J, Wang W, et al. InternVL: Scaling up Vision Foundation Models and Aligning for Generic Visual-Language Tasks [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024: 13142–13153.

[19] Zhu D, Chen J, Shen X, et al. MiniGPT-4: Enhancing Vision-language Understanding with Advanced Large Language Models [C]. The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024. 2024.

[20] Caffagni D, Cocchi F, Barsellotti L, et al. The Revolution of Multimodal Large Language Models: A Survey [C]. Findings of the Association for Computational Linguistics: ACL 2024. 2024: 13590–13618.

[21] Zhu Z, Zhang Y, Zhuang X, et al. Can We Trust AI Doctors? A Survey of Medical Hallucination in Large Language and Large Vision-Language Models [J]. Findings of the Association for Computational Linguistics: ACL 2025. 2025: 6748–6769.

[22] Goyal Y, Khanna G, Gupta A, et al. Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering [C]. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2017: 6904–6913.

[23] Hudson D A, Manning C D. GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 6700–6709.

[24] Yue X, Ni Y, Zhang K, et al. MMMU: A Massive Multi-discipline Multi-

modal Understanding and Reasoning Benchmark for Expert AGI [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024: 9556–9567.

[25] Kaplan J, McCandlish S, Henighan T, et al. Scaling Laws for Neural Language Models [J]. arXiv preprint arXiv:2001.08361. 2020: 1–22.

[26] Hoffmann J, Borgeaud S, Mensch A, et al. Training Compute-Optimal Large Language Models [C]. Advances in Neural Information Processing Systems (NeurIPS). 2022: 30016–30030.

[27] Sardana N, Frankle J. Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws [C]. Proceedings of the International Conference on Machine Learning (ICML). 2024: 29891–29922.

[28] Rae J W, Borgeaud S, Cai T, et al. Scaling Language Models: Methods, Analysis and Insights from Training Gopher [J]. arXiv preprint arXiv:2112.11446. 2022.

[29] DeepSeek-AI. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism [J]. arXiv preprint arXiv:2401.02954. 2024.

[30] Yang A, Yang B, Hui B, et al. Qwen2 Technical Report [J]. arXiv preprint arXiv:2407.10671. 2024.

[31] Touvron H, Martin L, Stone K, et al. Llama 2: Open Foundation and Fine-tuned Chat Models [J]. arXiv preprint arXiv:2307.09288. 2023.

[32] Dubey A, Jauhri A, Pandey A, et al. The LLaMA 3 Herd of Models [J]. arXiv preprint arXiv:2407.21783. 2024.

[33] Gemma Team, Mesnard T, Hardin C, et al. Gemma: Open Models Based on Gemini Research and Technology [J]. arXiv preprint arXiv:2403.08295. 2024.

[34] Jiang A Q, Sablayrolles A, Mensch A, et al. Mistral 7B [J]. arXiv preprint arXiv:2310.06825. 2023.

[35] Narayanan D, Shoeybi M, Casper J, et al. Efficient Large-scale Language Model Training on GPU Clusters Using Megatron-LM [C]. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. 2021: 1–15.

[36] Shoeybi M, Patwary M, Puri R, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism [J]. arXiv preprint arXiv:1909.08053. 2020: 1–12.

[37] Bai Y, Lv X, Zhang J, et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding [C]. Proceedings of the 62nd Annual Meeting of the

Association for Computational Linguistics (Volume 1: Long Papers). 2024: 3119–3137.

[38] Rajbhandari S, Rasley J, Ruwase O, et al. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models [C]. SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. 2020: 1–16.

[39] Rajbhandari S, Ruwase O, Rasley J, et al. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning [C]. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). 2021: 59.

[40] Li S, Zhao Y, Varma R, et al. PyTorch Distributed: Experiences on Accelerating Data Parallel Training [J]. Proceedings of the VLDB Endowment. 2020, 13 (12): 3005–3018.

[41] Huang Y, Cheng Y, Bapna A, et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism [C]. Advances in Neural Information Processing Systems (NeurIPS). 2019: 10215–10224.

[42] Narayanan D, Harlap A, Phanishayee A, et al. PipeDream: Generalized Pipeline Parallelism for DNN Training [C]. Proceedings of the 27th ACM Symposium on Operating Systems Principles. 2019: 1–15.

[43] Chen Z, Lepikhin D D, Lee H, et al. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding [C]. 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021. 2021.

[44] Micikevicius P, Narang S, Alben J, et al. Mixed Precision Training [C]. Proceedings of the 35th International Conference on Machine Learning. 2018: 403–411.

[45] Liu Y, Li S, Fang J, et al. Colossal-Auto: Unified Automation of Parallelization and Activation Checkpoint for Large-scale Models [J]. arXiv preprint. 2023.

[46] Dao T, Fu D Y, Ermon S, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness [C]. Advances in Neural Information Processing Systems (NeurIPS). 2022: 16344–16359.

[47] Dao T, Fu D Y, Ermon S, et al. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision [C]. Advances in Neural Information Processing Systems (NeurIPS). 2024: 1–16.

[48] NVIDIA Corporation. NVIDIA A100 Tensor Core GPU Architecture [R]. 2020. Ampere Architecture.

[49] NVIDIA Corporation. NVIDIA H100 Tensor Core GPU Architecture [R]. 2022. Hopper Architecture.

[50] NVIDIA Corporation. NVIDIA Blackwell Architecture and GPU Roadmap [R/OL]. 2024. <https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture/>.

[51] Jia X, Jiang L, Wang A, et al. Whale: Efficient Giant Model Training over Heterogeneous GPUs [C]. Proceedings of the 2022 USENIX Annual Technical Conference (USENIX ATC 22). 2022: 673–688.

[52] Park J H, Yun G, Chang M Y, et al. HetPipe: Enabling Large DNN Training on (Whimpy) Heterogeneous GPU Clusters through Integration of Pipelined Model Parallelism and Data Parallelism [C]. 2020 USENIX Annual Technical Conference (USENIX ATC 20). 2020: 307–321.

[53] Chang Z, Xiao S, He S, et al. Frenzy: Memory-aware Serverless Scheduling for Large Model Training [J]. arXiv preprint. 2024.

[54] NVIDIA Corporation. NVIDIA AI Training: Best Practices for Large Language Models [J]. 2024. GTC / Developer Blog.

[55] He G, Jiang Y, Xiao W, et al. Efficient Pre-Training of LLMs via Topology-Aware Communication Alignment on More Than 9600 GPUs [C/OL]. Advances in Neural Information Processing Systems (NeurIPS). 2025. <https://openreview.net/forum?id=xWYL9Ki32T>.

[56] Hu E J, Shen Y, Wallis P, et al. LoRA: Low-Rank Adaptation of Large Language Models [C]. International Conference on Learning Representations (ICLR). 2022: 1–35. arXiv:2106.09685, 2021.

[57] Dettmers T, Pagnoni A, Holtzman A, et al. QLoRA: Efficient Finetuning of Quantized LLMs [C]. Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10–16, 2023. 2023: 10088–10115.

[58] Lialin V, Deshpande V, Yao X, et al. Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning [J]. arXiv preprint arXiv:2303.15647. 2024.

[59] Schuhmann C, Beaumont R, Vencu R, et al. LAION-5B: An Open Large-Scale Dataset for Training Next Generation Image-Text Models [C]. Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track. 2022: 1–31. arXiv:2210.08402.

[60] Liang C X, Tian P, Yin C H, et al. A Comprehensive Survey and Guide to Multimodal Large Language Models in Vision-Language Tasks [J]. arXiv preprint arXiv:2411.06284. 2024: 1–45.

-
-
- [61] Bai T, Liang H, Wan B, et al. A Survey of Multimodal Large Language Model from A Data-centric Perspective [J]. arXiv preprint arXiv:2405.16640. 2024.
- [62] Kuznetsova A, Rom H, Alldrin N, et al. The Open Images Dataset V4: Unified Image Classification, Object Detection, and Visual Relationship Detection at Scale [J]. International Journal of Computer Vision. 2020, 128 (7): 1956–1981.
- [63] Lin T-Y, Maire M, Belongie S, et al. Microsoft COCO: Common Objects in Context [C]. European Conference on Computer Vision (ECCV). 2014: 740–755.
- [64] Gao L, Biderman S, Black S, et al. The Pile: An 800GB Dataset of Diverse Text for Language Modeling [J]. arXiv preprint arXiv:2101.00027. 2020.
- [65] Abu-El-Haija S, Kothari N, Lee J, et al. YouTube-8M: A Large-Scale Video Classification Benchmark [C]. Computer Vision – ECCV 2016. 2016: 868–884.
- [66] Bain M, Nagrani A, Varol G, et al. Frozen in Time: A Joint Video and Image Encoder for End-to-End Retrieval [C]. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). 2021: 1708–1718.
- [67] Singh A, Natarajan V, Shah M, et al. Towards VQA Models That Can Read [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 8317–8326.
- [68] Liu G, Miao Y, Lin Z, et al. Aceso: Efficient Parallel DNN Training through Iterative Bottleneck Alleviation [C]. Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys). 2024: 1–17.
- [69] Lin H, Wu K, Li J, et al. UniAP: Unifying Inter- and Intra-Layer Automatic Parallelism by Mixed Integer Quadratic Programming [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2025: 20947–20957.
- [70] Sergeev A, Balso M D. Horovod: fast and easy distributed deep learning in TensorFlow [J]. arXiv preprint arXiv:1802.05799. 2018.
- [71] Korthikanti V A, Casper J, Lym S, et al. Reducing Activation Recomputation in Large Transformer Models [C]. Proceedings of Machine Learning and Systems (MLSys). 2023: 1–20.
- [72] Wang G, Qin H, Jacobs S A, et al. ZeRO++: Extremely Efficient Collective Communication for Large Model Training [C]. The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024. 2024.
- [73] Zhang Z, Zhang Z, Jiang Y, et al. DistTrain: Addressing Model and Data Heterogeneity with Disaggregated Training for Multimodal Large Language Models [C]. Proceedings of the ACM SIGCOMM 2025 Conference. Coimbra, Portugal, 2025: 1–15.

-
-
- [74] Li D, Wang H, Xing E, et al. AMP: Automatically Finding Model Parallel Strategies with Heterogeneity Awareness [C]. *Advances in Neural Information Processing Systems*. 2022: 6630–6639.
- [75] Xu S, Huang Z, Zeng Y, et al. HETHUB: A Distributed Training System with Heterogeneous Cluster for Large-Scale Models [J]. *arXiv preprint arXiv:2405.16256*. 2024.
- [76] Yan R, Jiang Y, Nie X, et al. HexiScale: Accommodating Large Language Model Training over Heterogeneous Environment [J]. *arXiv preprint arXiv:2409.01143*. 2024.
- [77] Zhou Q, Xu F, Weng L, et al. Espresso: Cost-Efficient Large Model Training by Exploiting GPU Heterogeneity in the Cloud [C]. *IEEE INFOCOM 2025 – IEEE Conference on Computer Communications*. London, United Kingdom, 2025: 1–10. Implementation: <https://github.com/icloud-ecnu/Espresso>.
- [78] Um T, Oh B, Kang M, et al. Metis: Fast Automatic Distributed Training on Heterogeneous GPUs [C]. *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*. Santa Clara, CA, USA, 2024: 1–16.
- [79] Zhang S, Diao L, Wu C, et al. HAP: SPMD DNN Training on Heterogeneous GPU Clusters with Automated Program Synthesis [C]. *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 2024: 524–541.
- [80] Huang J, Zhang Z, Zheng S, et al. DISTMM: Accelerating Distributed Multimodal Model Training [C]. *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. Santa Clara, CA, USA, 2024: 1–16.
- [81] Zhang Z, Zheng S, Wang Y, et al. MiCS: Near-linear Scaling for Training Gigantic Model on Public Cloud [J]. *Proceedings of the VLDB Endowment*. 2022, 16 (1): 37–50.
- [82] Chen Q, Hu Q, Ye Z, et al. AMSP: Reducing Communication Overhead of ZeRO for Efficient LLM Training [J]. *arXiv preprint arXiv:2311.00257*. 2023. Also circulated under the title AMSP: Super-Scaling LLM Training via Advanced Model States Partitioning; preprint only as of 2026.
- [83] Mathew M, Karatzas D, Jawahar C V. DocVQA: A Dataset for VQA on Document Images [C]. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. 2021: 2200–2209.
- [84] Masry A, Long D X, Tan J Q, et al. ChartQA: A Benchmark for Question An-

swering about Charts with Visual and Logical Reasoning [C]. Findings of the Association for Computational Linguistics: ACL 2022. 2022: 2263–2279.

[85] Gan Z, Li L, Li C, et al. Vision-Language Pre-training: Basics, Recent Advances, and Future Trends [J]. Foundations and Trends in Computer Graphics and Vision. 2022.

[86] Lu P, Mishra S, Xia T, et al. Learn to Explain: Multimodal Reasoning via Thought Chains for Science Question Answering [C]. Advances in Neural Information Processing Systems. 2022: 2507–2521.

[87] Rombach R, Blattmann A, Lorenz D, et al. High-resolution Image Synthesis with Latent Diffusion Models [C]. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2022: 10684–10695.

[88] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation [C]. Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015. 2015: 234–241.

[89] Wu S, Fei H, Qu L, et al. NExT-GPT: Any-to-Any Multimodal LLM [C]. Proceedings of the 41st International Conference on Machine Learning (ICML). 2024: 53366–53397.

[90] Dong R, Han C, Peng Y, et al. DreamLLM: Synergistic Multimodal Comprehension and Creation [C]. The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024. 2024.

[91] Liu H, Li C, Wu Q, et al. LLaVA-NeXT: Improved Baselines with Visual Instruction Tuning [J]. arXiv preprint arXiv:2402.12975. 2024: 1–35.

[92] Gemini Team, Anil R, Borgeaud S, et al. Gemini: A Family of Highly Capable Multimodal Models [J]. arXiv preprint arXiv:2312.11805. 2023. Google DeepMind technical report (Gemini 1.0).

[93] Chowdhery A, Narang S, Devlin J, et al. PaLM: Scaling Language Modeling with Pathways [J]. Journal of Machine Learning Research. 2023, 24: 1–113. JMLR article 22-1144; arXiv:2204.02311, 2022.

[94] Li M. Scaling Distributed Machine Learning with the Parameter Server [C]. Proceedings of the 2014 International Conference on Big Data Science and Computing. Beijing, China, 2014: 1–1.

[95] Zhao Y, Gu A, Varma R, et al. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel [C]. Proceedings of the International Conference on Very Large Data Bases (VLDB). 2023: 3846–3856. PyTorch Native FSDP, 2022.

-
-
- [96] Kim S, Yu G I, Park H, et al. Parallax: Sparsity-aware Data Parallel Training of Deep Neural Networks [C]. Proceedings of the Fourteenth EuroSys Conference. Dresden, Germany, 2019: 1–15.
- [97] Cheng J, Gao N, Yue Y, et al. EDiT: A Local-SGD-Based Efficient Distributed Training Method for Large Language Models [C/OL]. The Thirteenth International Conference on Learning Representations (ICLR). 2025. <https://openreview.net/forum?id=xtlMtbVfWu>.
- [98] Kim G-W, Li J, Gandham S, et al. HALoS: Hierarchical Asynchronous Local SGD over Slow Networks for Geo-Distributed Large Language Model Training [C/OL]. Proceedings of the 42nd International Conference on Machine Learning (ICML). 2025: 30548–30566. <https://proceedings.mlr.press/v267/kim25y.html>.
- [99] Wang Z, Zhang J, Wu X, et al. From Promise to Practice: Realizing High-Performance Decentralized Training [C/OL]. The Thirteenth International Conference on Learning Representations (ICLR). 2025. <https://openreview.net/forum?id=lo3nlFHOft>.
- [100] Li Z, Zhuang S, Guo S, et al. TeraPipe: Token-Level Pipeline Parallelism for Training Large-Scale Language Models [C]. Proceedings of the International Conference on Machine Learning (ICML). 2021: 6543–6552.
- [101] Li S, Hoefler T. Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines [C]. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). 2021: 27.
- [102] Liu Z, Cheng S, Zhou H, et al. Hanayo: Harnessing Wave-like Pipeline Parallelism for Enhanced Large Model Training Efficiency [C]. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC). Denver, CO, USA, 2023: 1–13. arXiv:2308.15762.
- [103] Qi P, Wan X, Huang G, et al. Zero Bubble Pipeline Parallelism [C]. The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7–11, 2024. 2024: 1–21.
- [104] DeepSeek-AI. DualPipe: Bidirectional Pipeline Parallelism for Large-Scale Training. <https://github.com/deepseek-ai/DualPipe>. 2025.
- [105] Sun Z, Cao H, Wang Y, et al. AdaPipe: Optimizing Pipeline Parallelism with Adaptive Recomputation and Partitioning [C]. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2024: 1–16.
- [106] Sun Z, Chen S, Wang Y, et al. MEPipe: Democratizing LLM Training with

Memory-Efficient Slice-Level Pipeline Scheduling on Cost-Effective Accelerators [C]. Proceedings of the Twentieth European Conference on Computer Systems (EuroSys). 2025: 1–16.

[107] Lin J, Liu Z, You Y, et al. WeiPipe: Weight Pipeline Parallelism for Communication-Effective Long-Context Large Model Training [C]. Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP). Las Vegas, NV, USA, 2025.

[108] Jacobs S A, Tanaka M, Zhang C, et al. DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models [J]. arXiv preprint arXiv:2309.14509. 2023. Microsoft DeepSpeed technical report; preprint only as of 2026.

[109] DeepSeek-AI. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models [C]. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024: 1280–1297.

[110] Rasley J, Rajbhandari S, Ruwase O, et al. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters [C]. Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2020: 241–250.

[111] Guo R B, Anand U, Daudjee K, et al. Zorse: Optimizing LLM Training Efficiency on Heterogeneous GPU Clusters [J]. arXiv preprint arXiv:2507.10392. 2025. Preprint; no conference proceedings version located as of 2026.

[112] Mei Y, Zhuang Y, Miao X, et al. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow [C]. Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). Rotterdam, Netherlands, 2025: 1–16.

[113] Zhang H, Ju L, Wu C, et al. Rethinking Memory and Communication Costs for Efficient Data Parallel Training of Large Language Models [C/OL]. Advances in Neural Information Processing Systems (NeurIPS). 2024. <https://openreview.net/forum?id=4Un2TD9bNe>.

[114] Warraich E, Shabtai O, Manaa K, et al. OptiReduce: Resilient and Tail-Optimal AllReduce for Distributed Deep Learning in the Cloud [C/OL]. 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25). Philadelphia, PA, USA, 2025: 685–703. <https://www.usenix.org/conference/nsdi25/presentation/warraich>.

[115] Chen T, Xu B, Zhang C, et al. Training Deep Nets with Sublinear Memory Cost [C]. Proceedings of the 33rd International Conference on Machine Learning (ICML). 2016: 284–293.

[116] Li Z, Lin Z, Wang C, et al. Lynx: Efficient Memory Management for Large Model Training with Overlapped Activation Recomputation [J]. arXiv preprint arXiv:2406.08756. 2024. Preprint; no conference proceedings version located as of 2026.

[117] Lin Z, Liu G, Wang S, et al. Mist: Efficient Distributed Training of Large Language Models via Memory-Parallelism Co-Optimization [C]. Proceedings of the Twentieth European Conference on Computer Systems (EuroSys). 2025: 1–17.

[118] Benoit A, Robert Y, Thierry E. On the Complexity of Mapping Linear Chain Applications onto Heterogeneous Platforms [J]. Parallel Processing Letters. 2009, 19 (3): 383–397.

作者在学期间取得的学术成果

发表的学术论文

- [1] 第一作者. 会议论文. IEEE ISPA, 2025, CCF-C
